République Tunisienne
Ministère de l'Enseignement Supérieur et de la Recherche Scientifique
Université de Jendouba
Institut Supérieur d'Informatique du Kef

Rapport de Projet de Fin d'Etudes

Présenté à

Institut Supérieur de l'Informatique du Kef

En vue de l'obtention du

DIPLÔME DE MASTERE PROFESSIONNEL EN

Application Web Intelligente

Réalisé par :

Oumaima Amri

Conception et Développement d'un Site Web pour le gestion des agences de location

Soutenue le : 29/11/2025

Devant le jury composé de :

Président :Dr Jeddi Waritheddine

Rapporteur : Dr Louati Aymen

Encadrant : Dr El Kefi Akrem

Année Universitaire : 2024-2025



Dédicace

وَكَانَ فَضْلُ اللَّهِ عَلَيْكَ عَظِيمًا

Je dédie ce travail à tous ceux qui m'ont appris à aimer le savoir et m'ont accompagnée tout au long de mon apprentissage.

À ma famille, pour leur amour, leurs prières et leur soutien sans faille.

À mes professeurs, pour leur patience, leurs conseils et leur dévouement.

À mes amis et collègues, pour leur présence et leur encouragement.

À mon encadrant ***Mr Elkefi Akram*** *pour sa disponibilité, sa bienveillance et son accompagnement constant.*

Et à toutes les personnes qui, de près ou de loin, ont contribué à la réalisation de ce travail, je vous exprime ma sincère reconnaissance et mon profond respect.



Remerciements

Au terme de ce stage, je tiens à exprimer mes sincères remerciements au personnel de ***Société foufusa*** *ainsi qu'à* ***Mr Elkefi Akram*** *pour leur accueil, leur encadrement et leurs précieux conseils tout au long de cette période.*

Je remercie également toute l'équipe technique pour leur aide, leur collaboration et leur disponibilité, qui ont grandement contribué à la réussite de ce travail.

Je souhaite aussi adresser mes remerciements aux ***membres du jury*** *qui m'ont fait l'honneur de leur présence pour évaluer mon travail.*

Enfin, je ne pouvais achever ce projet sans exprimer ma profonde reconnaissance envers ***tous les enseignants de l'Institut Supérieur d'Informatique du Kef*** *pour leur dévouement et leur assistance tout au long de mes études.*

Table des matières

Introduction générale **1**

1 Présentation générale **3**

1.1 Présentation de l'organisme d'accueil 4

1.1.1 Aperçu général de l'entreprise 4

1.1.2 Activités 4

1.2 Étude de l'existant 6

1.2.1 Analyse de l'existant 6

1.2.2 Critique de l'existant 6

1.3 Solutions 7

1.4 Méthodologies de conception 8

1.4.1 Méthode de développement de logiciel adoptée 9

2 Spécification des besoins **12**

2.1 Identification des acteurs 13

2.2 Spécification des besoins 13

2.2.1 Les besoins fonctionnels 14

2.3 Diagramme de cas d'utilisation 17

2.3.1 Définition 17

2.3.2 Diagramme de cas d'utilisation global 17

2.4 Diagramme de cas d'utilisation détaillé 18

2.4.1 Raffinement du cas d'utilisation « S'authentifier et vérification personnelle » . 18

2.4.2 Les diagrammes des cas d'utilisations de l'acteur « Administrateur » 20

2.4.3 Diagrammes des cas d'utilisations de l'acteur « responsable d'agence » 24

2.4.4 Diagrammes des cas d'utilisations de l'acteur « Client» 35

2.4.5 Diagrammes des cas d'utilisations de l'acteur « Visiteur » 45

3 Conception **50**

3.1 Diagramme de classes 51

3.2 Diagramme de séquence 52

3.2.1 Diagrammes de séquence liés au cas d'utilisation «Authentification» 52

3.2.2 Diagrammes de séquence relatifs au cas d'utilisation « Gérer logements » . . 53

3.2.3 Diagrammes de séquence relatifs au cas d'utilisation « Estimer prix par IA » 58

3.2.4 2.4 Diagrammes de séquence relatifs au cas d'utilisation « Réserver un logement » 59

4 Système d'estimation intelligente des prix par machine learning **62**

4.1 Problématique 63

4.2 Objectifs 63

4.3 Préparation des données 63

4.3.1 Dataset 63

4.3.2 Nettoyage et transformation 64

4.3.3 Division des données 66

4.4 Comparaison des Algorithmes 66

4.4.1 Justification de Choix du modèle Gradient Boosting 68

4.5 Configuration du Modèle Gradient Boosting 68

4.5.1 Hyperparamètres Optimisés 68

4.5.2 Justification des Paramètres 69

4.6 Résultats et Performances 69

4.6.1 Évaluation du Modèle 69

4.6.2 Interprétation des Résultats 69

4.6.3 Validation Graphique : Prix Réel vs Prix Prédit 70

4.6.4 Validation sur Cas Réels : Analyse des Prédictions Individuelles 71

4.7 Analyse des Variables Prédictives 72
4.7.1 Variables les Plus Importantes 72
4.7.2 Interprétation des Résultats 72
4.8 Implémentation Technique 73
4.8.1 Architecture logicielle 73
4.8.2 Flux d'Utilisation du Système 73
4.8.3 Interface d'estimation du prix 74
4.9 Points Forts et Limitations 75
4.9.1 Points Forts du Modèle Gradient Boosting 75
4.9.2 Limites et Perspectives d'Amélioration 75

5 Réalisation **77**

5.1 Architecture de l'application 78
5.1.1 Architecture trois tiers 78
5.1.2 Architecture du modèle MVC 79
5.2 Environnement de développement 80
5.2.1 Outils matériels 80
5.2.2 Outils logiciels 80
5.2.3 Langages utilisés 81
5.2.4 Frameworks 82
5.2.5 Bibliothèques 83
5.3 Les interfaces de l'application web 84
5.3.1 Partie publique 84
5.3.2 Partie privée 91

Conclusion générale **95**

Table des figures

1.1 Logo de Foufusa Engineering and Services 4

1.2 Modèle en cascade générique [4] 9

2.1 Diagramme de cas d'utilisation globa 17

2.2 Diagramme de cas d'utilisation raffiné « S'authentifier et vérification personnelle » . 18

2.3 Diagramme de cas d'utilisation raffiné « Consulter les statistiques globales » 20

2.4 Diagramme de cas d'utilisation raffiné « Superviser les agences» 22

2.5 Diagramme de cas d'utilisation raffiné « Superviser les clients » 22

2.6 Diagramme de cas d'utilisation raffiné « Gérer les logements » 24

2.7 Diagramme de cas d'utilisation raffiné « Gérer les réservations » 26

2.8 Diagramme de cas d'utilisation raffiné « Gérer les visites » 27

2.9 Diagramme de cas d'utilisation raffiné « Gérer les négociations » 28

2.10 Diagramme de cas d'utilisation raffiné « Gérer les membres de l'équipe » 29

2.11 Diagramme de cas d'utilisation raffiné « Estimer le prix par IA » 30

2.12 Diagramme de cas d'utilisation raffiné « Gérer les commentaires » 31

2.13 Diagramme de cas d'utilisation raffiné « Échanger des messages » 33

2.14 Diagramme de cas d'utilisation raffiné « Consulter les emails et notifications » 34

2.15 Diagramme de cas d'utilisation raffiné « Réserver un logement » 35

2.16 Diagramme de cas d'utilisation raffiné « Demander une visite » 36

2.17 Diagramme de cas d'utilisation raffiné « Demander une négociation » 37

2.18 Diagramme de cas d'utilisation raffiné « Consulter ses réservations » 39

2.19 Diagramme de cas d'utilisation raffiné « Consulter ses négociations » 40

2.20 Diagramme de cas d'utilisation raffiné « Consulter ses visites » 41

2.21 Diagramme de cas d'utilisation raffiné « Consulter emails et notifications » 41

2.22 Diagramme de cas d'utilisation raffiné « Échanger des messages avec une agence » . 42

2.23 Diagramme de cas d'utilisation raffiné « Rechercher des logements » 43

2.24 Diagramme de cas d'utilisation raffiné « Gérer les commentaires » 45

2.25 Diagramme de cas d'utilisation « Consulter la liste des logements » 46

2.26 Diagramme de cas d'utilisation « Consulter les détails d'un logement » 46

2.27 Diagramme de cas d'utilisation « Consulter la liste des agences » 47

2.28 Diagramme de cas d'utilisation « Consulter les détails d'une agence » 48

2.29 Diagramme de cas d'utilisation « Gérer le profil » 49

3.1 Diagramme de classe 51

3.2 Diagrammes de séquences « Authentification » 52

3.3 Diagramme de séquence « Ajouter logement » 54

3.4 Diagramme de séquence « Modifier logement » 55

3.5 Diagramme de séquence « Rechercher logement » 57

3.6 Diagramme de séquence « Estimer prix par IA » 58

3.7 Diagramme de séquence « réserver un logement » 60

4.1 Analyse complète - Prix Réel vs Prix Prédit 70

4.2 Architecture du Système d'Estimation des Prix **ref6** 74

4.3 Interface d'estimation du prix 75

5.1 Architecture trois tiers [12] 78

5.2 Le modèle MVC [13] 79

5.3 Logo Microsoft Word [14] 80

5.4 Logo StarUML [15] 80

5.5 Logo XAMPP [16] 81

5.6 Logo Visual Studio Code [17] 81

5.7 Logo PHP [18] 81

5.8 Logo HTML5 [19] 82

5.9 Logo CSS3 [20] 82
5.10 Logo JavaScript [21] 82
5.11 Logo Laravel [22] 83
5.12 Logo Bootstrap [23] 83
5.13 Logo jQuery [24] 83
5.14 Logo AJAX [25] 84
5.15 Interface de page d'accueil 85
5.16 Interface de connexion 85
5.17 Interface d'inscription « sélection de rôle » 86
5.18 Formulaire d'inscription Client 86
5.19 Formulaire d'inscription Responsable Agence 87
5.20 Interface « vérification d'email » 88
5.21 Interface « Découvrir les logements » 88
5.22 Interface « Détail du logement » 89
5.23 Interface « Découvrir les agences » 89
5.24 Interface « Réserver un logement » 90
5.25 Interface « Demande de négociation » 90
5.26 Interface page de statistique 91
5.27 Interface « Liste des utilisateurs » 92
5.28 Interface « Ajouter un logement » 92
5.29 Interface « Modifier un logement » 93
5.30 Interface « Messagerie » 93
5.31 Interface « Interface des notifications reçues » 94

Liste des tableaux

1.1 Critique de l'existant 7

2.1 : descriptions textuelles du cas d'utilisation « S'authentifier et vérification personnelle » 19

2.2 Descriptions textuelles du cas d'utilisation « Consulter les statistiques globales » . . 21

2.3 Descriptions textuelles du cas d'utilisation « Superviser les agences et les clients » . . 23

2.4 Description textuelle du cas d'utilisation « Gérer les logements » 25

2.5 Description textuelle du cas d'utilisation « Gérer les réservations » 27

2.6 Description textuelle du cas d'utilisation « Gérer les visites » 28

2.7 Description textuelle du cas d'utilisation « Gérer les négociations » 29

2.8 Description textuelle du cas d'utilisation « Gérer les membres de l'équipe » 30

2.9 Description textuelle du cas d'utilisation « Estimer le prix par IA » 31

2.10 Description textuelle du cas d'utilisation « Gérer les commentaires » 32

2.11 Description textuelle du cas d'utilisation « Échanger des messages » 34

2.12 Description textuelle du cas d'utilisation « Consulter les emails et notifications » . . 35

2.13 Descriptions textuelles du cas d'utilisation « Réserver un logement » 36

2.14 Descriptions textuelles du cas d'utilisation « Demander une visite » 37

2.15 Descriptions textuelles du cas d'utilisation « Demander une négociation » 38

2.16 Descriptions textuelles du cas d'utilisation « Consulter ses réservations » 39

2.17 Descriptions textuelles du cas d'utilisation « Consulter ses négociations » 40

2.18 Descriptions textuelles du cas d'utilisation « Consulter ses visites » 41

2.19 Descriptions textuelles du cas d'utilisation « Consulter emails et notifications » . . . 42

2.20 Descriptions textuelles du cas d'utilisation « Échanger des messages avec une agence » 43

2.21 Descriptions textuelles du cas d'utilisation « Rechercher des logements » 44

2.22 Descriptions textuelles du cas d'utilisation « Gérer les commentaires » 45

2.23 Descriptions textuelles du cas d'utilisation « Consulter la liste des logements » . . . 46

2.24 Descriptions textuelles du cas d'utilisation « Consulter les détails d'un logement » . 47

2.25 Descriptions textuelles du cas d'utilisation « Consulter la liste des agences » 48

2.26 Descriptions textuelles du cas d'utilisation « Consulter les détails d'une agence » . . 48

2.27 Descriptions textuelles du cas d'utilisation « Gérer le profil » 49

4.1 Division du dataset pour l'apprentissage supervisé 66

4.2 Comparaison des performances des quatre modèles 67

4.3 Hyperparamètres du modèle Gradient Boosting 68

4.4 Évaluation du modèle sur les ensembles d'entraînement et de test 69

4.5 Validation du modèle sur 7 cas réels de logements 71

4.6 Classement des variables influençant le prix de location 72

5.1 Description de l'environnement matériel 80

Introduction générale

À l'ère numérique, la recherche et la gestion de logements nécessitent des solutions rapides, intelligentes et interactives. Les utilisateurs attendent des plateformes efficaces et personnalisées, tandis que les agences ont besoin d'outils performants pour optimiser leurs opérations et améliorer l'expérience client.

En Tunisie, le secteur immobilier souffre d'une double problématique. D'une part, une fragmentation importante : chaque agence dispose de son propre site web ou canal de communication, obligeant les clients à naviguer entre de multiples plateformes pour comparer les offres. D'autre part, un manque d'innovation technologique : les solutions existantes restent basiques, sans exploitation de l'intelligence artificielle ni de fonctionnalités avancées d'interaction et de gestion.

De plus, bien que de nombreux sites immobiliers proposent à la fois la vente et la location, le marché locatif représente un volume de transactions nettement supérieur au marché de la vente. Ce constat révèle un besoin spécifique : une plateforme dédiée exclusivement à la location, offrant une expérience optimisée et des fonctionnalités adaptées aux particularités de ce marché dynamique.

Face à ces enjeux, nous avons développé une application web innovante spécialisée dans la location de logements pour les agences immobilières. Contrairement aux plateformes généralistes, notre solution se concentre uniquement sur le marché locatif, permettant ainsi une approche ciblée et simplifiée. Elle vise à réunir les agences dans un écosystème unifié, offrant aux clients un accès centralisé à l'ensemble de l'offre locative.

La plateforme intègre des fonctionnalités avancées telles que la recherche dynamique, l'estimation intelligente des loyers par intelligence artificielle, la messagerie en temps réel, un système de négociation, ainsi que la gestion des visites et des réservations. Cette application constitue un outil stratégique pour les agences, leur permettant de digitaliser leur activité locative, tout en offrant aux utilisateurs une expérience complète et innovante.

Le présent rapport s'appuie sur la méthodologie en cascade et se décompose en cinq chapitres :

- Chapitre I : Étude préalable — analyse du contexte, de l'existant et de la méthode adoptée.
- Chapitre II : Spécification des besoins — définition des besoins fonctionnels et non fonctionnels.
- Chapitre III : Conception — modélisation et conception détaillée du système.

• Chapitre IV :Estimation intelligente des loyers par intelligence artificielle — description du modèle de machine learning et de son fonctionnement.

Enfin, la conclusion générale présente le bilan du travail réalisé et les perspectives d'évolution de la plateforme.

• Chapitre V : Réalisation — présentation des technologies utilisées et des interfaces développées.

Chapitre 1

Présentation générale

Plan

1 **Présentation de l'organisme d'accueil** 4

2 **Étude de l'existant** 6

3 **Solutions** 7

4 **Méthodologies de conception** 8

Introduction

Ce chapitre est entièrement consacré à une vue d'ensemble de l'entreprise Foufusa Engineering and Services, une société spécialisée dans le développement informatique et les services automobiles, où nous avons effectué notre stage. Nous y décrivons le projet sur lequel nous avons travaillé, en mettant en évidence son contexte et ses objectifs. Par la suite, nous analysons les solutions déjà existantes, identifions leurs limites et proposons une approche adaptée à nos besoins. Enfin, nous expliquons la méthode de modélisation retenue pour la réalisation du projet.

1.1 Présentation de l'organisme d'accueil

1.1.1 Aperçu général de l'entreprise

Foufusa Engineering and Services est une entreprise localisée au cœur du centre-ville du Kef en Tunisie. Elle a été fondée en 2019 et est constituée de 5 membres. La société propose des services complets de développement web ainsi que des services liés à l'automobile, utilise des technologies avancées, qui sont livrés par une équipe de professionnels qualifiés. Parmi leurs services, on retrouve la sécurité fonctionnelle, la sécurité informatique, la conception de systèmes intégrés, ainsi que la création d'applications web ergonomiques et faciles d'utilisation.

La figure 1.1 représente le logo de Foufusa Engineering and Services.



Figure 1.1 : Logo de Foufusa Engineering and Services

1.1.2 Activités

Les services de Foufusa font la différence sur le marché, l'équipe de Foufusa se concentre sur l'excellence dans tous les secteurs.

— **Développement Web** La société propose des solutions complètes de création de plateformes web, d'applications pour smartphones et de marketing numérique. Pour les sites web, Foufusa Engineering and Services utilise les dernières technologies pour concevoir et développer des sites web modernes, performants et conviviaux.

— **Sécurité fonctionnelle** L'entreprise Foufusa détient une expertise poussée en matière de sécurité fonctionnelle. L'entreprise offre des solutions intégrales pour accompagner les sociétés du secteur automobile à se conformer aux normes de sécurité et de fiabilité les plus sévères concernant leurs systèmes critiques.

Les prestations de sécurité fonctionnelle proposées par Foufusa englobent des analyses de risques, des contrôles de conformité, des essais et des certifications afin d'assurer que les systèmes essentiels des véhicules respectent les standards de protection les plus stricts. Les spécialistes de la sécurité fonctionnelle de Foufusa collaborent étroitement avec les clients pour comprendre les exigences de sécurité de chaque projet et pour élaborer des solutions personnalisées qui correspondent à ces exigences.

Par ailleurs, Foufusa offre aussi des prestations en sécurité informatique visant à protéger les systèmes automobiles des menaces cybernétiques. Les prestations en cybersécurité de Foufusa incluent des audits de sécurité, des analyses de vulnérabilité, de même que des solutions de maîtrise des risques et de respect des réglementations permettant aux clients de sécuriser leurs systèmes face aux menaces informatiques.

Dans l'ensemble, la sécurité fonctionnelle et la cybersécurité sont des domaines clés de l'expertise de Foufusa, et l'entreprise collabore activement avec ses clients pour garantir que leurs systèmes critiques sont fiables, sécurisés et respectent les standards de protection les plus élevés.

— **Développement de systèmes embarqués** Le développement de systèmes embarqués constitue également un domaine de compétence majeur de l'entreprise Foufusa. Celle-ci fournit des solutions complètes destinées à la création de systèmes informatiques intégrés à divers appareils ou machines afin d'exécuter des fonctions précises.

L'équipe d'ingénierie de Foufusa utilise des outils technologiques avancés afin de créer des systèmes embarqués performants et sécurisés. Ils collaborent en liaison directe avec les clients afin de définir leurs exigences et de mettre au point des solutions adaptées à leurs demandes.

Les prestations de Foufusa dans ce domaine regroupent la création et le développement de dispositifs électroniques, l'écriture de code pour microcontrôleurs, la mise en place de communications

sans fil, ainsi que la réalisation de circuits imprimés.

1.2 Étude de l'existant

Dans cette section, nous présenterons les principales solutions actuellement disponibles sur le marché locatif tunisien, puis nous procéderons à leur analyse afin d'évaluer leurs fonctionnalités, leurs limites et les perspectives d'amélioration qu'elles offrent.

1.2.1 Analyse de l'existant

Le marché tunisien compte aujourd'hui de nombreuses plateformes immobilières, mais celles-ci restent pour la plupart généralistes : elles regroupent à la fois la vente, la location et parfois d'autres services. Cette approche élargie ne répond pas toujours aux attentes des utilisateurs qui recherchent exclusivement un logement à louer. En effet, il n'existe pas de solution réellement dédiée à la location, pensée pour accompagner spécifiquement les besoins des clients souhaitant louer un bien.

Par ailleurs, l'innovation technologique, notamment l'intégration de fonctionnalités basées sur l'intelligence artificielle, demeure limitée dans les solutions actuelles. Sans affirmer leur absence totale, ces technologies ne sont ni mises en avant ni exploitées de manière à faciliter des aspects essentiels comme l'évaluation des biens, l'aide à la recherche ou l'accompagnement des décisions.

Enfin, les services proposés sont souvent dispersés d'une plateforme à une autre. Peu d'outils offrent une centralisation complète regroupant toutes les fonctionnalités nécessaires à la fois pour les utilisateurs et pour les agences : gestion des annonces, interaction, suivi, processus administratif et accompagnement intelligent. Cette absence de centralisation rend le parcours locatif moins fluide et moins efficace.

Ces constats révèlent un besoin clair : une application spécialisée, centralisée et enrichie par des fonctionnalités intelligentes, capable de répondre de manière ciblée aux exigences du marché locatif tunisien [1].

1.2.2 Critique de l'existant

L'analyse des plateformes tunisiennes dédiées à la location de logements met en évidence que, malgré leurs avantages, elles demeurent limitées par plusieurs insuffisances structurelles. Ces lacunes affectent non seulement la qualité de l'expérience utilisateur, mais aussi l'efficacité opérationnelle

des agences immobilières.

Nom du site	Points forts	Points faibles
Menzili.tn [2]	— Interface simple et intuitive. — Large choix d'annonces immobilières pour la vente et la location. — Bon référencement et bonne visibilité.	— Absence de fonctionnalités essentielles telles que la visite virtuelle, la négociation ou la réservation en ligne. — Faible interaction entre utilisateurs et agences. — Expérience limitée à une simple consultation d'annonces.
Immobiliere.tn [3]	— Plateforme spécialisée dans les annonces publiées par des agences. — Informations détaillées et navigation fluide. — Présentation claire et structurée des biens. — Filtres de recherche variés : prix, type de bien, localisation.	— Peu de services complémentaires (chat, simulation de crédit, suivi personnalisé). — Absence d'outils avancés comme l'estimation intelligente ou la recommandation IA. — Manque de centralisation complète des données entre les différentes agences.

Tableau 1.1 : Critique de l'existant

1.3 Solutions

Cette solution a pour objectif de moderniser et d'optimiser le processus de location en réunissant, au sein d'une seule plateforme, toutes les fonctionnalités essentielles pour les utilisateurs et les

agences. Elle est spécifiquement conçue pour le marché locatif, afin de simplifier la recherche, la réservation et la gestion des logements tout en offrant une expérience centralisée, efficace et intelligente.

La plateforme vise à combler les lacunes identifiées dans les sites existants [2], [3] en proposant une expérience complète et centralisée. Elle regroupe plusieurs agences au sein d'un même espace numérique, permettant aux utilisateurs d'accéder facilement à un large choix de logements sans avoir à consulter plusieurs sites.

L'application se distingue par son système de gestion des visites, où chaque utilisateur peut planifier un créneau adapté à ses disponibilités, facilitant ainsi la coordination avec les agences. Elle intègre également un module de négociation en ligne, favorisant des échanges transparents et rapides, ainsi qu'un système de réservation et de paiement sécurisé en ligne, simplifiant les démarches et offrant aux utilisateurs la possibilité de régler leurs réservations directement depuis la plateforme.

Un des aspects les plus innovants de la plateforme réside dans l'intégration d'un outil d'estimation de prix intelligent basé sur l'Intelligence Artificielle. Cet outil permet aux agences de gagner un temps considérable, en générant automatiquement des estimations précises, tout en leur laissant se concentrer sur les interactions avec les clients et la gestion stratégique plutôt que sur des tâches répétitives.

Enfin, la messagerie instantanée accompagnée de notifications en temps réel et par e-mail assure une communication fluide entre utilisateurs et agences. L'application se positionne ainsi comme une solution moderne et performante, alliant simplicité, efficacité, innovation technologique et paiement en ligne sécurisé pour répondre pleinement aux besoins du marché locatif.

1.4 Méthodologies de conception

Le choix d'une méthodologie de développement est un élément clé pour garantir la réussite du projet et le respect des délais. Une approche structurée permet d'organiser les différentes phases de travail, d'anticiper les obstacles et de coordonner efficacement les efforts de l'équipe. Elle fournit également des pratiques et des outils adaptés, favorisant la collaboration entre les membres et les parties prenantes. Ainsi, une méthodologie bien définie contribue directement à la qualité du projet, en assurant des livrables conformes aux attentes dans le respect des ressources disponibles [4].

1.4.1 Méthode de développement de logiciel adoptée

Pour ce projet, une méthodologie rigoureuse a été adoptée afin d'assurer un développement structuré, prévisible et maîtrisé. Le modèle en cascade a été retenu comme référence, en raison de la stabilité et de la définition exhaustive des fonctionnalités dès la phase initiale. Cette approche linéaire et progressive permet de valider rigoureusement chaque étape avant de passer à la suivante, garantissant ainsi la cohérence architecturale et la qualité des livrables. Elle facilite également la planification optimale des ressources et la maîtrise des délais, tout en réduisant considérablement les risques liés aux erreurs structurelles ou aux modifications tardives.

L'ensemble des fonctionnalités du système étant clairement spécifié et stable dès la conception, le modèle en cascade s'impose comme le choix méthodologique optimal. Sa simplicité organisationnelle et sa structure séquentielle garantissent une traçabilité complète du processus de développement et facilitent le contrôle rigoureux de l'avancement, tout en assurant la conformité documentaire requise dans un contexte académique.

La figure 1.2 illustre les différentes étapes du modèle en cascade appliquées à ce projet, chacune étant développée pour assurer une maîtrise complète du processus de développement.

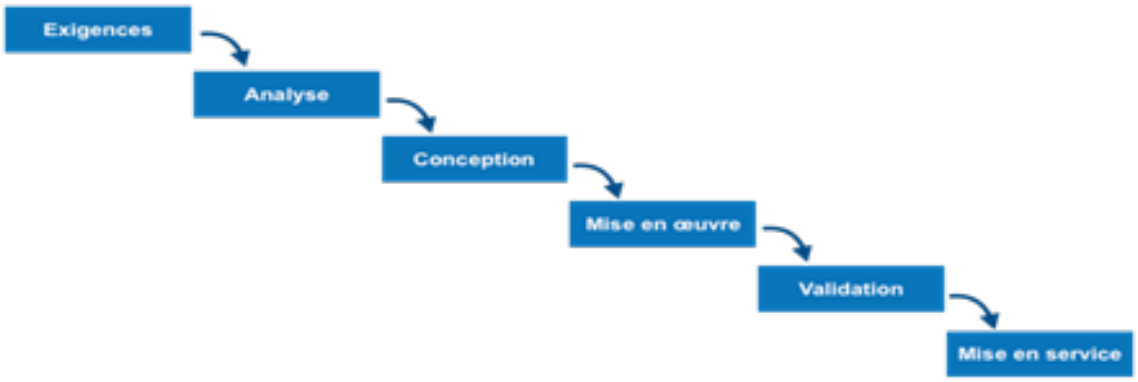


Figure 1.2 : Modèle en cascade générique [4]

Le modèle en cascade se compose de plusieurs phases successives et interdépendantes, permettant une maîtrise complète du processus de développement. Pour ce projet, chaque phase a été développée comme suit :

— **Exigences :** Identifier et formaliser les besoins des utilisateurs et des parties prenantes. Définir les objectifs, les fonctionnalités attendues et les contraintes techniques, aboutissant à un cahier des charges clair servant de référence pour l'ensemble du projet.

— **Analyse :** Étudier en détail les exigences recueillies et les traduire en spécifications techniques. Cette étape inclut l'analyse des processus métiers, l'identification des flux de données et la préparation d'un modèle conceptuel guidant la conception du système.

— **Conception :** Définir l'architecture logicielle et les composants du système. Concevoir les interfaces utilisateur, modéliser la base de données et planifier les interactions entre les modules, assurant cohérence technique et faisabilité.

— **Mise en œuvre :** Programmer et intégrer les modules et fonctionnalités du logiciel, incluant notamment la réservation, la négociation et l'estimation automatique des prix via intelligence artificielle.

— **Validation :** Vérifier que le système répond aux exigences définies. Tester les fonctionnalités, évaluer les performances et la sécurité, et corriger les anomalies détectées pour garantir un logiciel fiable et stable.

— **Mise en service :** Déployer l'application dans son environnement opérationnel, former les utilisateurs si nécessaire, et assurer un suivi initial pour garantir le bon fonctionnement et la disponibilité de l'application.

Ainsi, le modèle en cascade fournit un cadre rigoureux et structuré, permettant de suivre chaque étape du développement de manière maîtrisée, tout en assurant la qualité, la cohérence et la conformité du produit final aux besoins des utilisateurs et des agences.

Conclusion

En conclusion de ce chapitre, nous avons présenté le cadre global de l'étude et examiné l'existant afin de définir clairement les objectifs du projet. Cette analyse a permis d'identifier les opportunités d'amélioration et de poser les bases d'une conception alignée sur les besoins réels. Le chapitre suivant

exposera la méthodologie de développement retenue ainsi que les différentes étapes de réalisation de la solution envisagée.

Chapitre 2

Spécification des besoins

Plan

1 **Identification des acteurs** 13

2 **Spécification des besoins** 13

3 **Diagramme de cas d'utilisation** 17

4 **Diagramme de cas d'utilisation détaillé** 18

Introduction

Au cours de ce chapitre, nous introduisons initialement une vue d'ensemble de l'application, avant de détailler ses fonctionnalités. Nous définissons les besoins fonctionnels en utilisant les diagrammes de cas d'utilisation, permettant d'illustrer clairement les différentes actions que le système doit pouvoir réaliser. Par ailleurs, nous exposons les besoins non fonctionnels, en indiquant les contraintes et exigences auxquelles le système doit se conformer.

2.1 Identification des acteurs

Un système interactif efficace doit garantir une utilisation fluide et intuitive pour ses utilisateurs. On appelle acteur toute entité externe interagissant avec le système au moyen de ses interfaces. Dans le contexte de ce projet, les acteurs clés du système sont présentés comme suit :

— **Administrateur :** L'administrateur possède des privilèges étendus et la possibilité d'utiliser toutes les fonctions disponibles sur la plateforme. Il est chargé de superviser toutes les activités du système, d'assurer la maintenance, de superviser les utilisateurs et de veiller au bon fonctionnement du système.

— **Responsable-Agence :**L'agence gère et supervise l'ensemble des logements. Elle peut ajouter et modifier des fiches de logements, gérer les visites, les messages, les réservations et suivre l'ensemble de ses annonces sur la plateforme.

— **Visiteur :**Le visiteur peut explorer le site, consulter les logements disponibles, envoyer des demandes de réservation ou de visite et initier des négociations, sans avoir besoin de créer un compte.

— **Client :**Le client correspond à un visiteur inscrit sur la plateforme. Il bénéficie de fonctionnalités supplémentaires, telles que la gestion de ses réservations, la consultation de l'historique de ses interactions et la sécurisation de ses données personnelles.

2.2 Spécification des besoins

L'étude des besoins et leur formalisation forment une phase importante dans la création d'une application. Elles permettent de définir le périmètre fonctionnel du projet, de clarifier les attentes des utilisateurs et d'orienter les choix techniques. Dans le cadre de cette application, dédiée à la

location de logements, plusieurs types d'acteurs interviennent : l'administrateur, les responsables d'agence et les clients. Chaque acteur dispose de rôles et de fonctionnalités spécifiques.

2.2.1 Les besoins fonctionnels

a) Administrateur

L'administrateur assure la gestion globale et le contrôle de la plateforme. Il a pour mission de garantir son bon fonctionnement et de superviser les activités des différents utilisateurs. Ses principales responsabilités sont :

— **Authentification** : accès sécurisé via e-mail et mot de passe.

— **Gestion des agences** : validation, modification, suspension ou suppression des comptes agences.

— **Gestion des utilisateurs** : ajout, suppression, recherche, blocage ou modification des profils clients.

— **Gestion des annonces** : surveillance des annonces publiées et bloquer les annonces non conformes.

— **Suivi de l'activité** : consultation du suivi global des réservations, visites et négociations.

— **Gestion des contacts** : consultation et traitement des demandes envoyées par les clients ou les agences.

— **Statistiques** : visualisation des performances générales de la plateforme (nombre d'annonces, réservations, utilisateurs actifs).

— **Sécurité** : blocage d'un utilisateur ou d'une agence en cas de non-respect des règles.

b) Responsable d'agence

Le responsable d'agence s'inscrit de manière autonome via un formulaire de registre. Une fois inscrit, il devient gestionnaire des annonces de son agence et assure la relation directe avec les clients. Ses principales responsabilités sont :

— **Authentification** : connexion sécurisée à son compte via identifiants.

— **Gestion des annonces** : ajout, modification, suppression et publication des logements de son agence.

— **Gestion des visites** : planification, validation ou annulation des visites demandées par les clients.

— **Gestion des négociations** : suivi des discussions, réponses aux propositions et gestion des contre-offres.

— **Gestion des réservations** : confirmation, annulation ou consultation des réservations reçues.

— **Messagerie** : communication directe avec les clients et l'administrateur via un système de messagerie interne.

— **Notifications** : réception d'alertes concernant les nouvelles réservations, visites ou négociations.

— **Estimation des prix via IA** : obtention automatique d'une estimation du prix d'un logement lors de la publication d'une annonce à partir des informations et images fournies.

— **Suivi d'activité** : consultation des statistiques locales (nombre de vues, demandes et réservations).

c) Client

Le client correspond à un visiteur inscrit sur la plateforme. Il dispose d'un espace personnel lui permettant d'interagir directement avec les agences et de gérer ses demandes. Ses principales fonctionnalités sont :

— **Authentification** : accès sécurisé à son espace personnel via identifiants.

— **Consultation des annonces** : navigation et recherche de logements selon plusieurs critères (localisation, prix, type de bien, surface, etc.).

— **Demandes** : envoi de demandes de réservation, de visite ou de négociation concernant un logement.

— **Gestion des négociations** : suivi de l'état de ses négociations en cours avec les agences.

— **Dashboard personnel** : visualisation de l'état de ses actions (réservations confirmées ou refusées, visites validées ou annulées, négociations ouvertes).

— **Profil** : consultation et modification de ses informations personnelles.

— **Messagerie** : communication directe avec une agence ou l'administrateur via un système de messagerie interne.

— **Historique** : consultation de ses réservations passées, visites effectuées et interactions précédentes avec les agences.

d) Visiteur

Le visiteur correspond à un utilisateur non enregistré disposant d'un accès partiel aux données de la plateforme. Ses fonctionnalités sont limitées mais essentielles pour lui permettre de découvrir les services proposés :

— **Consultation des annonces** : visualisation des logements disponibles sans authentification.

— **Recherche de logements** : possibilité de filtrer les biens selon des critères simples (localisation, prix, type de bien).

— **Première demande** : possibilité d'envoyer une demande initiale de visite ou de réservation, nécessitant une inscription pour finaliser l'action.

Cette spécification des besoins fonctionnels constitue la base du développement de l'application. Elle définit clairement les rôles et responsabilités de chaque acteur afin de garantir une expérience utilisateur fluide et une gestion optimale de la plateforme.

2.2.1.1 Les besoins non fonctionnels

En plus des fonctionnalités offertes aux différents utilisateurs, l'application doit respecter un ensemble de contraintes non fonctionnelles afin d'assurer sa performance, sa sécurité et sa pérennité.

- **Sécurité** :L'application assure la protection des informations par un contrôle d'accès sécurisé et une gestion des droits adaptée à chaque profil utilisateur. Elle intègre des dispositifs de sécurité pour préserver l'intégrité du système et des données.
- **Performance** : L'application est optimisée pour garantir des temps de réponse rapides grâce à des requêtes efficaces et une architecture performante. Elle est conçue pour gérer plusieurs utilisateurs simultanément
- **Accessibilité et ergonomie** L'application propose une interface intuitive et facile à utiliser, adaptée à tous les profils d'utilisateurs. Elle est accessible sur tous les supports (ordinateurs, tablettes, smartphones) grâce à son design responsive.
- **Fiabilité et disponibilité** : Elle doit être protégée contre l'accès non autorisé et illégal en limitant l'accès ouvert.
- **Évolutivité** : L'expérience utilisateur doit être agréable et sans heurts, avec une interface facile à utiliser avec des interactions sans complexité pour tous les types d'utilisateurs (les médecins, les secrétaires médicaux et les patients).

Avec ces besoins non fonctionnels, l'application garantit non seulement des fonctionnalités adaptées aux attentes des utilisateurs, mais également un haut niveau de sécurité, de performance et de fiabilité.

2.3 Diagramme de cas d'utilisation

2.3.1 Définition

Les interactions utilisateur-système sont modélisées à travers le diagramme de cas d'utilisation. Il constitue un outil fondamental du langage UML, permettant de visualiser les différentes fonctionnalités offertes ainsi que les relations entre les acteurs et ces fonctionnalités. Ce diagramme adopte la perspective de l'utilisateur afin d'identifier, structurer et formaliser ses besoins. Il aide également à délimiter le périmètre fonctionnel du système et à préciser ses objectifs.

2.3.2 Diagramme de cas d'utilisation global

On observe dans la figure 2.1 le diagramme complet des cas d'utilisation qui décrit les relations entre l'administrateur, le visiteur, le client et le responsable d'agence.

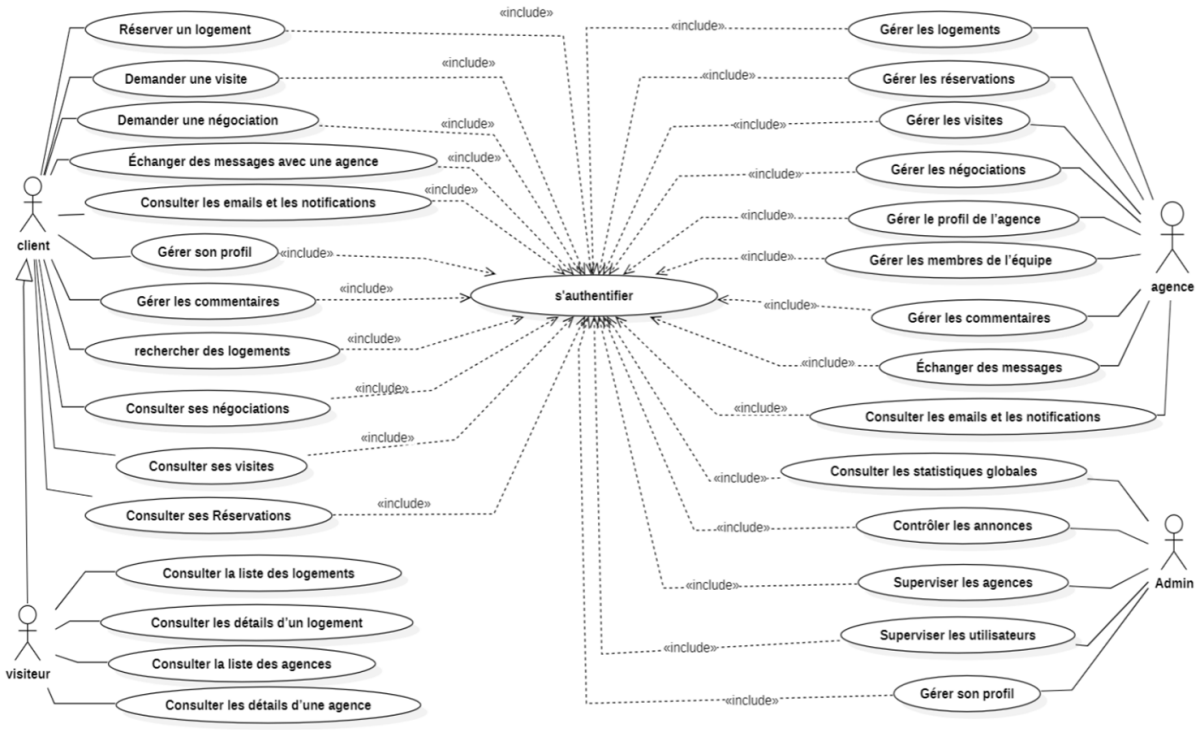


Figure 2.1 : Diagramme de cas d'utilisation globa

2.4 Diagramme de cas d'utilisation détaillé

Cette représentation approfondie affine le diagramme de cas d'utilisation classique. Il illustre de manière plus précise les échanges entre les utilisateurs et le système, en se concentrant sur les fonctions spécifiques de l'application de location de logements de courte durée. Ce diagramme permet de mieux définir et clarifier les exigences fonctionnelles associées à chaque fonctionnalité du système.

2.4.1 Raffinement du cas d'utilisation « S'authentifier et vérification personnelle »

Le processus d'authentification est essentiel pour garantir un accès sécurisé aux fonctionnalités de l'application. Il nécessite la saisie de l'email et du mot de passe de l'utilisateur, et inclut une vérification par e-mail afin de confirmer l'identité, applicable à tous les types d'utilisateurs.

Dans la figure 2.2, nous illustrons l'affinement du processus d'authentification et de la vérification personnelle, permettant un accès sécurisé aux espaces et pages de l'application pour chaque utilisateur.

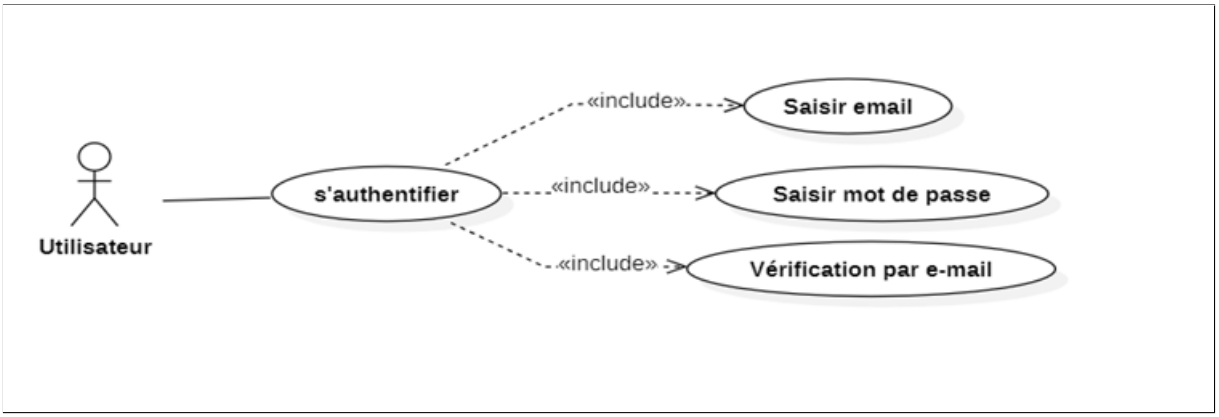


Figure 2.2 : Diagramme de cas d'utilisation raffiné « S'authentifier et vérification personnelle »

Description textuelle du cas d'utilisation «S'authentifier et vérification personnelle»

Titre	Connexion (login) et vérification par e-mail
Acteur	Administrateur
Résumé	L'utilisateur se connecte en entrant son adresse e-mail et son mot de passe sur l'interface dédiée. Si les identifiants sont valides, il est connecté techniquement. Toutefois, il ne peut pas accéder aux fonctionnalités de l'application tant qu'il n'a pas validé le lien reçu par e-mail. L'authentification complète repose donc sur deux étapes : connexion (login) et vérification par e-mail. Cette double étape est indispensable pour accéder à son espace personnel ou effectuer des actions dans la plateforme.
Pré condition(s)	- L'utilisateur possède un compte inscrit. - L'utilisateur dispose d'une adresse e-mail valide enregistrée.
Post condition(s)	- L'utilisateur est complètement authentifié (login + e-mail vérifié) et peut accéder à toutes les fonctionnalités, y compris son espace et la réservation. - Si l'e-mail n'est pas validé, l'utilisateur reste limité malgré la connexion.
Scénario nominal	1- Le client clique sur « Connexion ». 2- L'interface de connexion est présentée à l'utilisateur. 3- Le client complète les champs email et mot de passe. 4- La vérification des identifiants est effectuée. 5- L'application vérifie si l'adresse e-mail est validée. • Si oui → Le client est redirigé vers son tableau de bord et peut accéder à son espace et aux fonctionnalités de la plateforme. • Si non → L'utilisateur reste bloqué tant que la vérification n'est pas réalisée. 6- L'utilisateur consulte sa boîte mail et clique sur le lien de vérification. 7- Le système confirme la vérification et débloque l'accès.
Scénario(s) alternatif(s)	**Identifiants invalides (e-mail ou mot de passe incorrect)** : - L'application renvoie un message d'erreur. - Les champs erronés sont mis en évidence. - L'utilisateur est invité à ressaisir ses informations. **Connexion avec e-mail non vérifié** : - L'utilisateur peut se connecter techniquement mais ne peut pas accéder à son espace ni effectuer des actions (comme une réservation). - Lorsqu'il tente une action, un message d'erreur s'affiche : « Veuillez valider votre e-mail pour continuer. » - Le système propose un renvoi du lien de vérification. **Lien expiré** : - Le système affiche une erreur. - L'utilisateur peut demander un nouvel envoi du lien.

Tableau 2.1 : : descriptions textuelles du cas d'utilisation « S'authentifier et vérification personnelle »

2.4.2 Les diagrammes des cas d'utilisations de l'acteur « Administrateur »

2.4.2.1 Raffinement du cas d'utilisation « Consulter les statistiques globales »

Le diagramme présenté dans la figure 2.3 expose les étapes du cas d'utilisation permettant à l'administrateur de visualiser les statistiques globales de la plateforme.

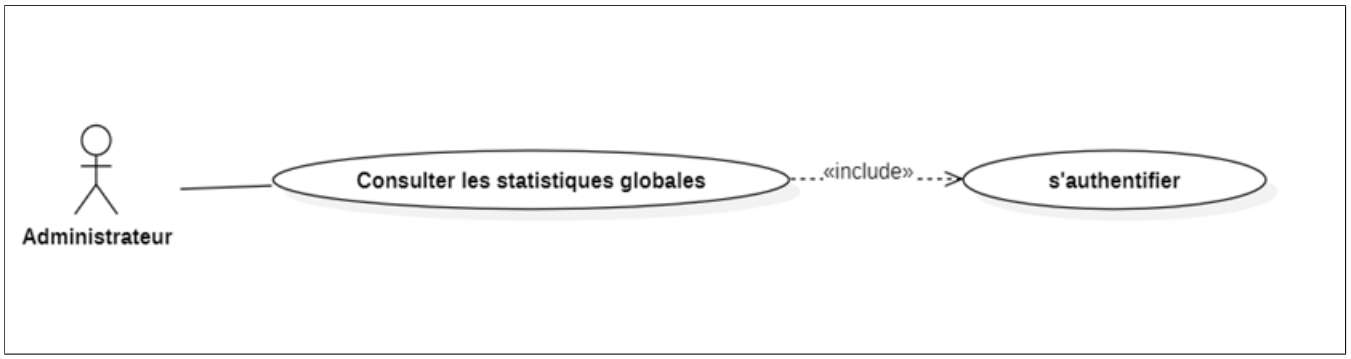


Figure 2.3 : Diagramme de cas d'utilisation raffiné « Consulter les statistiques globales »

Descriptions textuelles du cas d'utilisation « Consulter les statistiques globales »

Titre	Consulter les statistiques globales
Acteur	Administrateur
Résumé	Ce processus offre à l'administrateur la possibilité de visualiser un tableau de bord regroupant les données clés de la plateforme de location de logements. Les statistiques offrent une vue d'ensemble sur les utilisateurs, les agences et les logements. Elles incluent également des graphiques d'évolution, des indicateurs de performance et des listes récentes pour suivre l'activité globale. Ce processus garantit une visibilité claire et synthétique permettant à l'administrateur de superviser l'ensemble du système.
Pré condition(s)	- L'administrateur est connecté et authentifié. - Les données (utilisateurs, agences, logements, réservations) sont présentes dans la base.
Post condition(s)	- Les statistiques sont affichées de manière visuelle (chiffres, graphiques, listes récentes). - L'administrateur peut suivre et analyser l'activité de la plateforme.
Scénario nominal	1- L'administrateur se connecte au système. 2- Il ouvre l'interface «Consultation des statistiques». 3- Il peut visualiser des statistiques sur différents domaines, tels que : • Utilisateurs (nombre d'inscriptions, évolution, etc.) • Agences (actives, inactives, répartition) • Logements (disponibles, réservés, taux d'occupation)
Scénario(s) alternatif(s)	- Pas de données disponibles : L'interface montre un message d'information : « Aucune donnée statistique disponible. » - Erreur technique lors du chargement.

Tableau 2.2 : Descriptions textuelles du cas d'utilisation « Consulter les statistiques globales »

2.4.2.2 Raffinement du cas d'utilisation « Superviser les agences »

Le diagramme présenté dans la figure 2.4 illustre les étapes du cas d'utilisation pour superviser les agences.

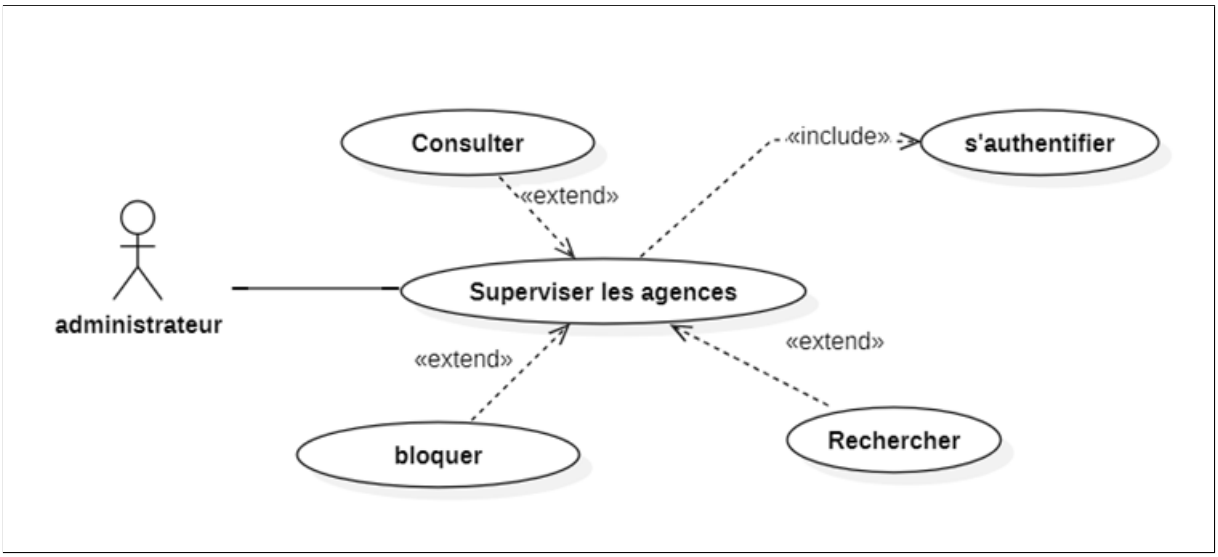


Figure 2.4 : Diagramme de cas d'utilisation raffiné « Superviser les agences»

2.4.2.3 Raffinement du cas d'utilisation « Superviser les clients »

Ce diagramme, présenté dans la figure2.5, illustre les étapes du cas d'utilisation pour superviser les clients.

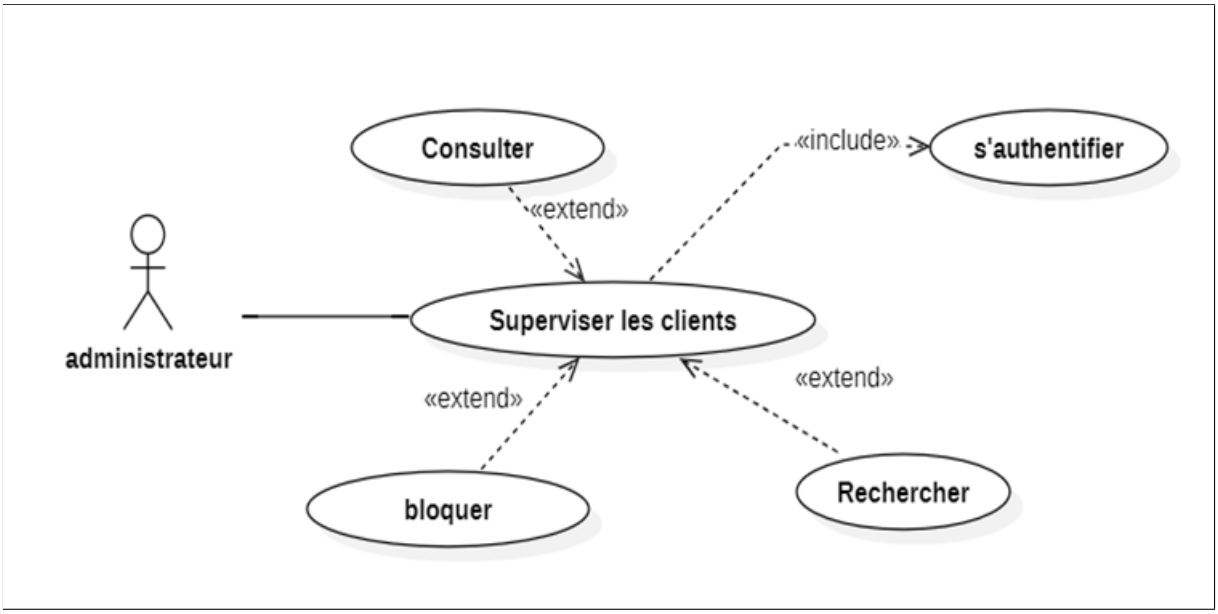


Figure 2.5 : Diagramme de cas d'utilisation raffiné « Superviser les clients »

Descriptions textuelles du cas d'utilisation « Superviser les agences et superviser les clients

Titre	Superviser les utilisateurs et agences
Acteur	Administrateur
Résumé	L'administrateur consulte et recherche dans les listes des utilisateurs et des agences pour vérifier leurs informations et supprimer ceux qui ne respectent pas les règles ou publient des contenus inappropriés.
Pré condition(s)	- L'administrateur est authentifié et peut accéder aux listes «Utilisateurs» et «Agences» depuis la barre de navigation de l'espace admin.
Post condition(s)	- Les utilisateurs ou agences supprimés n'apparaissent plus dans la liste et perdent leur accès à la plateforme.
Scénario nominal	1- L'administrateur s'authentifie et accède à son tableau de bord. 2- Il ouvre la liste des utilisateurs ou des agences depuis la barre de navigation. 3- Il consulte les informations de base de chaque compte (nom, e-mail, autres informations enregistrées). 4- Lorsqu'il identifie un comportement ou des informations non conformes, il utilise la fonction de recherche pour retrouver le compte exact. 5- Il procède au blocage du compte concerné. 6- Le système met à jour la liste et bloque le compte supprimé.
Scénario(s) alternatif(s)	1- Si le compte n'est pas trouvé → le système indique « Aucun résultat ». 2- En cas d'échec dû à un problème technique, un message s'affiche et le compte demeure inchangé.

Tableau 2.3 : Descriptions textuelles du cas d'utilisation « Superviser les agences et les clients »

2.4.3 Diagrammes des cas d'utilisations de l'acteur « responsable d'agence »

2.4.3.1 Raffinement du cas d'utilisation « Gérer les logements »

Le diagramme présenté dans la figure 2.6 illustre les étapes du cas d'utilisation permettant au responsable d'agence de gérer les logements.

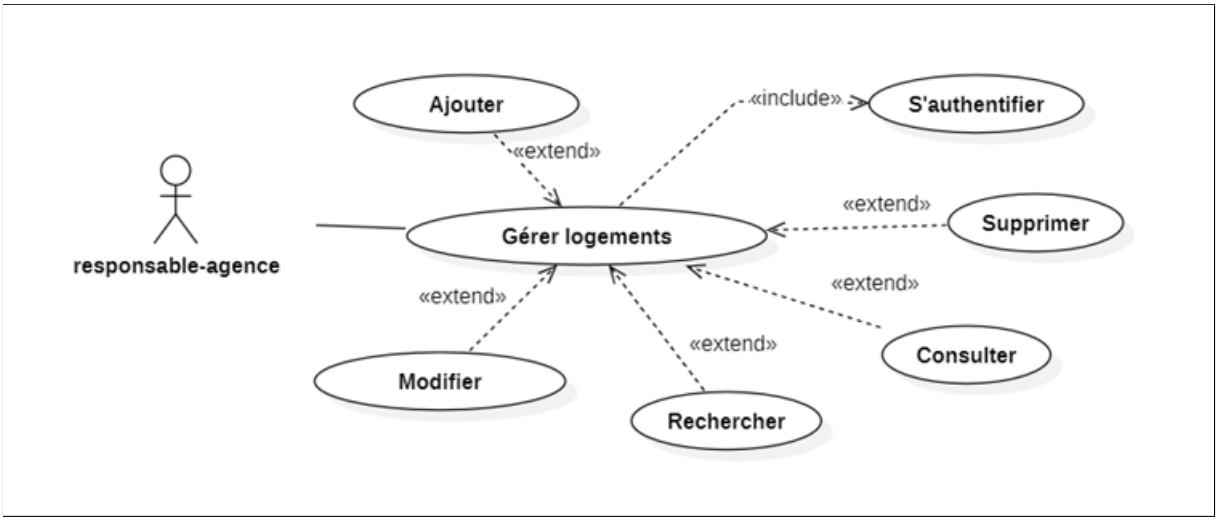


Figure 2.6 : Diagramme de cas d'utilisation raffiné « Gérer les logements »

Descriptions textuelles du cas d'utilisation « Gérer les logements »

Titre	Gérer les logements
Acteur	Responsable d'agence
Résumé	Ce cas d'utilisation permet au responsable d'agence de gérer les logements de son portefeuille à travers l'ajout, la modification, la suppression, la consultation et la recherche.
Précondition(s)	Le responsable d'agence doit être authentifié sur la plateforme.
Postcondition(s)	Les informations relatives aux logements sont mises à jour après chaque opération (ajout, modification, suppression, consultation ou recherche).
Scénario nominal	1. Le responsable accède à l'interface de gestion des logements. 2. Il ajoute un logement en saisissant les informations nécessaires. 3. Il consulte la liste complète des logements. 4. Il recherche un logement selon différents critères (localisation, prix, type, etc.). 5. Il modifie un logement existant. 6. Il supprime un logement sélectionné.
Scénario(s) alternatif(s)	Données invalides : un message d'erreur est affiché.

Tableau 2.4 : Description textuelle du cas d'utilisation « Gérer les logements »

2.4.3.2 Raffinement du cas d'utilisation « Gérer les réservations »

Le diagramme présenté dans la figure 2.6 illustre les étapes du cas d'utilisation permettant au responsable d'agence de gérer les logements. Ce diagramme décrit le déroulement des actions relatives à la gestion des réservations, comme montré dans la figure 2.7.

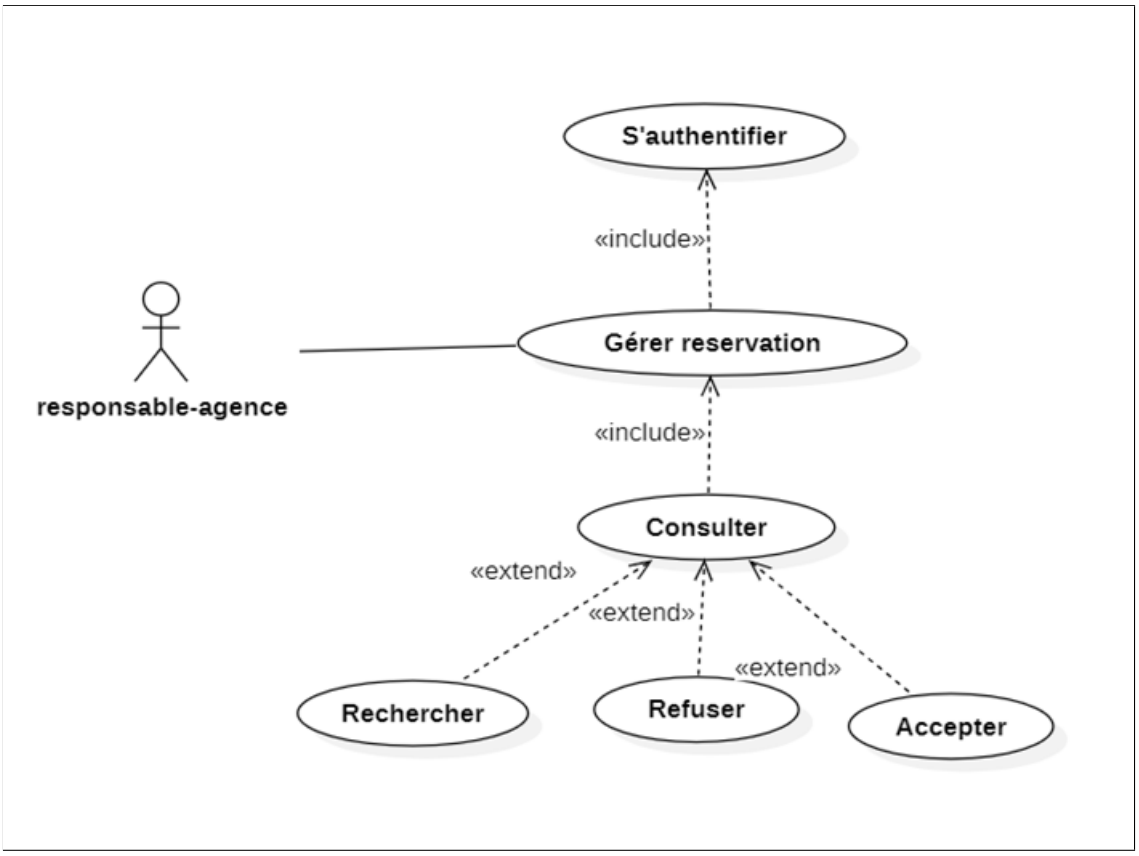


Figure 2.7 : Diagramme de cas d'utilisation raffiné « Gérer les réservations »

Descriptions textuelles du cas d'utilisation « Gérer les réservations »

Titre	Gérer les réservations
Acteur	Responsable d'agence
Résumé	Ce cas d'utilisation permet au responsable d'agence de consulter, rechercher, accepter ou refuser les demandes de réservation déposées par les utilisateurs.
Précondition(s)	Le responsable doit être authentifié. Il doit avoir accès à la section « Réservations ».
Postcondition(s)	Les réservations sont mises à jour suite aux actions du responsable (acceptation ou refus).
Scénario nominal	1. Le responsable consulte la liste des demandes de réservation. 2. Il recherche une demande selon différents critères. 3. Il consulte les détails d'une réservation. 4. Il accepte ou refuse la demande. 5. Le système met à jour la base de données et rafraîchit la liste.
Scénario(s) alternatif(s)	Aucune réservation trouvée. Erreur technique lors du traitement.

Tableau 2.5 : Description textuelle du cas d'utilisation « Gérer les réservations »

2.4.3.3 Raffinement du cas d'utilisation « Gérer les visites »

Ce diagramme met en évidence le processus détaillé pour gérer une visite, comme illustré dans la figure 2.8.

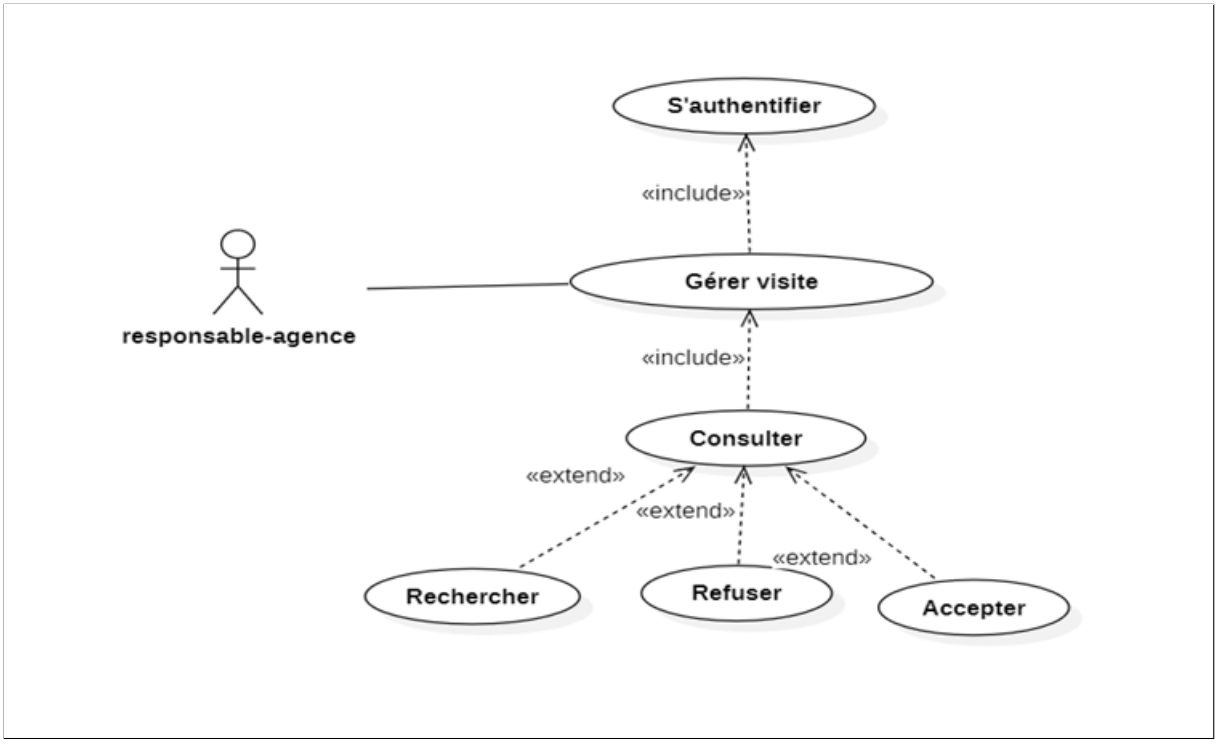


Figure 2.8 : Diagramme de cas d'utilisation raffiné « Gérer les visites »

Descriptions textuelles du cas d'utilisation « Gérer les visites »

Titre	Gérer les visites
Acteur	Responsable d'agence
Résumé	Ce cas d'utilisation permet au responsable d'agence de consulter, rechercher, accepter ou refuser les demandes de visite déposées par les utilisateurs.
Précondition(s)	Le responsable doit être authentifié et avoir accès à la section « Visites ».
Postcondition(s)	Les informations relatives aux visites sont mises à jour après traitement (acceptation ou refus).
Scénario nominal	1. Le responsable consulte la liste des visites. 2. Il effectue une recherche selon des critères définis. 3. Il consulte les détails d'une demande de visite. 4. Il accepte ou refuse la demande. 5. Le système met à jour le statut de la visite.
Scénario(s) alternatif(s)	Aucune visite trouvée. Erreur technique lors du traitement.

Tableau 2.6 : Description textuelle du cas d'utilisation « Gérer les visites »

2.4.3.4 Raffinement du cas d'utilisation « Gérer les négociations »

Ce diagramme décrit le déroulement des actions liées à la gestion des négociations, comme montré dans la figure 2.9.

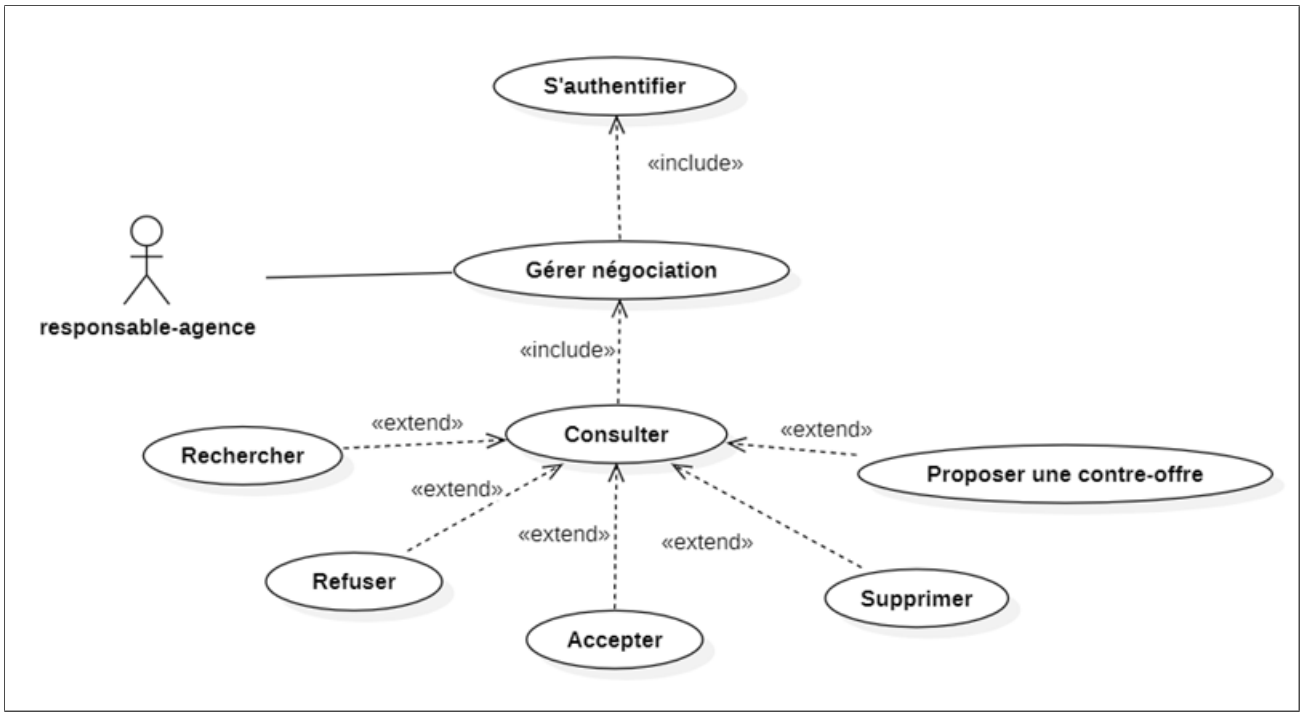


Figure 2.9 : Diagramme de cas d'utilisation raffiné « Gérer les négociations »

Descriptions textuelles du cas d'utilisation « Gérer les négociations »

Titre	Gérer les négociations
Acteur	Responsable d'agence, Client
Résumé	Ce cas d'utilisation permet de consulter, rechercher, accepter, refuser ou envoyer une contre-offre dans le cadre d'une négociation active.
Précondition(s)	Le responsable doit être authentifié.
Postcondition(s)	L'état de la négociation est mis à jour en fonction de l'action effectuée.
Scénario nominal	1. Le responsable accède à la section des négociations. 2. Il consulte la liste des négociations en cours. 3. Il effectue une recherche selon des critères spécifiques. 4. Il accepte, refuse ou envoie une contre-offre. 5. Le système met à jour le statut de la négociation.
Scénario(s) alternatif(s)	Erreur technique lors du traitement.

Tableau 2.7 : Description textuelle du cas d'utilisation « Gérer les négociations »

2.4.3.5 Raffinement du cas d'utilisation « Gérer les membres de l'équipe »

Dans la figure 2.10, nous illustrons le raffinement du cas d'utilisation « gérer les membres de l'équipe ».

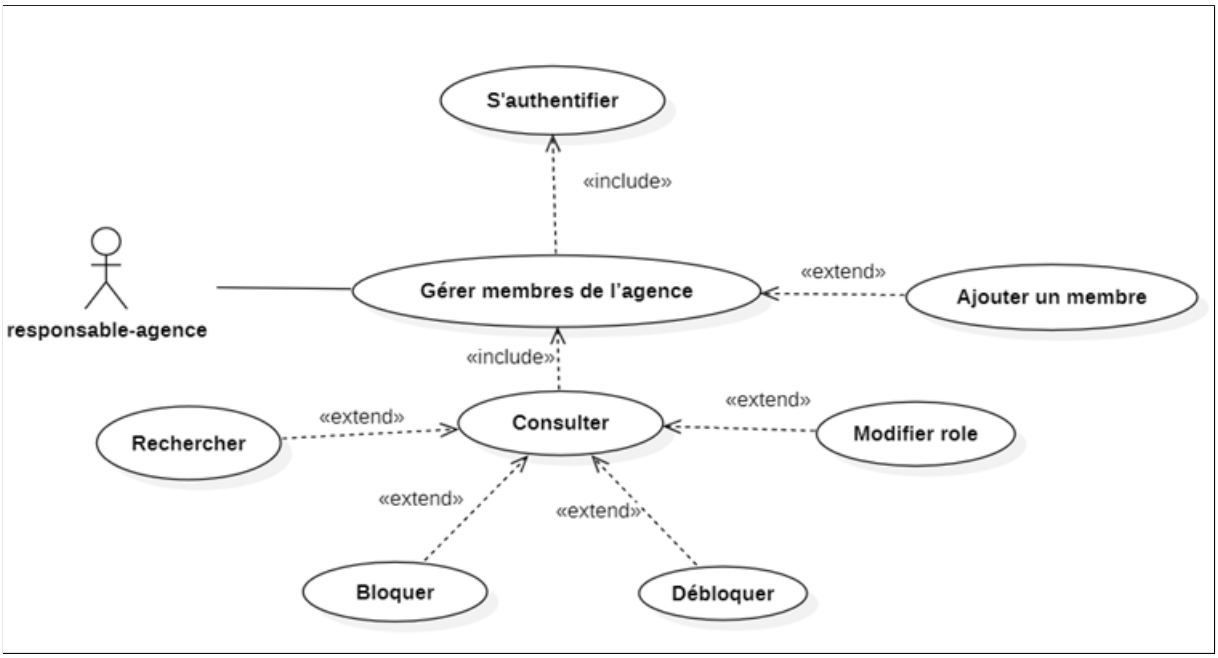


Figure 2.10 : Diagramme de cas d'utilisation raffiné « Gérer les membres de l'équipe »

Descriptions textuelles du cas d'utilisation « Gérer les membres de l'équipe »

Titre	Gérer les membres de l'équipe
Acteur	Responsable d'agence
Résumé	Ce cas d'utilisation permet au responsable d'agence d'ajouter, consulter, rechercher, bloquer/débloquer un membre ou modifier son rôle au sein de l'équipe.
Précondition(s)	Le responsable doit être authentifié et avoir accès à la section « Équipe ».
Postcondition(s)	Les informations des membres sont mises à jour selon l'action effectuée.
Scénario nominal	1. Le responsable accède à la section « Équipe ». 2. Il choisit d'ajouter un membre ou de consulter la liste existante. 3. Il effectue une recherche d'un membre si nécessaire. 4. Il modifie le rôle d'un membre ou bloque/débloque un membre.
Scénario(s) alternatif(s)	Erreur lors de l'ajout d'un membre (champ obligatoire manquant). Rôle sélectionné invalide ou inexistant. Aucun membre correspondant aux critères de recherche trouvé.

Tableau 2.8 : Description textuelle du cas d'utilisation « Gérer les membres de l'équipe »

2.4.3.6 Raffinement du cas d'utilisation « Estimer le prix par IA »

Dans la figure 2.11, nous illustrons le raffinement du cas d'utilisation « estimer prix par IA ».

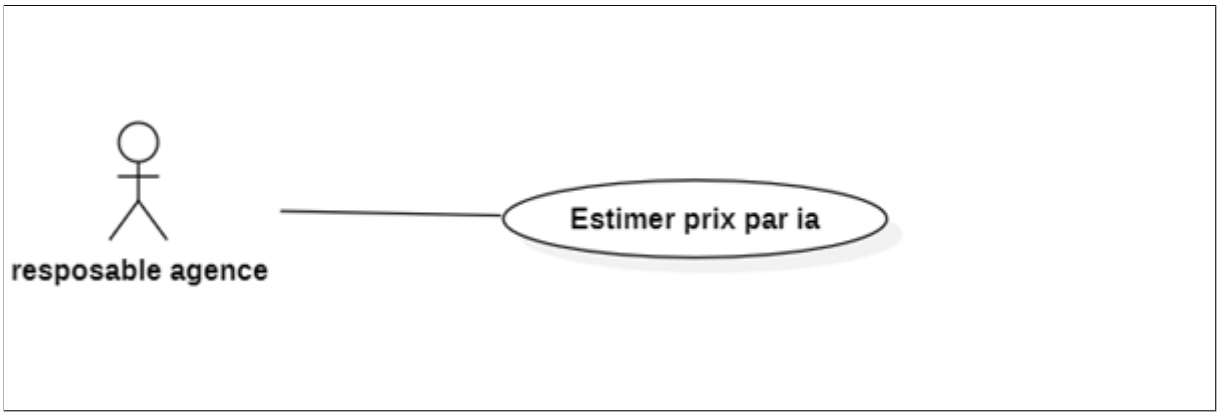


Figure 2.11 : Diagramme de cas d'utilisation raffiné « Estimer le prix par IA »

Descriptions textuelles du cas d'utilisation « Estimer le prix par IA »

Titre	Estimer le prix par IA
Acteur	Responsable d'agence
Résumé	Ce cas d'utilisation permet au responsable d'agence d'obtenir une estimation automatique du prix d'un logement grâce à un modèle d'intelligence artificielle.
Précondition(s)	Le responsable doit être authentifié et avoir accès au formulaire d'ajout de logement.
Postcondition(s)	Le prix estimé est affiché dans un modal et peut être utilisé pour compléter le champ prix du formulaire.
Scénario nominal	1. Le responsable saisit les caractéristiques du logement (surface, localisation, type, nombre de pièces, équipements, etc.). 2. Il clique sur le bouton « Estimer prix ». 3. Le système envoie les données au modèle IA. 4. Le modèle calcule et renvoie un prix estimé. 5. Le système affiche le prix estimé dans un modal.
Scénario(s) alternatif(s)	- Champs obligatoires non remplis → message d'avertissement. - Erreur du modèle IA ou données insuffisantes → message d'erreur et possibilité de saisir le prix manuellement.

Tableau 2.9 : Description textuelle du cas d'utilisation « Estimer le prix par IA »

2.4.3.7 Raffinement du cas d'utilisation « Gérer les commentaires »

Ce diagramme décrit le déroulement des actions liées à la gestion des commentaires, comme montré dans la figure 2.12.

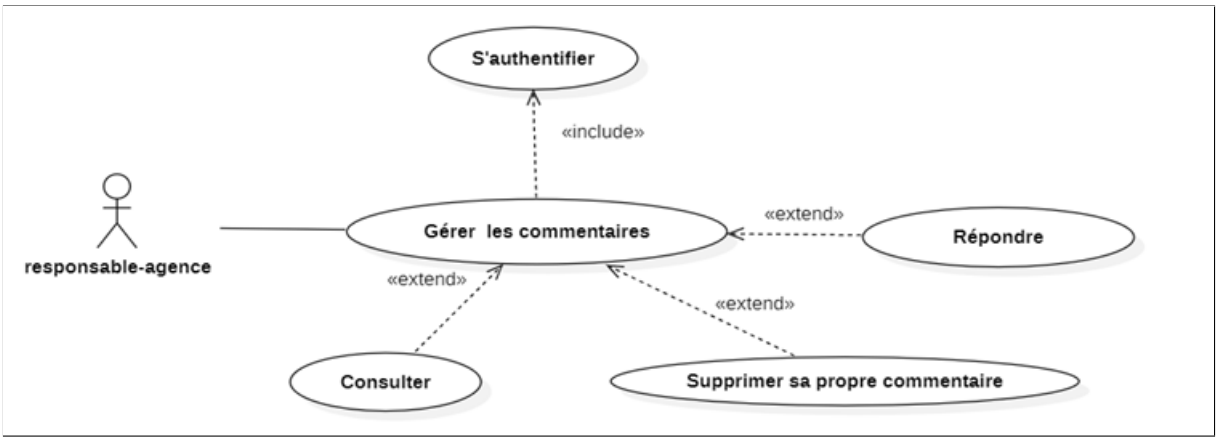


Figure 2.12 : Diagramme de cas d'utilisation raffiné « Gérer les commentaires »

Descriptions textuelles du cas d'utilisation « Gérer les commentaires »

Titre	Gérer les commentaires
Acteur	Responsable d'agence
Résumé	Ce cas d'utilisation permet au responsable d'agence de consulter les commentaires laissés par les clients, d'y répondre et de supprimer uniquement les réponses émises par l'agence.
Précondition(s)	Le responsable doit être authentifié et connecté. Au moins un logement doit être enregistré et disposer de commentaires.
Postcondition(s)	Les réponses de l'agence sont enregistrées ou supprimées. Les commentaires des clients restent inchangés et l'historique des interactions est conservé.
Scénario nominal	1. Le responsable accède à la page des logements et consulte les commentaires associés. 2. Le système affiche la liste des commentaires avec auteur, date et contenu. 3. Depuis chaque commentaire, le responsable peut répondre ou supprimer une réponse déjà donnée (uniquement celles de l'agence). 4. Le système met à jour la liste et affiche une confirmation.
Scénario(s) alternatif(s)	- Aucun commentaire disponible → affichage du message « Aucun commentaire pour ce logement ». - Erreur technique lors de l'enregistrement ou de la suppression → message d'échec de l'opération.

Tableau 2.10 : Description textuelle du cas d'utilisation « Gérer les commentaires »

2.4.3.8 Raffinement du cas d'utilisation « Échanger des messages »

Ce diagramme visualise le raffinement du cas d'utilisation «Échanger des messages», présenté dans la figure 2.13.

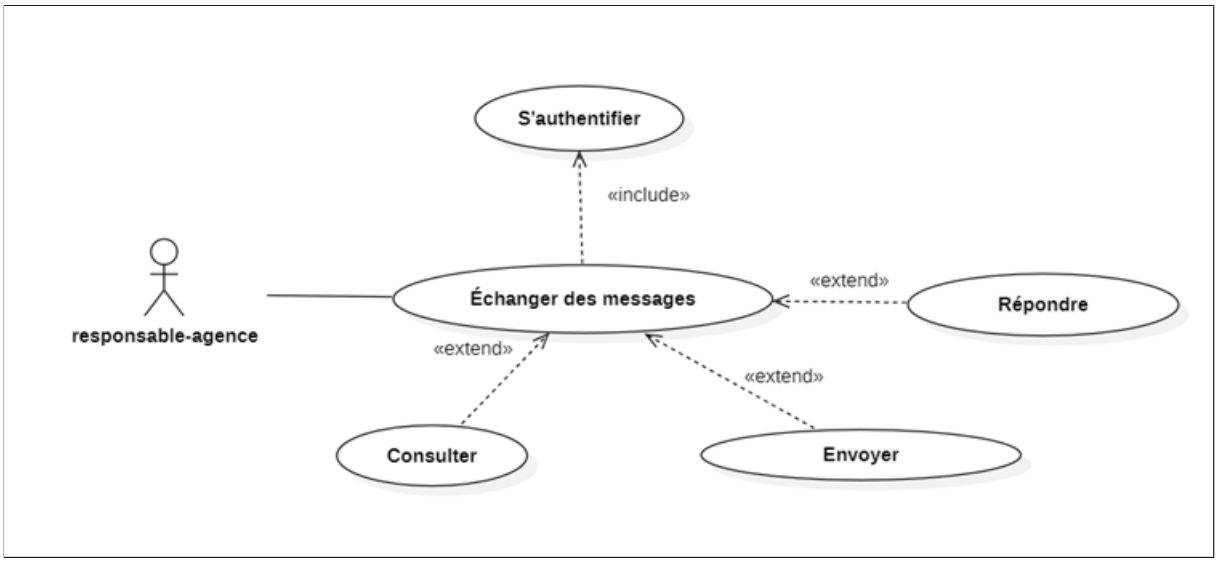


Figure 2.13 : Diagramme de cas d'utilisation raffiné « Échanger des messages »

Descriptions textuelles du cas d'utilisation « changer des messages »

Titre	Échanger des messages
Acteur	Responsable d'agence
Résumé	Ce cas d'utilisation permet au responsable d'agence de consulter et répondre aux messages échangés avec ses contacts. L'accès à la messagerie est protégé par authentification.
Précondition(s)	Le responsable doit être authentifié et connecté. Des messages doivent exister ou peuvent être envoyés aux contacts de l'agence.
Postcondition(s)	Les messages envoyés ou les réponses sont enregistrés et visibles dans la conversation. L'historique des échanges est conservé.
Scénario nominal	1. Le responsable ouvre la section messagerie. 2. La liste complète des conversations apparaît à l'écran. 3. Depuis chaque conversation, le responsable peut lire les messages reçus et répondre ou envoyer un nouveau message. 4. Le système enregistre la réponse et affiche la confirmation de l'envoi.
Scénario(s) alternatif(s)	- Aucun message disponible → affichage du message « Aucun message pour le moment ». - Échec de l'envoi pour une raison technique → affichage d'un message d'erreur.

Tableau 2.11 : Description textuelle du cas d'utilisation « Échanger des messages »

2.4.3.9 Raffinement du cas d'utilisation « Consulter les emails et notifications»

Le diagramme ci-dessous illustre les étapes suivies par un utilisateur pour consulter ses emails et notifications. La figure 2.14 présente ce raffinement du cas d'utilisation.

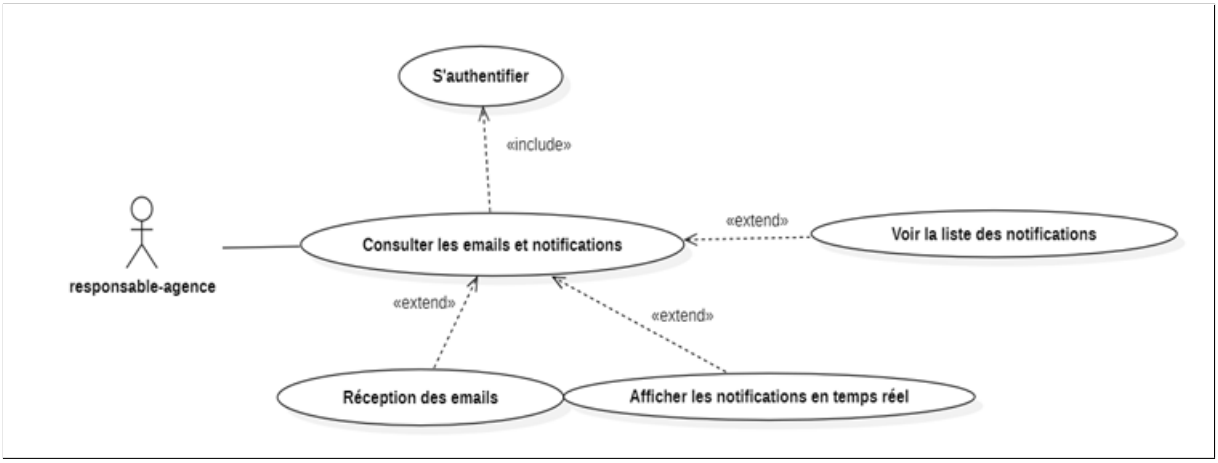


Figure 2.14 : Diagramme de cas d'utilisation raffiné « Consulter les emails et notifications »

Descriptions textuelles du cas d'utilisation « Consulter les emails et notifications »

Titre	Consulter les emails et notifications
Acteur	Responsable d'agence
Résumé	Ce cas d'utilisation permet au responsable d'agence de consulter les notifications reçues en temps réel concernant les demandes de réservation, de visite ou de négociation. Les notifications s'affichent directement dans l'application avec un compteur. Les emails correspondants sont également reçus dans la boîte email de l'agence.
Précondition(s)	Le responsable est authentifié et connecté. L'agence a reçu au moins un email ou une notification concernant une demande.
Postcondition(s)	Les notifications consultées sont marquées comme lues dans l'application. Les emails restent disponibles dans la boîte mail externe. L'historique des demandes reste intact.
Scénario nominal	1. Le responsable voit le compteur des notifications en temps réel dans l'application. 2. Il clique pour afficher la liste des notifications. 3. Le système présente à l'utilisateur la liste complète de ses notifications. 4. L'utilisateur peut lire chaque notification directement dans l'application.
Scénario(s) alternatif(s)	- Aucune notification disponible → affichage du message « Aucune notification pour le moment ». - Erreur technique → impossibilité de charger les notifications.

Tableau 2.12 : Description textuelle du cas d'utilisation « Consulter les emails et notifications »

2.4.4 Diagrammes des cas d'utilisations de l'acteur « Client»

2.4.4.1 Raffinement du cas d'utilisation « Réserver un logement »

Le diagramme présenté dans la figure 5.24 illustre les différentes étapes permettant à un utilisateur de réserver un logement.



Figure 2.15 : Diagramme de cas d'utilisation raffiné « Réserver un logement »

Descriptions textuelles du cas d'utilisation « Réserver un logement »

Titre	Réserver un logement
Acteur	Client
Résumé	Permettre au client de réserver un logement disponible sur la plateforme.
Précondition(s)	Le client doit se connecter pour accéder au système.
Postcondition(s)	Une réservation est créée et enregistrée dans le tableau de bord du client.
Scénario nominal	1- Le client ouvre la page détaillée d'un logement. 2- Il sélectionne le bouton « Réserver ». 3- Un formulaire de réservation s'affiche avec les champs (date, durée, informations nécessaires). 4- Le client complète les champs et valide sa réservation. 5- Le système confirme et enregistre la réservation.
Scénario(s) alternatif(s)	- Si le logement est déjà réservé, le système affiche un message d'indisponibilité. - Si des informations sont incomplètes, le système demande de corriger les champs obligatoires.

Tableau 2.13 : Descriptions textuelles du cas d'utilisation « Réserver un logement »

2.4.4.2 Raffinement du cas d'utilisation « Demander une visite »

Le diagramme suivant explicite les étapes qu'un utilisateur franchit pour réserver une visite. La figure 2.16 détaille ce scénario.



Figure 2.16 : Diagramme de cas d'utilisation raffiné « Demander une visite »

Descriptions textuelles du cas d'utilisation « Demander une visite »

Titre	Demander une visite
Acteur	Client
Résumé	Permettre au client de planifier une visite du logement avec l'agence.
Précondition(s)	Le client doit être authentifié et avoir choisi un logement.
Postcondition(s)	Une demande de visite est créée et enregistrée dans le tableau de bord du client.
Scénario nominal	1- Le client consulte un logement. 2- Il sélectionne « Demander une visite ». 3- Le système affiche un formulaire pour choisir la date et l'heure. 4- Le client valide sa demande. 5- Le système enregistre et notifie l'agence.
Scénario(s) alternatif(s)	- Si la date/heure choisie n'est pas disponible, le système invite le client à sélectionner une autre plage horaire.

Tableau 2.14 : Descriptions textuelles du cas d'utilisation « Demander une visite »

2.4.4.3 Raffinement du cas d'utilisation « Demander une négociation »

La figure 2.17 présente le raffinement du cas d'utilisation « Demander une négociation ».



Figure 2.17 : Diagramme de cas d'utilisation raffiné « Demander une négociation »

Descriptions textuelles du cas d'utilisation « Demander une négociation »

Titre	Demander une négociation
Acteur	Client
Résumé	Donne la possibilité au client de négocier le prix ou les conditions d'un logement avec l'agence.
Précondition(s)	Le client doit être authentifié et avoir sélectionné un logement.
Postcondition(s)	Une demande de négociation est envoyée et enregistrée dans le tableau de bord.
Scénario nominal	1- Le client consulte l'annonce d'un logement. 2- Il choisit « Demander une négociation ». 3- Le système ouvre un espace de saisie (prix proposé, message). 4- Le client soumet sa demande. 5- Le système enregistre et notifie l'agence.
Scénario(s) alternatif(s)	- Le système vérifie le formulaire et alerte le client si des informations sont manquantes ou si le prix proposé est anormal.

Tableau 2.15 : Descriptions textuelles du cas d'utilisation « Demander une négociation »

2.4.4.4 Raffinement du cas d'utilisation « Consulter ses réservations »

Le diagramme suivant décrit les étapes pour consulter ses réservations. La figure 2.18 illustre ce raffinement.

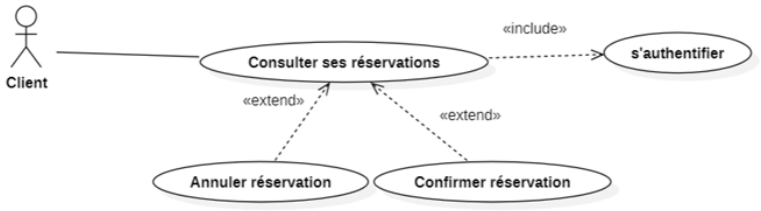


Figure 2.18 : Diagramme de cas d'utilisation raffiné « Consulter ses réservations »

Descriptions textuelles du cas d'utilisation « Consulter ses réservations »

Titre	Consulter ses réservations
Acteur	Client
Résumé	Le client accède à la page de ses réservations pour visualiser toutes les réservations effectuées, leur statut et les actions possibles.
Précondition(s)	Le client doit être authentifié.
Postcondition(s)	Les réservations sont affichées correctement et le client peut effectuer les actions autorisées.
Scénario nominal	1- Accès à la page de réservations. 2- Affichage des statistiques globales. 3- Liste détaillée des réservations (logement, date, prix, statut, réponse). 4- Actions selon le statut : annuler, contacter l'agence, supprimer ou payer. 5- Mise à jour et confirmation des actions.
Scénario(s) alternatif(s)	- Si aucune réservation n'existe, le système affiche "Aucune réservation". - En cas d'erreur lors d'une action, un message d'erreur est affiché.

Tableau 2.16 : Descriptions textuelles du cas d'utilisation « Consulter ses réservations »

2.4.4.5 Raffinement du cas d'utilisation « Consulter ses négociations »

Le diagramme ci-dessous illustre les étapes permettant au client de consulter ses négociations. La figure 2.19 montre ce raffinement.

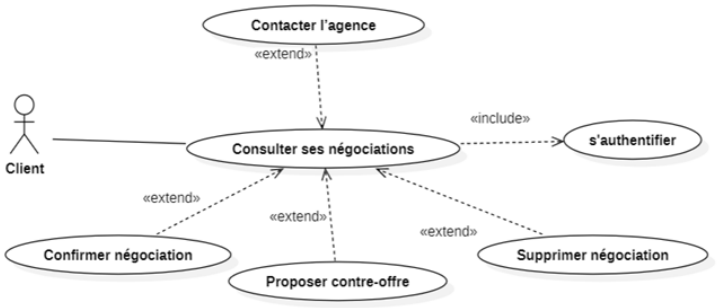


Figure 2.19 : Diagramme de cas d'utilisation raffiné « Consulter ses négociations »

Descriptions textuelles du cas d'utilisation « Consulter ses négociations »

Titre	Consulter ses négociations
Acteur	Client
Résumé	Le client consulte toutes ses négociations en cours, visualise les messages et propose des contre-offres ou confirme/annule les négociations.
Précondition(s)	Le client doit être authentifié.
Postcondition(s)	Les négociations sont affichées avec leur statut et les actions possibles.
Scénario nominal	1- Accès à la page des négociations. 2- Affichage des statistiques globales. 3- Liste détaillée (logement, date, émetteur, message, prix proposé, statut). 4- Actions possibles : confirmer, proposer contre-offre, annuler. 5- Suppression possible pour les négociations terminées envoyées par le client. 6- Mise à jour et confirmation des actions.
Scénario(s) alternatif(s)	- Si aucune négociation n'existe, le système affiche "Aucune négociation". - En cas d'erreur lors d'une opération, un message d'alerte apparaît.

Tableau 2.17 : Descriptions textuelles du cas d'utilisation « Consulter ses négociations »

2.4.4.6 Raffinement du cas d'utilisation « Consulter ses visites »

Le diagramme suivant présente le raffinement du cas d'utilisation « Consulter ses visites ». Voir figure 2.20.

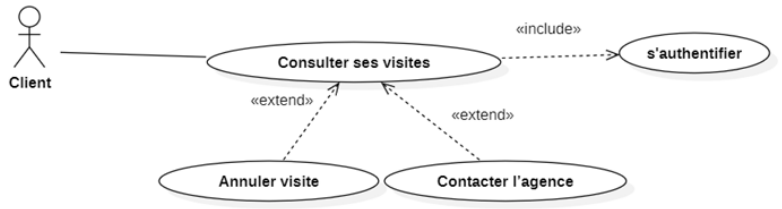


Figure 2.20 : Diagramme de cas d'utilisation raffiné « Consulter ses visites »

Descriptions textuelles du cas d'utilisation « Consulter ses visites »

Titre	Consulter ses visites
Acteur	Client
Résumé	Le client peut consulter toutes les visites prévues et interagir selon leur statut.
Précondition(s)	Le client doit être authentifié.
Postcondition(s)	Les visites sont affichées correctement et les actions disponibles peuvent être effectuées.
Scénario nominal	1- Accès à la page des visites. 2- Affichage d'un calendrier avec toutes les visites planifiées. 3- Liste détaillée (logement, adresse, date, statut). 4- Actions selon le statut : annuler ou contacter l'agence. 5- Confirmation et mise à jour du calendrier et de la liste.
Scénario(s) alternatif(s)	- Si aucune visite n'est planifiée, le système affiche "Aucune visite". - En cas d'échec lors de l'annulation, un message d'erreur est affiché.

Tableau 2.18 : Descriptions textuelles du cas d'utilisation « Consulter ses visites »

2.4.4.7 Raffinement du cas d'utilisation « Consulter emails et notifications »

Le diagramme ci-dessous illustre la consultation des emails et notifications. La figure 2.21 présente ce raffinement.

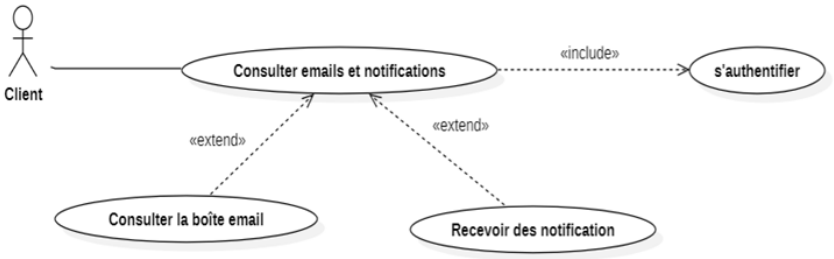


Figure 2.21 : Diagramme de cas d'utilisation raffiné « Consulter emails et notifications »

Descriptions textuelles du cas d'utilisation « Consulter emails et notifications »

Titre	Consulter emails et notifications
Acteur	Client
Résumé	Le client consulte ses emails et notifications concernant ses actions sur la plateforme.
Précondition(s)	Client authentifié et adresse email valide.
Postcondition(s)	Le client est informé des évolutions de ses demandes.
Scénario nominal	1- Une action survient sur la plateforme. 2- Le système envoie une notification en temps réel. 3- Le système génère un email récapitulatif. 4- Le client consulte les notifications sur l'application. 5- Le client ouvre sa boîte email pour lire le message.
Scénario(s) alternatif(s)	- Si l'email n'a pas pu être délivré, le client reste informé via la notification interne.

Tableau 2.19 : Descriptions textuelles du cas d'utilisation « Consulter emails et notifications »

2.4.4.8 Raffinement du cas d'utilisation « Échanger des messages avec une agence »

Le diagramme suivant illustre les étapes pour échanger des messages. Voir figure 2.22.

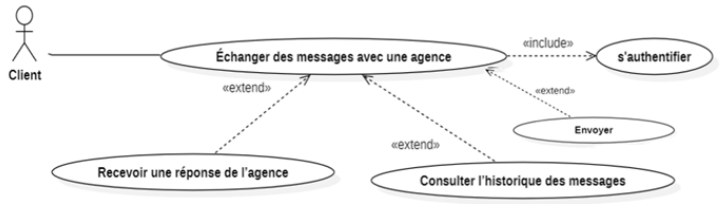


Figure 2.22 : Diagramme de cas d'utilisation raffiné « Échanger des messages avec une agence »

Descriptions textuelles du cas d'utilisation « Échanger des messages avec une agence »

Titre	Échanger des messages avec une agence
Acteur	Client
Résumé	Le client utilise la messagerie intégrée pour contacter une agence et suivre les échanges.
Précondition(s)	Client connecté et accès à une page de contact agence.
Postcondition(s)	Les messages sont envoyés et enregistrés.
Scénario nominal	1- Sélection de l'option « Contacter l'agence ». 2- Ouverture de la page de messagerie. 3- Saisie et envoi du message. 4- Transmission et ajout à l'historique. 5- Consultation et poursuite des échanges via le tableau de bord.
Scénario(s) alternatif(s)	- Service indisponible : le système renvoie un message d'erreur.

Tableau 2.20 : Descriptions textuelles du cas d'utilisation « Échanger des messages avec une agence »

2.4.4.9 Raffinement du cas d'utilisation « Rechercher des logements »

Le diagramme ci-dessous illustre la recherche de logements. La figure 2.23 présente ce raffinement.



Figure 2.23 : Diagramme de cas d'utilisation raffiné « Rechercher des logements »

Descriptions textuelles du cas d'utilisation « Rechercher des logements »

Titre	Rechercher des logements
Acteur	Utilisateur
Résumé	L'utilisateur effectue une recherche de logements avec ou sans critères et consulte les résultats proposés.
Précondition(s)	Accès à la plateforme sans obligation de connexion.
Postcondition(s)	Le système affiche la liste des logements correspondant aux critères ou l'ensemble des logements disponibles.
Scénario nominal	1- Accès à la section « Recherche de logements ». 2- Saisie éventuelle des critères (ville, type, budget...). 3- Exécution de la recherche. 4- Affichage de la liste des logements trouvés. 5- Consultation des détails d'un logement sélectionné.
Scénario(s) alternatif(s)	- Si aucun critère n'est saisi, le système affiche tous les logements disponibles. - Si aucun logement ne correspond, un message informe qu'aucun résultat n'a été trouvé.

Tableau 2.21 : Descriptions textuelles du cas d'utilisation « Rechercher des logements »

2.4.4.10 Raffinement du cas d'utilisation « Gérer les commentaires »

Le diagramme ci-dessous montre le raffinement du cas d'utilisation pour gérer les commentaires. Voir figure 2.24.

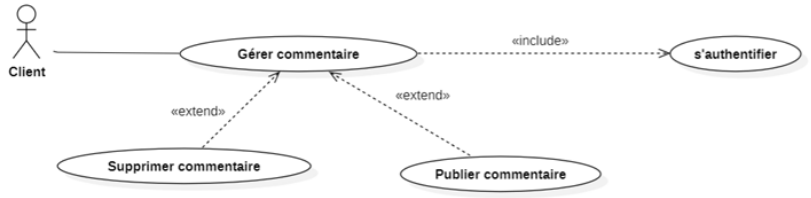


Figure 2.24 : Diagramme de cas d'utilisation raffiné « Gérer les commentaires »

Descriptions textuelles du cas d'utilisation « Gérer les commentaires »

Titre	Gérer les commentaires
Acteur	Client
Résumé	Le client peut interagir avec la section commentaires d'un logement : ajouter ou supprimer ses propres commentaires.
Précondition(s)	Client connecté et sur la page de détail d'un logement.
Postcondition(s)	Le commentaire est enregistré ou retiré de la section correspondante.
Scénario nominal	1- Consultation de la page du logement. 2- Rédaction et validation du commentaire. 3- Enregistrement et affichage du commentaire. 4- Possibilité de supprimer son commentaire.
Scénario(s) alternatif(s)	- Si le client n'est pas connecté, le système demande l'authentification. - Si aucun commentaire n'est saisi, un message d'erreur apparaît.

Tableau 2.22 : Descriptions textuelles du cas d'utilisation « Gérer les commentaires »

2.4.5 Diagrammes des cas d'utilisations de l'acteur « Visiteur »

2.4.5.1 Raffinement du cas d'utilisation « Consulter la liste des logements »

Le diagramme ci-dessous montre le cheminement pour qu'un utilisateur puisse explorer la liste des biens disponibles. La figure 2.25 illustre ce diagramme.



Figure 2.25 : Diagramme de cas d'utilisation « Consulter la liste des logements »

Descriptions textuelles du cas d'utilisation « Consulter la liste des logements »

Titre	Consulter la liste des logements
Acteur	Visiteur / Client
Résumé	L'utilisateur peut parcourir l'ensemble des logements disponibles sur la plateforme avec leurs informations principales (titre, description, localisation, prix, etc.).
Précondition(s)	Des logements doivent être ajoutés et publiés sur la plateforme.
Postcondition(s)	L'utilisateur visualise la liste des logements disponibles.
Scénario nominal	1- L'utilisateur navigue vers la section « Logements ». 2- Le système expose la liste des logements disponibles avec leurs informations essentielles. 3- L'utilisateur parcourt la liste et peut cliquer sur un logement pour consulter la page de détails.
Scénario(s) alternatif(s)	- Si aucun logement n'est disponible, le système affiche un message informatif.

Tableau 2.23 : Descriptions textuelles du cas d'utilisation « Consulter la liste des logements »

2.4.5.2 Raffinement du cas d'utilisation « Consulter les détails d'un logement »

Le diagramme ci-dessous montre les étapes permettant à un utilisateur de visualiser les détails d'un logement. La figure 2.26 illustre ce diagramme.



Figure 2.26 : Diagramme de cas d'utilisation « Consulter les détails d'un logement »

Descriptions textuelles du cas d'utilisation « Consulter les détails d'un logement »

Titre	Consulter les détails d'un logement
Acteur	Visiteur / Client
Résumé	L'utilisateur accède à la fiche détaillée d'un logement sélectionné avec toutes les informations nécessaires (description, prix, localisation, photos, coordonnées de l'agence).
Précondition(s)	Des logements doivent être disponibles ; l'utilisateur a sélectionné un logement.
Postcondition(s)	L'utilisateur visualise toutes les informations détaillées du logement choisi.
Scénario nominal	1- L'utilisateur choisit un logement dans la liste. 2- Le système ouvre la page de détails. 3- L'utilisateur parcourt les informations disponibles.
Scénario(s) alternatif(s)	Pas d'exception.

Tableau 2.24 : Descriptions textuelles du cas d'utilisation « Consulter les détails d'un logement »

2.4.5.3 Raffinement du cas d'utilisation « Consulter la liste des agences »

Le diagramme ci-dessous décrit le processus pour qu'un visiteur consulte la liste des agences. La figure 2.27 illustre ce diagramme.



Figure 2.27 : Diagramme de cas d'utilisation « Consulter la liste des agences »

Descriptions textuelles du cas d'utilisation « Consulter la liste des agences »

2.4.5.4 Raffinement du cas d'utilisation « Consulter les détails d'une agence »

Le diagramme ci-dessous illustre les étapes pour qu'un visiteur consulte les détails d'une agence. La figure 2.28 présente ce cas d'usage.

Titre	Explorer les agences disponibles
Acteur	Visiteur / Client
Résumé	L'utilisateur accède à la section des agences et peut parcourir la liste avec leurs informations principales et annonces publiées.
Précondition(s)	Des agences doivent être enregistrées et actives.
Postcondition(s)	L'utilisateur visualise la liste des agences disponibles.
Scénario nominal	1- L'utilisateur navigue vers la section « Agences ». 2- Le système affiche les agences avec leurs informations essentielles.
Scénario(s) alternatif(s)	- Si aucune agence n'est enregistrée, le système affiche un message indiquant que la liste est vide.

Tableau 2.25 : Descriptions textuelles du cas d'utilisation « Consulter la liste des agences »



Figure 2.28 : Diagramme de cas d'utilisation « Consulter les détails d'une agence »

Descriptions textuelles du cas d'utilisation « Consulter les détails d'une agence »

Titre	Consulter les détails d'une agence
Acteur	Visiteur / Client
Résumé	L'utilisateur accède aux informations détaillées d'une agence et peut consulter ses annonces et coordonnées principales.
Précondition(s)	Des agences doivent être enregistrées et actives.
Postcondition(s)	L'utilisateur visualise les informations détaillées de l'agence choisie.
Scénario nominal	1- L'utilisateur accède à la section « Agences ». 2- Le système affiche les informations détaillées de l'agence.
Scénario(s) alternatif(s)	- Si aucune agence n'est présente, le système affiche un message indiquant que la liste est vide.

Tableau 2.26 : Descriptions textuelles du cas d'utilisation « Consulter les détails d'une agence »

2.4.5.5 Raffinement du cas d'utilisation « Gérer le profil »

Le diagramme ci-joint explicite la décomposition complète du scénario « Gérer le profil », en détaillant les interactions variées entre l'utilisateur et la solution. Ce diagramme apparaît dans la figure 2.29.

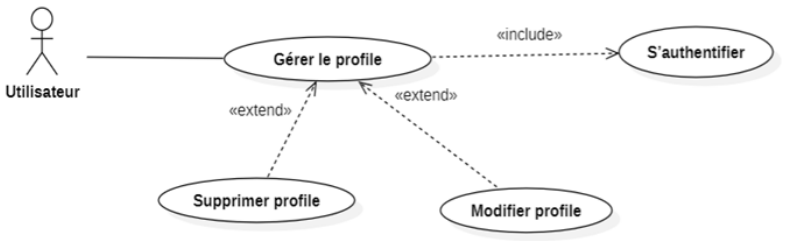


Figure 2.29 : Diagramme de cas d'utilisation « Gérer le profil »

Descriptions textuelles du cas d'utilisation « Gérer le profil »

Titre	Gérer le profil
Acteur	Utilisateur
Résumé	Les utilisateurs (administrateur, responsable d'agence et client) peuvent accéder à leur espace personnel afin de consulter et mettre à jour leurs informations. Cette fonctionnalité leur permet d'assurer que leurs données sont correctes et à jour, et de gérer leurs préférences de connexion.
Précondition(s)	L'utilisateur doit être authentifié sur la plateforme.
Postcondition(s)	Le profil est modifié et les changements sont enregistrés.
Scénario nominal	1- L'utilisateur se connecte à son espace personnel. 2- Les informations actuelles du profil sont affichées. 3- L'utilisateur choisit de mettre à jour ses données (informations de contact, photo, mot de passe, etc.). 4- Le système valide et enregistre les modifications. 5- L'utilisateur consulte le profil mis à jour.
Scénario(s) alternatif(s)	- Si les données saisies sont incorrectes (exemple : email invalide ou mot de passe trop court), le système affiche un message d'erreur. - Si l'utilisateur choisit de supprimer son profil (selon ses droits), le système demande une confirmation avant de procéder.

Tableau 2.27 : Descriptions textuelles du cas d'utilisation « Gérer le profil »

Conclusion

Nous avons présenté, dans ce chapitre, les différents utilisateurs intervenant dans le système et également leurs attentes fonctionnelles et non fonctionnelles, Ces éléments sont illustrés par des diagrammes de cas d'utilisation. Dans le chapitre qui suit on focalisera sur les étapes de conception et de développement de la solution proposée.

Chapitre 3

Conception

Plan

1 Diagramme de classes 51

2 Diagramme de séquence 52

Introduction

Suite à l'analyse des exigence et des fonctionnalités de l'application présentée au sein du chapitre précédent, ce chapitre aborde la phase de conception, une phase importante du du processus de développement. Elle établit le lien entre l'analyse réalisée en amont et l'implémentation à venir, garantissant ainsi une cohérence et une continuité dans la construction du système. Cette phase conceptuelle vise à transformer les spécifications fonctionnelles en modèles concrets qui guideront le développement technique.

3.1 Diagramme de classes

Le diagramme de classes correspond a un élément fondamental dans une approche orientée objet. Il permet de modéliser la structure interne du système en identifiant les différentes classes, leurs attributs et les associations qui les relient. Ce modèle offre une vision claire et organisée de l'architecture, facilitant la compréhension du fonctionnement global et servant de référence pour la phase d'implémentation.

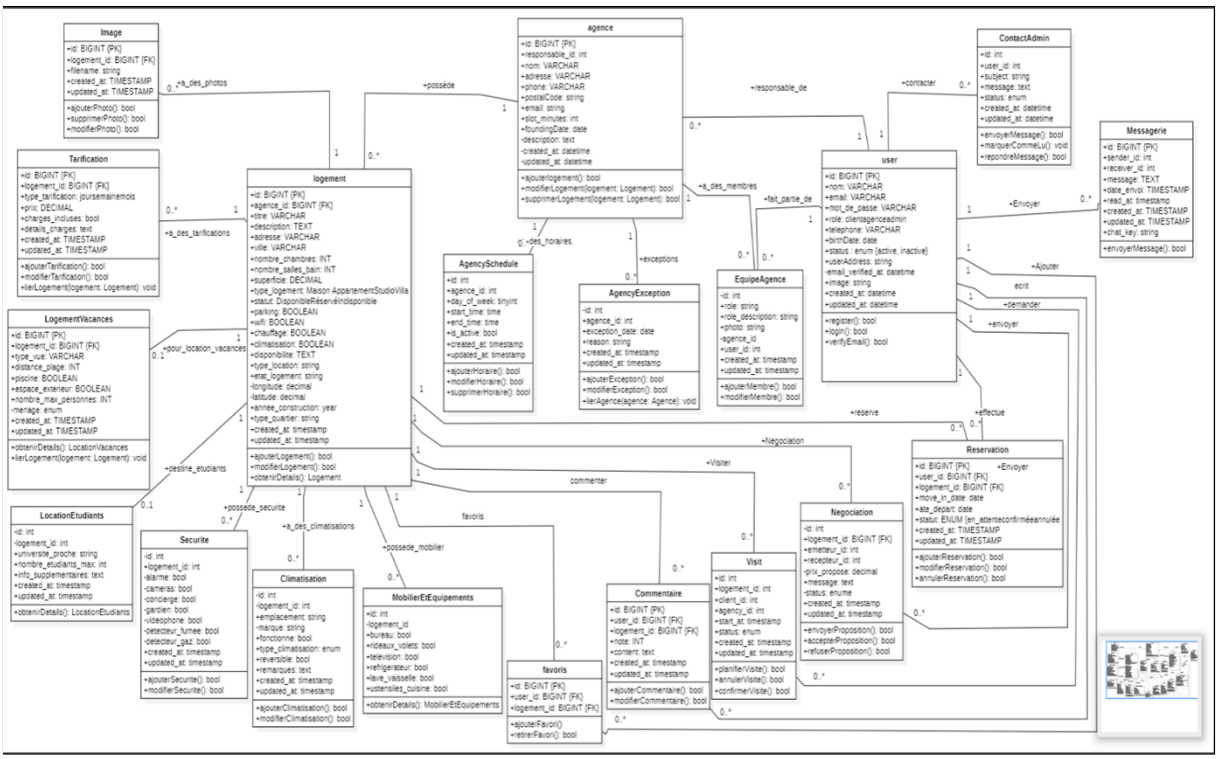


Figure 3.1 : Diagramme de classe

3.2 Diagramme de séquence

Les diagrammes de séquence permettent d'illustrer, sous forme chronologique, les échanges entre les acteurs et le système en s'appuyant sur la notation UML. Ils mettent en évidence l'enchaînement des messages et des actions, ce qui facilite la compréhension du déroulement des différents scénarios d'utilisation. Dans cette section, nous exposons ci-dessous les diagrammes de séquence correspondant aux principaux cas d'utilisation de notre application

3.2.1 Diagrammes de séquence liés au cas d'utilisation «Authentification»

3.2.1.1 Diagramme de séquence « Authentification »

Le diagramme de séquence ci-dessous présente le processus complet d'authentification d'un utilisateur, depuis la saisie des identifiants jusqu'à la validation du système. La figure 3.2 présente ce diagramme de séquence associé à l'« Authentification ».

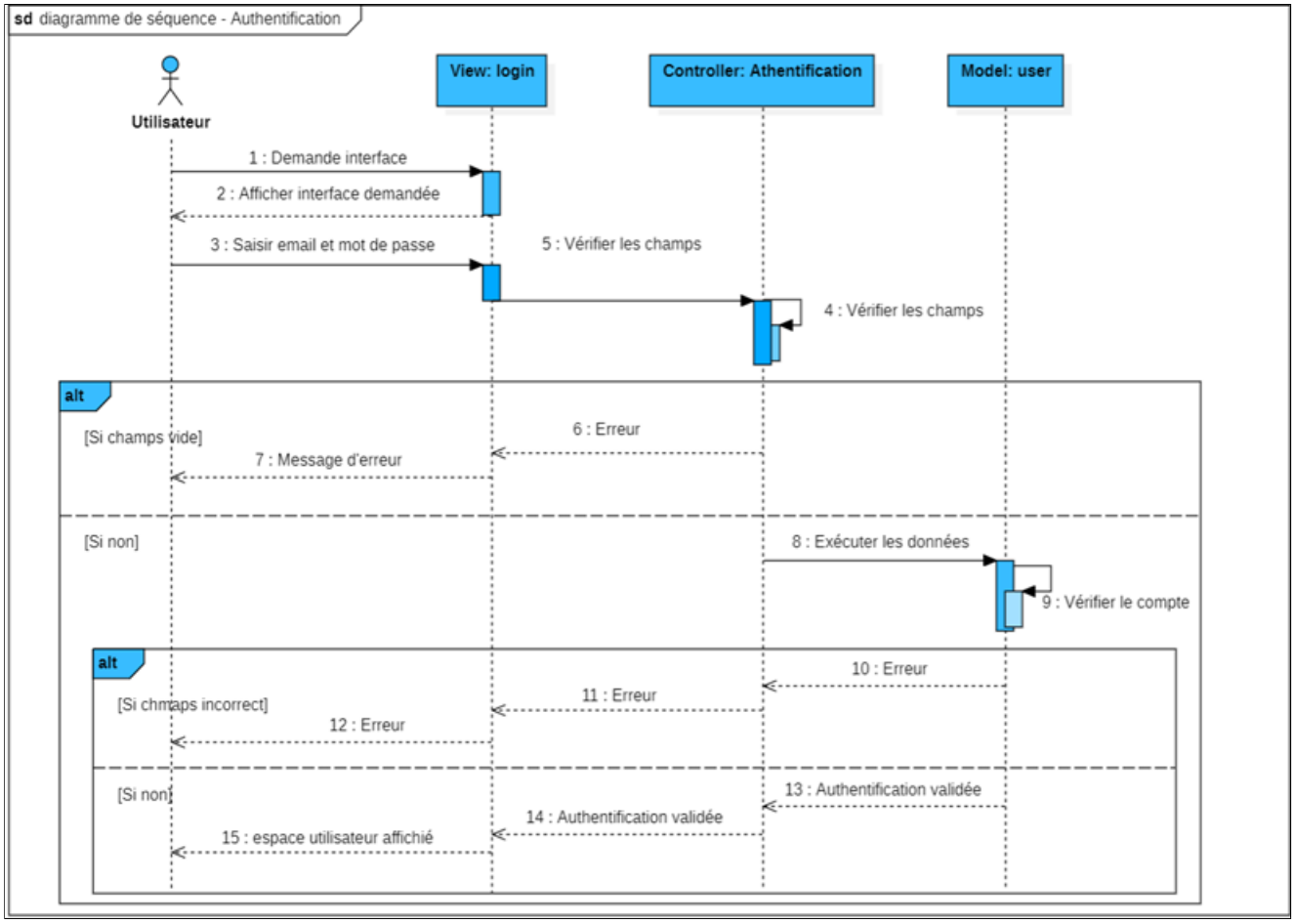


Figure 3.2 : Diagrammes de séquences « Authentification »

3.2.1.2 Détail du diagramme de séquences relatif à l'authentification

Étape 1–2 : Lorsque l'utilisateur accède à l'interface de connexion, la vue correspondante s'affiche et présente le formulaire d'authentification.

Étape 3 : L'utilisateur saisit son adresse e-mail ainsi que son mot de passe dans les champs dédiés.

Étape 4–5 : La vue transmet les données saisies au contrôleur, qui procède à une première vérification de la validité des champs.

— **[Si champ vide]** : Le contrôleur détecte qu'un ou plusieurs champs sont vides. Il renvoie une erreur, et la vue affiche un message signalant le problème à l'utilisateur.

— **[Sinon]** : Tous les champs sont correctement renseignés, le processus se poursuit.

Étape 8–9 : Le contrôleur transmet les informations au modèle, qui vérifie l'existence du compte utilisateur ainsi que la conformité des identifiants dans la base de données.

— **[Si identifiants incorrects]** : Le modèle renvoie une erreur au contrôleur indiquant soit l'inexistence du compte, soit des identifiants erronés. Le contrôleur transmet alors un message d'erreur à la vue, qui l'affiche à l'utilisateur.

— **[Sinon]** : Le compte existe et les identifiants sont corrects. La procédure d'authentification est validée.

Étape 13–15 : Le modèle confirme la réussite de l'authentification. Le contrôleur informe la vue, qui redirige l'utilisateur vers son espace personnel.

Résultat final : Si l'authentification réussit, l'utilisateur accède à son espace personnel avec les privilèges associés. En cas d'échec (champs vides ou identifiants incorrects), un message d'erreur est affiché pour guider l'utilisateur dans la correction du problème.

3.2.2 Diagrammes de séquence relatifs au cas d'utilisation « Gérer logements »

3.2.2.1 Diagramme de séquence « Ajouter logement »

Ce diagramme décrit le processus d'enregistrement d'un nouveau logement par un responsable d'agence. La figure 3.3 présente les différentes interactions successives entre la vue, le contrôleur et le modèle.

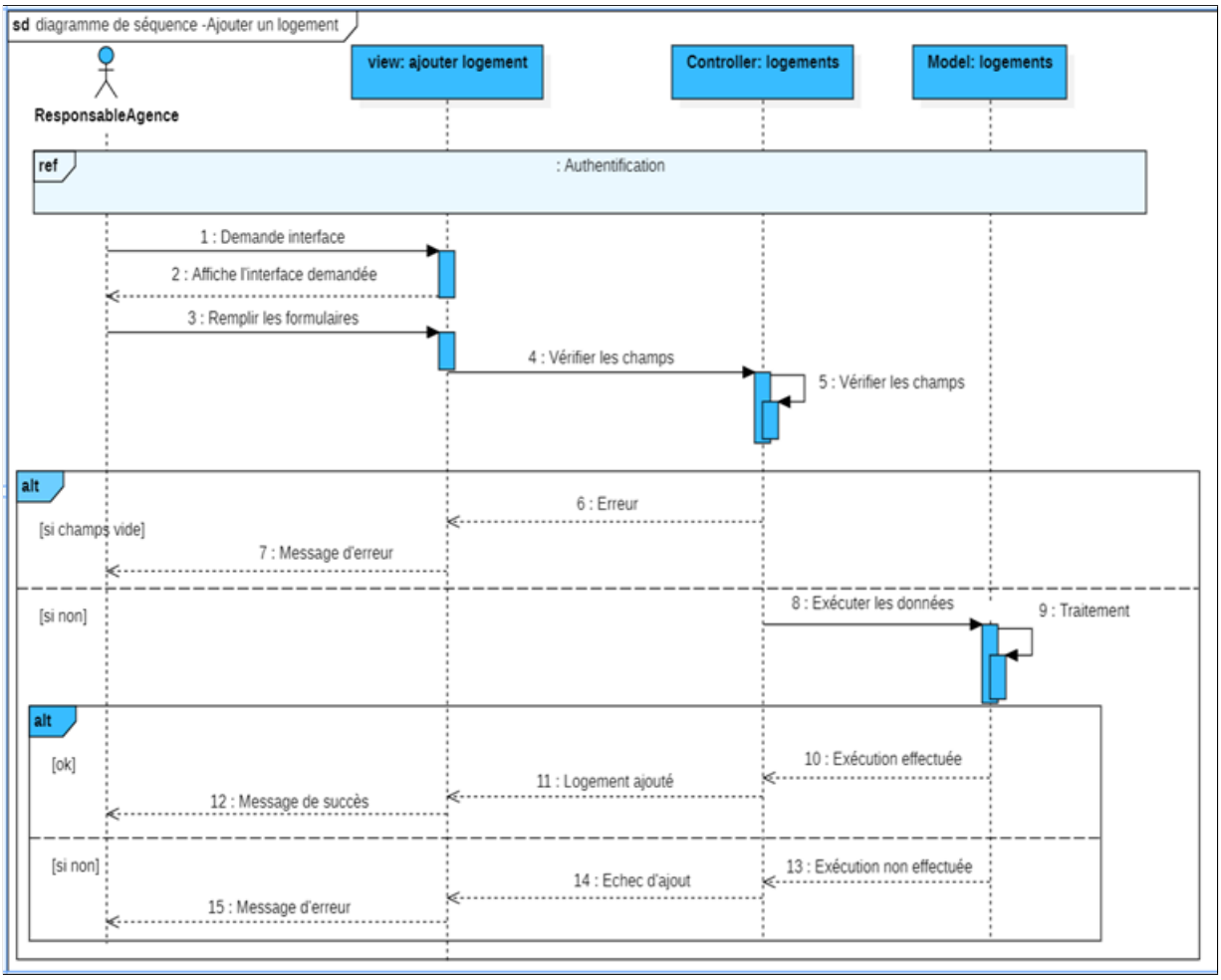


Figure 3.3 : Diagramme de séquence « Ajouter logement »

3.2.2.2 Détail du diagramme de séquence « Ajouter logement »

Étape 1–2 : Le responsable d'agence demande l'interface d'ajout de logement. La vue affiche l'interface prévue avec le formulaire d'ajout.

Étape 3 : Le responsable remplit le formulaire avec les informations nécessaires : adresse, prix, superficie, type, description, photos, etc.

Étape 4–5 : La vue transmet les données saisies au contrôleur, qui procède à la vérification des champs.

— **[Si champ vide]** : Certains champs obligatoires sont vides. Le contrôleur renvoie une erreur et la vue affiche un message d'avertissement au responsable.

— **[Sinon]** : Tous les champs nécessaires sont correctement remplis. Le processus continue.

Étape 8–9 : Le contrôleur transmet les données validées au modèle, qui exécute le traitement

d'ajout du logement dans la base de données.

— **[Ok]** : L'ajout est effectué avec succès. Le modèle confirme l'enregistrement du logement, le contrôleur informe la vue et un message de succès apparaît.

— **[Sinon]** : L'ajout échoue (erreur technique ou problème avec la base de données). Le modèle signale l'erreur au contrôleur, qui informe la vue. Un message d'erreur s'affiche.

Résultat final : Si l'ajout réussit, le logement est enregistré et devient disponible pour la consultation et la réservation. En cas d'échec (champs vides ou erreur technique), un message guide le responsable afin de corriger le problème.

3.2.2.3 Diagramme de séquence « Modifier logement »

Ce diagramme illustre le processus de mise à jour des informations d'un logement existant par un responsable d'agence. La figure 3.4 présente les interactions entre les différentes composantes du système.

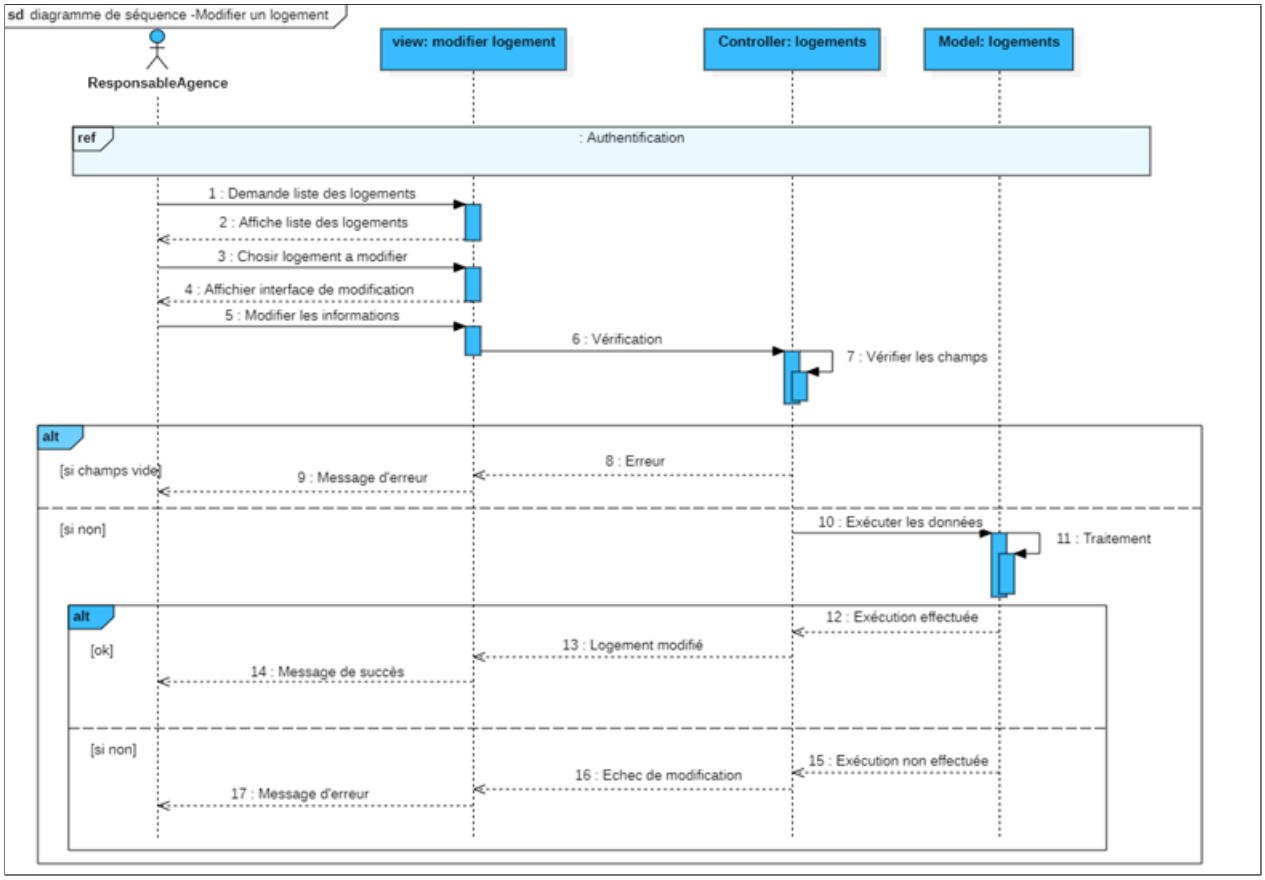


Figure 3.4 : Diagramme de séquence « Modifier logement »

3.2.2.4 Détail du diagramme de séquence « Modifier logement »

Étape 1–2 : Le responsable d'agence affiche la liste des logements disponibles. La vue présente les logements existants.

Étape 3 : Le responsable sélectionne le logement à modifier.

Étape 4 : La vue affiche l'interface de modification contenant les informations actuelles du logement pré-remplies.

Étape 5 : Le responsable modifie les informations souhaitées : prix, description, photos, type, disponibilité, etc.

Étape 6–7 : La vue transmet les données modifiées au contrôleur, qui vérifie la validité des informations.

— **[Si champ vide]** : Certains champs obligatoires sont vides. Le contrôleur renvoie une erreur et la vue affiche un message au responsable.

— **[Sinon]** : Les champs sont correctement remplis. Le processus continue.

Étape 10–11 : Le contrôleur envoie les données validées au modèle, qui exécute la modification dans la base de données.

— **[Ok]** : La modification est réussie. Le modèle confirme la mise à jour, le contrôleur informe la vue, et un message de succès s'affiche.

— **[Sinon]** : La modification échoue (problème technique ou erreur base de données). Le modèle renvoie un échec au contrôleur, qui en informe la vue. Un message d'erreur est affiché.

Résultat final : Si la modification réussit, les nouvelles informations sont enregistrées et immédiatement visibles par les utilisateurs. En cas d'échec (champs vides ou erreur technique), un message guide le responsable dans les corrections nécessaires.

3.2.2.5 Diagramme de séquence « Rechercher logement »

Ce diagramme modélise le processus de recherche de logements selon des critères spécifiques. La figure 3.5 illustre les interactions nécessaires entre la vue, le contrôleur et le modèle.

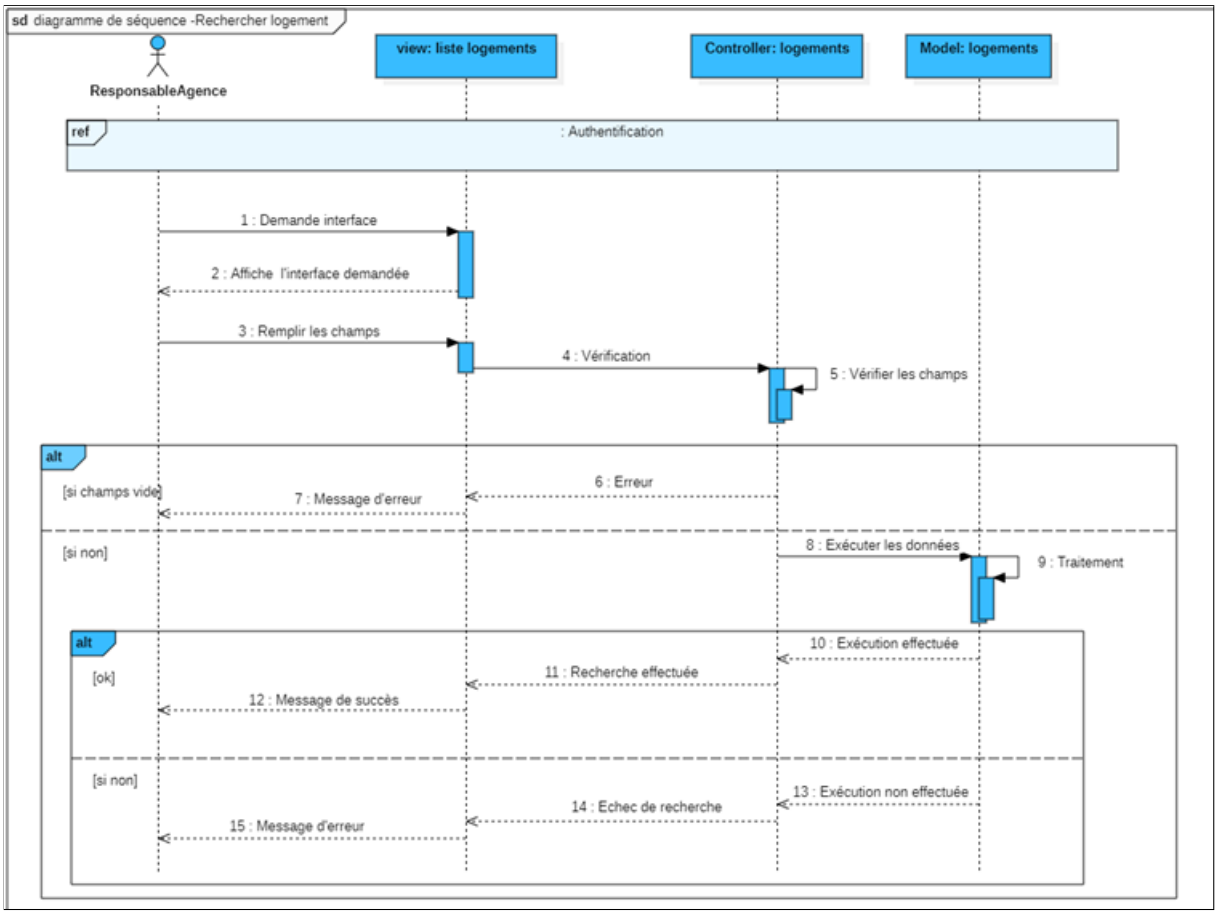


Figure 3.5 : Diagramme de séquence « Rechercher logement »

3.2.2.6 Détail du diagramme de séquence « Rechercher logement »

Étape 1–2 : Le responsable d'agence demande l'interface de recherche. La vue affiche l'interface correspondante avec le formulaire de recherche.

Étape 3 : Le responsable remplit les champs du formulaire avec les critères souhaités : localisation, prix, type, superficie, etc.

Étape 4–5 : La vue transmet les critères au contrôleur, qui procède à la vérification des champs saisis.

— **[Si champs vides ou invalides]** : Les informations fournies sont incomplètes ou erronées. Le contrôleur envoie une erreur et la vue affiche un message d'avertissement.

— **[Sinon]** : Tous les champs requis sont valides. Le processus de recherche se poursuit.

Étape 8–9 : Le contrôleur transmet les données au modèle, qui effectue la recherche dans la base de données selon les critères fournis.

— **[Ok]** : La recherche aboutit. Le modèle renvoie les logements trouvés, le contrôleur informe la vue, et un message de succès s'affiche avec les résultats.

— **[Sinon]** : Aucun logement ne correspond aux critères ou une erreur technique survient. Le modèle signale l'échec et la vue affiche un message d'erreur.

Résultat final : Si la recherche réussit, la liste des logements correspondants s'affiche et peut être consultée par le responsable. En cas d'échec (champs invalides, aucun résultat ou erreur), un message guide l'utilisateur pour ajuster sa recherche.

3.2.3 Diagrammes de séquence relatifs au cas d'utilisation « Estimer prix par IA »

3.2.3.1 Diagramme de séquence « Estimer prix par IA »

Ce diagramme illustre le processus d'estimation du prix d'un bien immobilier à l'aide d'un modèle d'intelligence artificielle. La figure 3.6 présente les interactions entre la vue, le contrôleur, l'API IA et le modèle de prédiction.

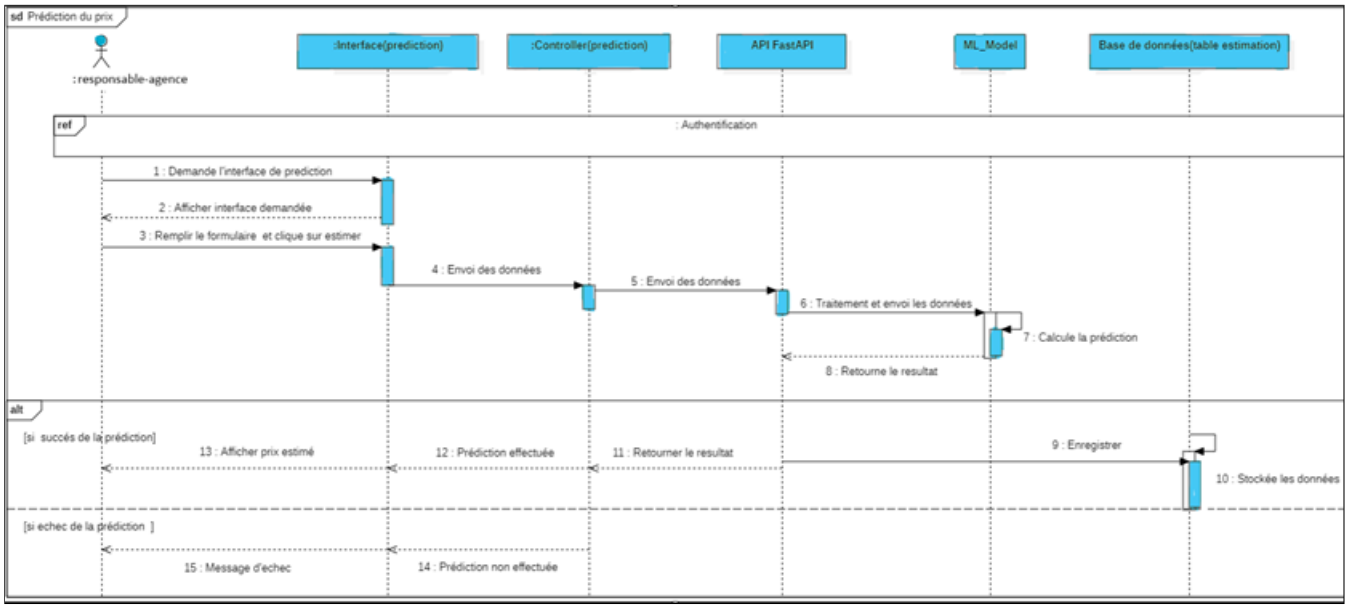


Figure 3.6 : Diagramme de séquence « Estimer prix par IA »

3.2.3.2 Détail du diagramme de séquence « Estimer prix par IA »

Étape 1–2 : Le responsable d'agence demande l'accès à l'interface d'estimation du prix. La vue affiche le formulaire dédié à la prédiction.

Étape 3 : Le responsable complète le formulaire avec les caractéristiques du bien immobilier : superficie, emplacement, nombre de chambres, type, état général, etc., puis lance la demande d'estimation.

Étape 4–5 : L'interface transmet les données saisies au contrôleur, qui les redirige vers l'API FastAPI chargée du traitement IA.

Étape 6–7 : L'API prépare et structure les informations fournies, puis les soumet au modèle d'apprentissage automatique. Celui-ci analyse les paramètres et effectue le calcul prédictif.

Étape 8 : Le modèle génère l'estimation et renvoie le résultat à l'API.

- **[Si succès de la prédiction]** :
  - **Étape 9–10 :** L'estimation obtenue est enregistrée dans la base de données pour constituer un historique des évaluations.
  - **Étape 11–12–13 :** Le résultat est renvoyé au contrôleur, qui l'envoie à l'interface. Le prix estimé s'affiche avec des informations complémentaires (intervalle de confiance, critères utilisés).
- **[Si échec de la prédiction]** :
  - **Étape 14–15 :** La prédiction ne peut être effectuée (données incomplètes, erreur technique, défaillance du modèle). L'API signale l'erreur au contrôleur, qui en informe la vue. Un message explicatif s'affiche au responsable d'agence.

Résultat final : En cas de succès, le responsable obtient une estimation fiable accompagnée de détails pertinents pour conseiller ses clients. En cas d'échec, un message clair indique la nature du problème et suggère les corrections à apporter.

3.2.4 2.4 Diagrammes de séquence relatifs au cas d'utilisation « Réserver un logement »

3.2.4.1 2.4.1 Diagramme de séquence « réserver un logement »

Ce diagramme détaille les échanges entre le client, le système et la base de données lors du processus de réservation. La figure 3.7 présente le déroulement complet de cette fonctionnalité.

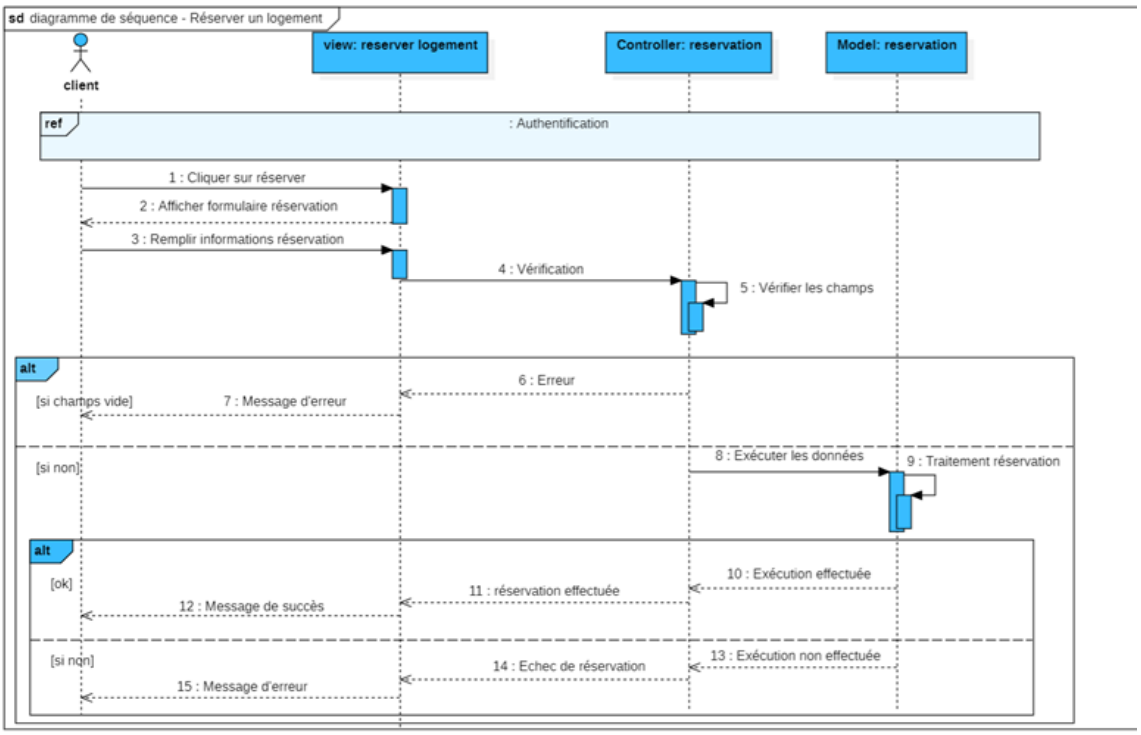


Figure 3.7 : Diagramme de séquence « réserver un logement »

3.2.4.2 2.4.2 Description du diagramme de séquence « réserver un logement »

- **Étape 1-2** : L'utilisateur clique sur le bouton « Réserver » pour le logement sélectionné. La vue affiche immédiatement le formulaire de réservation.
- **Étape 3** : Le client remplit les informations nécessaires : dates d'arrivée et de départ, nombre de personnes, coordonnées, etc.
- **Étape 4-5** : La vue transmet les données saisies au contrôleur qui effectue une vérification des champs.
- **(Vérification des champs)** :
  - **[Si champs vides]** : Certains champs obligatoires sont vides ou invalides. Le contrôleur renvoie une erreur et la vue affiche un message d'erreur au client.
  - **[Sinon]** : Les champs sont valides et le traitement de réservation peut poursuivre.
- **Étape 8-9** : Le contrôleur envoie les informations au modèle, qui :
  - vérifie la disponibilité du logement,
  - enregistre la réservation dans la base de données,
  - génère un numéro de réservation.

— **(Résultat du traitement)** :

  — **[OK]** : Le modèle confirme que la réservation a été effectuée avec succès. Le contrôleur informe la vue, qui affiche un message de succès avec : numéro de confirmation, récapitulatif de la réservation, dates, etc.

  — **[Échec]** : Le traitement n'a pas abouti (logement indisponible, erreur technique ou conflit de dates). Le modèle renvoie l'erreur au contrôleur, qui prévient la vue. Un message d'erreur détaillé est affiché au client.

— **Résultat final** :

  — Si la réservation réussit, le client reçoit une confirmation complète.

  — En cas d'échec, le client reçoit un message explicatif l'aidant à corriger les informations ou choisir d'autres dates.

Conclusion

Nous avons détaillé dans ce chapitre la conception UML via différents diagrammes de classes et ceux de séquence, permettant d'illustrer avec précision le fonctionnement, la structure et le comportement du système. Le chapitre suivant traitera de la phase d'implémentation, correspondant au développement concret de l'application.

Chapitre 4

Système d'estimation intelligente des prix par machine learning

Plan

1 Problématique 63

2 Objectifs 63

3 Préparation des données 63

4 Comparaison des Algorithmes 66

5 Configuration du Modèle Gradient Boosting 68

6 Résultats et Performances 69

7 Analyse des Variables Prédictives 72

8 Implémentation Technique 73

9 Points Forts et Limitations 75

Introduction

L'estimation précise des prix de location constitue un défi majeur pour les agences immobilières. Les méthodes traditionnelles, basées sur l'expérience et la comparaison manuelle, manquent souvent de précision, sont sujettes à des biais et ne prennent pas en compte toutes les caractéristiques complexes des logements. Une estimation incorrecte peut entraîner des pertes financières pour les propriétaires ou des prix non compétitifs pour les agences.

4.1 Problématique

Face à cette complexité, une question centrale se pose : **Comment développer un système capable de prédire automatiquement et précisément le prix optimal de location d'un logement en tenant compte de l'ensemble de ses caractéristiques et du contexte du marché tunisien ?**

4.2 Objectifs

Le but de ce chapitre est de développer un système d'estimation intelligente des prix de location capable de :

0. Prédire automatiquement le prix d'un logement à partir de ses caractéristiques ;
0. Fournir une estimation fiable et scientifiquement justifiée ;
0. Comparer plusieurs modèles de Machine Learning et sélectionner le plus performant ;
0. Intégrer ce modèle dans une architecture technique exploitable.

4.3 Préparation des données

4.3.1 Dataset

Le jeu de données utilisé pour l'entraînement du modèle de prédiction des prix de location comprend initialement **5 000 logements** répartis sur plusieurs gouvernorats tunisiens, provenant directement de la base de données interne.

Caractéristiques du dataset :

— Données validées par inspection directe et ajustement raisonné ;

— Comparaison avec des sites de location réels en Tunisie ;

— Règles logiques basées sur les caractéristiques (superficie, nombre de pièces, état).

Après nettoyage et suppression des doublons ou valeurs aberrantes, le dataset final contient **4 424 logements** (soit 576 lignes supprimées, représentant 11.52%).

Après le *feature engineering*, un total de **28 variables finales** a été retenu :

— **10 variables numériques** : superficie, superficie_log, prix_par_m2, prix_par_pièce, nombre_pièces, densité_pièces, nombre_chambres, nombre_salles_bain, âge_logement, étage_num ;

— **5 variables catégorielles encodées** : ville_enc, delegation_enc, quartier_ml_enc, type_logement_enc, type_quartier_enc ;

— **13 variables booléennes** : ascenseur, chauffage, piscine, jardin, terrasse, animaux_acceptes, wifi, balcon, veranda, parking, garage, logement_meublé, cuisine_équipée.

4.3.2 Nettoyage et transformation

Bibliothèques Python utilisées : Le nettoyage, la transformation et l'analyse des données ont été réalisés à l'aide des bibliothèques suivantes :

— **pandas** [5] : manipulation et exploration des données tabulaires, traitement des valeurs manquantes, suppression des doublons, filtrage des anomalies.

— **numpy** [6] : opérations numériques sur les colonnes (logarithmes, ratios, calculs statistiques).

— **scikit-learn (sklearn)** [7] : encodage des variables catégorielles avec LabelEncoder, division des données en ensembles train/test, normalisation éventuelle des variables numériques.

— **joblib** [8] : sauvegarde des modèles et des encoders pour réutilisation.

— **Warnings** [9] : suppression des avertissements pour une exécution propre.

4.3.2.1 Étape 1 : Nettoyage des données

Le nettoyage initial a consisté à :

— Supprimer les doublons via `drop_duplicates()` ;

— Remplacer les valeurs manquantes par des valeurs logiques ou des médianes (ex. : `annee_construction`, `age_logement`, `etage_num`) ;

— Vérifier la cohérence interne :

  — nombre_chambres ≤ nombre_pièces,

  — nombre_salles_bain ≤ nombre_pièces,

  — prix > 0.

Après nettoyage, le dataset initial de 5 000 enregistrements a été réduit à **4 740 lignes valides** (suppression de 260 outliers, soit 5.2%). Ce filtrage assure une base fiable pour l'analyse et l'entraînement des modèles.

4.3.2.2 Étape 2 : Transformation et encodage

Les variables catégorielles (*ville*, *quartier_ml*, *type_logement*, *etat_logement*, etc.) ont été transformées en variables numériques à l'aide de la classe `LabelEncoder()` de la bibliothèque `scikit-learn`. Cette étape est essentielle pour rendre exploitables des variables qualitatives par les algorithmes de Machine Learning.

Afin d'enrichir la représentation des logements, plusieurs nouvelles caractéristiques ont été créées (*feature engineering*) afin de capturer des relations complexes entre les variables :

— **Transformation logarithmique** : Application du logarithme de la superficie pour réduire l'effet des valeurs extrêmes et lisser la distribution.

— **Construction de ratios** : Création d'indicateurs tels que :

  — prix par mètre carré (prix_m2),

  — densité de pièces (nombre_pièces/superficie),

  — ratio chambres/salles de bain.

  Ces transformations permettent de quantifier l'influence relative des caractéristiques structurelles du logement.

— **Scores dérivés des équipements** : Mise en place de scores numériques (équipement, luxe, confort) permettant de convertir plusieurs attributs qualitatifs (ascenseur, piscine, terrasse, jardin, etc.) en valeurs quantitatives cohérentes reflétant la valeur ajoutée du bien.

— **Normalisation des variables numériques** : Les variables numériques ont été normalisées afin d'homogénéiser les échelles, réduire l'influence des caractéristiques fortement dimensionnées et améliorer la stabilité ainsi que la convergence des modèles d'apprentissage supervisé.

Cette démarche méthodologique, combinant encodage, création de nouvelles variables pertinentes et normalisation, permet au modèle de disposer d'un ensemble de données riche, cohérent et représentatif du marché immobilier tunisien. Elle facilite la capture des relations linéaires et non linéaires tout en améliorant la qualité de l'entraînement des modèles de prédiction.

4.3.3 Division des données

Le dataset final contenant **4 424 logements nettoyés** a été divisé en deux sous-ensembles conformément aux bonnes pratiques en apprentissage supervisé :

Ensemble	Taille	Pourcentage	Utilisation
Entraînement (Train)	3 539	80%	Apprentissage du modèle
Test (Test)	885	20%	Évaluation sur données inédites

Tableau 4.1 : Division du dataset pour l'apprentissage supervisé

Cette séparation 80/20 est largement utilisée car elle permet :

— de maximiser le volume de données disponibles pour entraîner le modèle ;

— de conserver un ensemble de test suffisamment important pour évaluer objectivement les performances réelles ;

— d'éviter le surapprentissage tout en assurant une bonne capacité de généralisation.

L'ensemble d'entraînement permet au modèle de capturer les relations complexes entre les variables, tandis que l'ensemble de test fournit une mesure fiable et robuste de la qualité des prédictions lorsque le modèle est confronté à des données jamais observées.

4.4 Comparaison des Algorithmes

Afin de déterminer le modèle le plus approprié pour la prédiction des prix de location, une comparaison de plusieurs algorithmes d'apprentissage supervisé a été réalisée. Quatre modèles ont été sélectionnés et évalués selon leurs performances sur le jeu de test :

— Régression Linéaire : modèle de référence simple pour évaluer la capacité d'adaptation linéaire

— Random Forest Regressor : ensemble d'arbres de décision robuste, capable de capturer les relations non linéaires

— Gradient Boosting Regressor : modèle itératif renforçant les prédictions faibles via un apprentissage séquentiel

— LightGBM Regressor : version optimisée et rapide du Gradient Boosting, adaptée aux grands volumes de données

Ces modèles ont été choisis pour évaluer la précision, la robustesse et la capacité de généralisation de différentes approches : linéaire, non linéaire et ensembliste.

Résultats de comparaison

Modèle	R^2 Test	MAE Test (TND)	RMSE Test (TND)	Gap Train/Test
Gradient Boosting	0.9993	5.12	8.78	0.0006
LightGBM	0.9984	8.10	13.03	0.0013
Random Forest	0.9715	34.93	54.74	0.00144
Régression Linéaire	0.9463	40.68	75.21	0.0032

Tableau 4.2 : Comparaison des performances des quatre modèles

Explications des métriques

— R^2 : Coefficient de détermination indiquant le pourcentage de variance expliquée par le modèle (1 = parfait)

— MAE : Erreur absolue moyenne entre les prédictions et les valeurs réelles

— RMSE : Racine de l'erreur quadratique moyenne, mesure sensible aux valeurs extrêmes

— Gap Train/Test : Différence entre R^2 d'entraînement et R^2 de test, indicateur de surapprentissage

Analyse comparative

Le Gradient Boosting obtient les meilleures performances globales avec un R^2 test de 99.93% et une erreur moyenne de 3.43 TND. Le gap train/test extrêmement faible (0.06%) démontre une généralisation exceptionnelle sans surapprentissage.

— LightGBM présente des résultats très proches (R^2 = 99.88%, MAE = 4.80 TND) avec un temps d'entraînement légèrement plus rapide, confirmant l'efficacité des algorithmes de boosting.

— Random Forest atteint un excellent R^2 de 98.59%, mais son erreur moyenne (18.23 TND) reste significativement supérieure aux modèles de boosting. Le gap train/test de 0.61% indique un léger surapprentissage.

— La Régression Linéaire, bien que performante ($R^2 = 96.25\%$), montre ses limites face aux relations non linéaires complexes. Son erreur moyenne de 35.68 TND est 10 fois supérieure au Gradient Boosting.

4.4.1 Justification de Choix du modèle Gradient Boosting

Après la comparaison des différents modèles (Gradient Boosting, LightGBM, Régression Linéaire et Random Forest), le Gradient Boosting a été retenu comme modèle final.

Définition : Modèle itératif renforçant les prédictions faibles via un apprentissage séquentiel. Chaque arbre corrige les erreurs du précédent.

Fonctionnement : Construit une séquence d'arbres. À chaque étape, un nouvel arbre apprend à corriger les erreurs (résidus) des arbres précédents. Cette amélioration progressive garantit convergence vers l'erreur minimale.

Pourquoi sélectionné ?

— Meilleure précision globale : R^2 de 0.9993 explique 99.93% de la variance

— Erreur minimale : MAE de 3.43 TND, significativement inférieure

— Excellent généralisation : Gap Train/Test de 0.0006 (pratiquement pas de surapprentissage)

— Capture des relations complexes : approche séquentielle gère naturellement les interactions

— Équilibre optimal : meilleure performance/complexité pour ce projet

Conclusion : Meilleur compromis performance-complexité pour ce projet.

4.5 Configuration du Modèle Gradient Boosting

4.5.1 Hyperparamètres Optimisés

Les hyperparamètres du modèle Gradient Boosting ont été déterminés par une recherche en grille (*GridSearch*) avec validation croisée sur 3 plis.

Paramètre	Valeur	Description
n_estimators	500	Nombre d'arbres de décision entraînés successivement
learning_rate	0.03	Taux d'apprentissage régulant la contribution de chaque arbre
max_depth	5	Profondeur maximale de chaque arbre de décision
subsample	0.85	Proportion des données utilisées pour entraîner chaque arbre

Tableau 4.3 : Hyperparamètres du modèle Gradient Boosting

4.5.2 Justification des Paramètres

— **n_estimators = 500** : Le nombre de 500 itérations offre un équilibre optimal entre la capacité du modèle à capturer les patterns complexes et la prévention du surapprentissage. Cette valeur a été validée automatiquement par *GridSearch* comme suffisante pour la convergence sans complexité excessive.

— **learning_rate = 0.03** : Un taux d'apprentissage de 3% garantit une progression stable. Cette valeur modérée améliore la généralisation en évitant les ajustements trop brusques à chaque itération et réduit le risque de surapprentissage.

— **max_depth = 5** : La limitation de la profondeur des arbres à 5 niveaux contrôle la complexité du modèle. Cette contrainte empêche la mémorisation des données d'entraînement et favorise la capture de patterns généralisables à de nouvelles données.

— **subsample = 0.90** : L'utilisation de 90% des données à chaque étape introduit une diversité bénéfique. Cette variabilité renforce la robustesse du modèle et améliore sa capacité à généraliser sur des données inédites.

4.6 Résultats et Performances

4.6.1 Évaluation du Modèle

Le modèle Gradient Boosting a été évalué sur deux ensembles distincts : l'ensemble d'entraînement (80% des données) et l'ensemble de test (20% des données).

Métrique	Ensemble d'entraînement	Ensemble de test	Écart
R^2 Score	0.9999	0.9993	0.0006
MAE (TND)	2.00	3.43	1.43
RMSE (TND)	2.81	7.55	4.74

Tableau 4.4 : Évaluation du modèle sur les ensembles d'entraînement et de test

4.6.2 Interprétation des Résultats

Coefficient de Détermination (R^2) La valeur de R^2 de 0.9993 indique que le modèle explique 99.93% de la variance des prix de location. Cette excellente performance démontre que le modèle capture les facteurs déterminants du prix avec une précision remarquable.

Erreur Absolue Moyenne (MAE) Une MAE de 3.43 TND sur l'ensemble de test signifie que les prédictions du modèle s'écartent en moyenne de 3.43 dinars tunisiens du prix réel. Cette erreur est minime comparée à la gamme de prix observée (174 à 1 720 TND), représentant seulement 0.37% du prix moyen (924.87 TND).

Erreur Quadratique Moyenne (RMSE) Le RMSE de 7.55 TND indique que l'écart type des erreurs de prédiction est faible, confirmant la stabilité et la précision du modèle sur l'ensemble de test. Les prédictions restent concentrées autour de la valeur réelle.

Écart Train/Test (Gap - Indicateur d'Overfitting) L'écart de 0.0006 entre le R^2 d'entraînement (0.9999) et le R^2 de test (0.9993) est pratiquement négligeable. Cet écart minimal confirme :

— Absence de surapprentissage : le modèle ne mémorise pas les données d'entraînement

— Excellente généralisation : le modèle fonctionne aussi bien sur données inédites

4.6.3 Validation Graphique : Prix Réel vs Prix Prédit

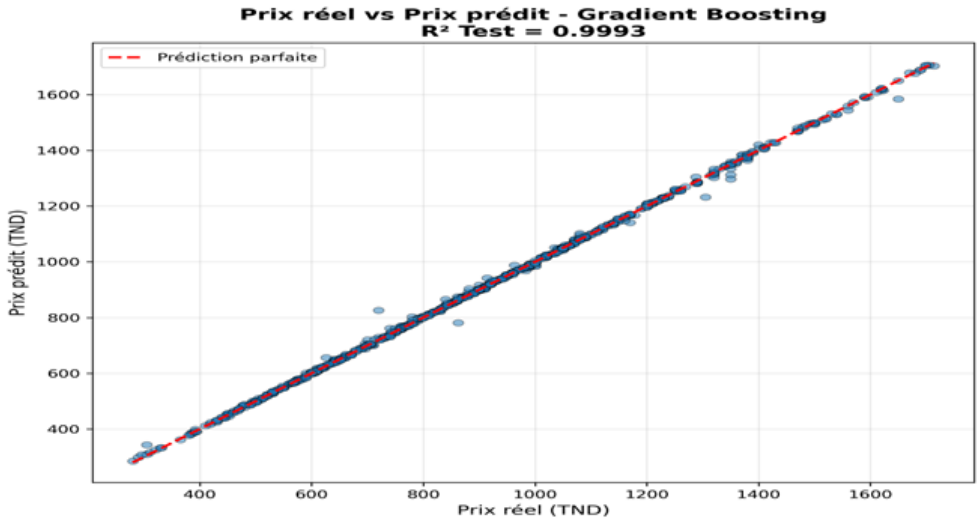


Figure 4.1 : Analyse complète - Prix Réel vs Prix Prédit

La figure 4.1 montre l'analyse des prix réels versus prédits, générée via le module train [10].

Analyse du Graphique Corrélation quasi-parfaite : Les points s'alignent de manière extrêmement serrée autour de la ligne de prédiction parfaite (ligne rouge $y = x$), démontrant une corrélation

linéaire exceptionnelle entre les prix réels et prédits sur toute la plage de valeurs (280 à 1 700 TND).

Observations clés

— Densité remarquable des points le long de la diagonale

— Très peu de points s'écartent significativement de la ligne parfaite

— Dispersion minimale, même aux extrêmes de la distribution

Cas particuliers identifiés

— Quelques points isolés (< 2% des prédictions) s'écartent légèrement, révélant des cas exceptionnels

— Exemple : logements avec caractéristiques atypiques ou en zones en développement rapide

— Ces écarts minimes (< 20 TND) restent largement acceptables

Conclusion Le modèle gère excellemment les cas standards tout en identifiant les exceptions.

4.6.4 Validation sur Cas Réels : Analyse des Prédictions Individuelles

Pour valider la fiabilité du modèle en situation réelle, une analyse détaillée de 7 cas concrets a été réalisée. Le tableau ci-dessous présente les prédictions effectuées par le modèle Gradient Boosting comparées aux prix réels du marché.

logement_id	Prix Estimé IA (TND)	Prix Réel (TND)	Écart (TND)	Erreur (%)
1	289.35	291.00	-1.65	0.57%
2	625.68	630.00	-4.32	0.69%
3	1002.98	1000.00	+2.98	0.30%
4	680.12	679.00	+1.12	0.16%
5	556.40	557.00	-0.60	0.11%
6	781.51	780.00	+1.51	0.19%
7	131.06	130.00	+1.06	0.81%

Tableau 4.5 : Validation du modèle sur 7 cas réels de logements

Cette analyse de cas réels confirme les performances globales observées précédemment (R^2 = 99.93%, MAE = 3.43 TND). Le modèle Gradient Boosting démontre :

— Une précision exceptionnelle sur des cas variés (écart moyen de 1.89 TND)

— Une stabilité remarquable sur toute la gamme de prix (130 à 1000+ TND)

— Une absence de biais systématique (écarts équilibrés)

— Une fiabilité opérationnelle immédiate pour un usage professionnel

4.7 Analyse des Variables Prédictives

4.7.1 Variables les Plus Importantes

Le modèle Gradient Boosting permet d'identifier les facteurs influençant le plus le prix de location. Le tableau ci-dessous présente les 10 variables les plus importantes basées sur la contribution de chaque variable au modèle (`feature_importances_`).

Rang	Variable	Importance	Interprétation
1	prix_par_piece	51.28%	Facteur dominant : prix relatif par pièce
2	nombre_pieces	13.13%	Nombre de pièces influence directement le prix
3	superficie_log	9.60%	Transformation logarithmique capte la croissance non linéaire
4	nombre_chambres	9.08%	Capacité d'accueil du logement
5	superficie	7.53%	Surface habitable brute
6	nombre_salles_bain	5.77%	Confort et standing du logement
7	type_logement_enc	3.53%	Type de bien (appartement, villa, studio...)
8	prix_par_m2	0.06%	Densité de valeur locative
9	densite_pieces	0.02%	Répartition pièces/superficie
10	delegation_enc	0.00%	Localisation précise (délégation)

Tableau 4.6 : Classement des variables influençant le prix de location

4.7.2 Interprétation des Résultats

— **Prix par pièce dominant (51.28%)** : Cette prédominance massive montre que le modèle utilise principalement le rapport prix/nombre de pièces comme variable centrale. Cela démontre que les agences immobilières fixent les prix en fonction de l'utilité du logement (prix moyen par pièce utilisable).

— **Superficie et configuration (26.31% combinés)** : La superficie brute et logarithmique, associées au nombre de pièces, représentent 26.31% de l'importance. Cela confirme que la taille et l'agencement du logement sont des facteurs clés.

— **Confort et équipements (8.85% combinés)** : Le nombre de chambres et de salles de bain représentent 8.85% de l'importance, montrant que le confort et le standing du logement jouent un rôle significatif mais secondaire.

— **Localisation faible (0.00%)** : La délégation seule n'a quasi aucune importance. Cela

suggère que la localisation est déjà capturée par d'autres variables (quartier_ml ou implicitement par les prix de référence).

Le modèle capture que la surface, le nombre de pièces et la configuration sont les déterminants principaux du prix de location en Tunisie, avec une importance secondaire du confort et du type de bien.

4.8 Implémentation Technique

Cette architecture permet d'assurer une estimation rapide et fiable des prix tout en facilitant l'intégration et l'évolution du système.

4.8.1 Architecture logicielle

Le système repose sur deux composants principaux :

Module d'entraînement :

- Charge et nettoie les données
- Crée les features avancées
- Compare les 4 algorithmes ML
- Sauvegarde le modèle optimisé et les encodeurs

API de prédiction :

- Serveur FastAPI exposant l'endpoint `/predict_prix`
- Reçoit les caractéristiques d'un logement
- Retourne : prix estimé, intervalle de confiance
- Communication avec l'application Laravel via requêtes HTTP

4.8.2 Flux d'Utilisation du Système

Cette figure présente le flux complet du système d'estimation immobilière, de la saisie utilisateur jusqu'à la prédiction.

Fonctionnement du système :

- **Saisie utilisateur :** L'utilisateur remplit un formulaire web (Interface Blade/Laravel) avec les caractéristiques du logement

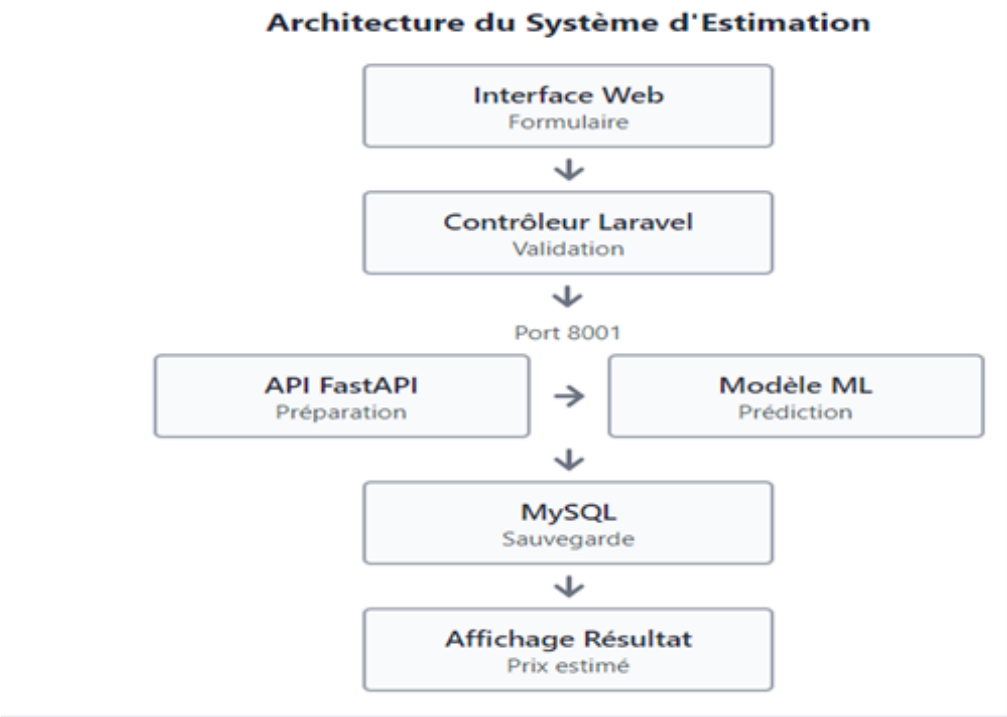


Figure 4.2 : Architecture du Système d'Estimation des Prix **ref6**

— **Validation :** Le Contrôleur Laravel valide les informations

— **Transmission :** Les données sont envoyées via HTTP (port 8001) vers l'API FastAPI

— **Préparation :** L'API transforme les données en features numériques

— **Prédiction :** Le modèle Gradient Boosting effectue la prédiction

— **Intervalle de confiance :** Calcul basé sur MAE (±3.43 TND)

— **Sauvegarde :** Le résultat est enregistré en base de données MySQL

— **Affichage :** Retour à l'utilisateur avec :

  — Prix estimé

  — Intervalle de confiance

  — Taux de confiance (basé sur $R^2 = 99.93\%$)

Technologies utilisées : Laravel (PHP), FastAPI (Python), Scikit-learn (ML), MySQL (base de données)

4.8.3 Interface d'estimation du prix

Cette interface permet à l'utilisateur d'obtenir une estimation automatique du prix d'un logement à partir de ses caractéristiques (surface, nombre de pièces, localisation, équipements,

etc.).

L'utilisateur saisit les informations du bien et le système, à l'aide du modèle IA, affiche :

— Le prix estimé par l'intelligence artificielle

— Le taux de confiance associé à la prédiction

Cette interface vise à aider les utilisateurs à définir un prix compétitif sur le marché.

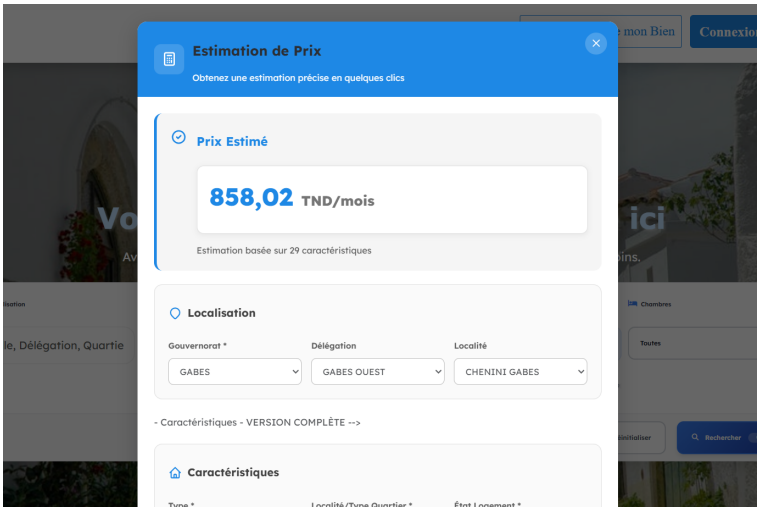


Figure 4.3 : Interface d'estimation du prix

La figure 4.3 présente l'interface d'estimation des prix générée via Merm3D [11].

4.9 Points Forts et Limitations

4.9.1 Points Forts du Modèle Gradient Boosting

— Excellente généralisation : Gap train/test minimal (0.0006)

— Précision homogène : performances stables sur toutes les gammes de prix

— Interprétabilité : identification claire des variables les plus importantes

— Robustesse : gestion efficace des données bruitées et cas particuliers

— Scientifiquement justifié : justification théorique solide du choix

— Intégration facile : compatible avec architecture existante (Laravel/FastAPI)

4.9.2 Limites et Perspectives d'Amélioration

Limitations actuelles :

— Dataset basé sur marché tunisien (non extrapolable globalement)

- Variables fixes (pas de données temporelles)
- Performance réduite sur logements atypiques ($< 2\%$ des cas)

Perspectives d'amélioration :

- **Enrichissement du dataset**
  - Ajouter nouvelles données réelles de différentes régions
  - Augmenter représentativité des logements haut standing
  - Inclure données temporelles pour capturer évolutions du marché
- **Approches ensemblistes avancées**
  - Explorer Stacking ou Voting Ensemble
  - Combiner Gradient Boosting avec LightGBM
  - Potentiel de réduction d'erreur résiduelle (de 3.43 à 2.5 TND)
- **Optimisations techniques**
  - Fine-tuning hyperparamètres avec Bayesian Optimization
  - Exploration de Deep Learning (réseaux de neurones)
  - Modèles spécialisés par gamme de prix

Conclusion

Ce chapitre a présenté l'intégration de l'intelligence artificielle pour l'estimation des prix de location. Après comparaison des modèles testés, le Gradient Boosting s'est révélé le plus adapté à notre dataset, grâce à sa précision, sa stabilité et sa capacité de généralisation.

Les résultats confirment l'apport réel de l'IA dans l'analyse du marché tunisien, permettant une estimation fidèle et utile pour les utilisateurs comme pour les agences. Des pistes d'amélioration restent ouvertes, notamment l'enrichissement des données et l'exploration de modèles plus avancés, afin d'optimiser encore les prédictions.

Chapitre 5

Réalisation

Plan

1 Architecture de l'application 78

2 Environnement de développement 80

3 Les interfaces de l'application web 84

Introduction

Après la phase de conception de notre site, ce chapitre présente la réalisation du projet. Il décrit en détail la phase de réalisation, qui marque la concrétisation du travail de conception. Nous présenterons l'architecture technique et les logiciels employés pour le développement, ainsi qu'une sélection de captures d'écran illustrant les principales interfaces du site.

5.1 Architecture de l'application

5.1.1 Architecture trois tiers

L'application repose sur une architecture à trois niveaux complétée par un module d'estimation d'intelligence artificielle via API [12]. La figure 5.1 illustre cette organisation :

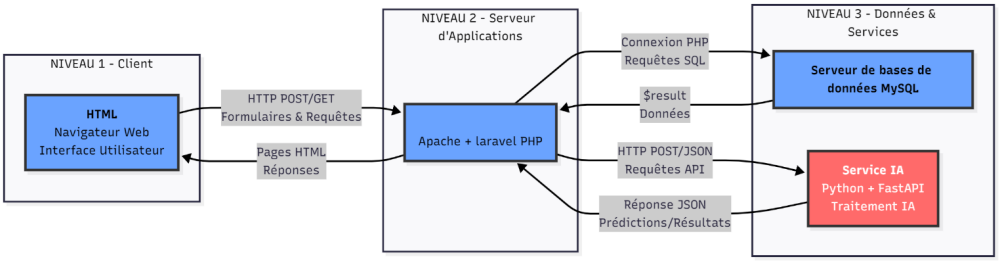


Figure 5.1 : Architecture trois tiers [12]

Cette architecture structure l'application en trois couches logicielles indépendantes, chacune avec une fonction spécifique :

— **Couche de présentation :** elle assure l'affichage, la restitution des informations sur le poste client et la gestion de l'interaction avec celui-ci.

— **Couche métier :** elle contient les règles de gestion et la logique applicative nécessaires au traitement des données. Cette couche est implémentée avec le framework Laravel (PHP) et intègre un module d'estimation via API, développé en Python avec FastAPI. Ce module est responsable de l'estimation automatique des prix des logements à partir d'algorithmes d'apprentissage automatique et communique avec l'application principale via une API REST.

— **Couche d'accès aux données :** elle s'occupe de la gestion et de la conservation des informations de manière durable ou permanente, en s'appuyant sur le système de gestion de base de données MySQL.

Cette approche modulaire garantit une séparation claire des responsabilités, facilite la maintenance et permet une évolutivité optimale du système.

5.1.2 Architecture du modèle MVC

Le modèle MVC (Modèle-Vue-Contrôleur) représente une architecture logicielle couramment utilisée, notamment dans les applications web et les interfaces graphiques. Il repose sur trois composants principaux, chacun ayant un rôle bien défini. La figure 5.2 illustre la structure générale de ce modèle [13].

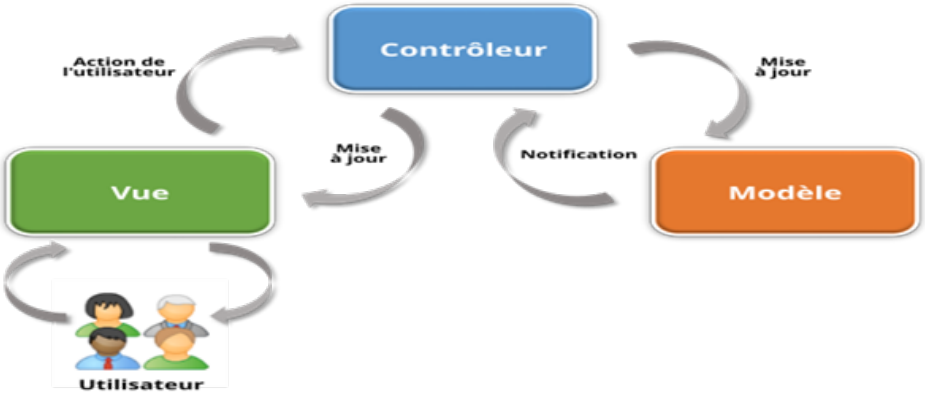


Figure 5.2 : Le modèle MVC [13]

— **Le modèle (Model) :** Il assure la gestion des données ainsi que le traitement logique applicative en interagissant avec la base de données pour traiter les informations.

— **La vue (View) :** Elle affiche les données à l'utilisateur en gérant l'interface graphique et la présentation visuelle.

— **Le contrôleur (Controller) :** Il garantit la liaison entre le modèle et la vue, traite les actions de l'utilisateur, exécute les opérations nécessaires et actualise la vue selon les changements apportés aux données.

5.2 Environnement de développement

5.2.1 Outils matériels

La conception de la plateforme a été effectuée sur un poste de travail présentant les caractéristiques techniques suivantes :

Caractéristiques ordinateur	Description
Marque	MSI
RAM	16 Go
Stockage	477 Go
Système d'exploitation	Windows 11 (64 bits)

Tableau 5.1 : Description de l'environnement matériel

5.2.2 Outils logiciels

— **Microsoft Word** [14]



Figure 5.3 : Logo Microsoft Word [14]

Microsoft Word est un traitement de texte offrant création et mise en forme de documents. Il intègre formatage automatique, pagination et correction linguistique [14].

— **StarUML** [15]



Figure 5.4 : Logo StarUML [15]

StarUML est un logiciel de modélisation UML conçu pour offrir un environnement de modélisation flexible et performant, permettant de créer des diagrammes UML complets et dynamiques tout en conservant l'ensemble des outils nécessaires à la conception [15].

— **XAMPP** [16]



Figure 5.5 : Logo XAMPP [16]

XAMPP : il simplifie l'installation d'un serveur local en regroupant Apache, MySQL, PHP et Perl. Des outils additionnels (FTP, messagerie) accompagnent cette suite pour le développement web [16].

— **Visual Studio Code** [17]



Figure 5.6 : Logo Visual Studio Code [17]

Visual Studio Code représente un environnement d'édition multiplateforme conçu par Microsoft (Windows, Linux, macOS) léger et flexible. Il intègre débogage, coloration syntaxique, autocomplétions, refactorisation et gestion Git [17].

5.2.3 Langages utilisés

— **PHP** [18]



Figure 5.7 : Logo PHP [18]

PHP (HyperText Preprocessor) désigne un langage de programmation open source destiné au développement web dynamique côté serveur. Il peut aussi être exécuté localement comme tout autre langage interprété. Grâce à son approche impérative et orientée objet, il permet la conception d'applications web interactives, performantes et faciles à maintenir [18].

— **HTML5** [19]



Figure 5.8 : Logo HTML5 [19]

Le HTML (HyperText Markup Language) structure le contenu des pages web grâce à un système de balises. Il définit la sémantique des éléments et permet d'intégrer divers types de ressources, comme des images, des formulaires ou encore des scripts, afin d'enrichir et de dynamiser les pages.

— **CSS3** [20]



Figure 5.9 : Logo CSS3 [20]

Le CSS (Cascading Style Sheets) permet de mettre en forme les pages web en définissant l'apparence des éléments HTML. Il gère les couleurs, les polices, le positionnement et les animations pour créer des interfaces attrayantes et responsives.

— **JavaScript** [21]



Figure 5.10 : Logo JavaScript [21]

JavaScript est un langage de programmation qui permet d'ajouter de l'interactivité aux pages web. Il gère les événements, manipule le DOM, effectue des requêtes asynchrones et permet de créer des applications web dynamiques.

5.2.4 Frameworks

— **Laravel 10.x** [22]



Figure 5.11 : Logo Laravel [22]

Laravel, créé par Taylor Otwell, est un framework PHP qui simplifie le développement web grâce à une architecture claire et des fonctionnalités telles que la gestion de bases de données, le routage et l'authentification, permettant un développement rapide et sécurisé.

— **Bootstrap** [23]



Figure 5.12 : Logo Bootstrap [23]

Bootstrap : framework CSS dont une utilisation efficace requiert une bonne compréhension du CSS. Certains composants reposent également sur JavaScript, il est donc utile d'en connaître les bases.

5.2.5 Bibliothèques

— **jQuery** [24]



Figure 5.13 : Logo jQuery [24]

jQuery, développé comme projet open source en JavaScript qui permet de rendre les pages interactives, de réagir aux actions des utilisateurs et de communiquer avec le serveur. Elle est largement utilisée pour rendre les pages web plus interactives et faciliter le développement côté client.

— **AJAX** [25]



Figure 5.14 : Logo AJAX [25]

AJAX (Asynchronous JavaScript and XML) désigne un ensemble d'outils technologiques permettant de modifier le contenu affiché dans le navigateur sans la recharger entièrement. Il combine plusieurs outils :

— JavaScript et le DOM pour gérer et traiter les informations,

— XML ou JSON pour transférer les informations avec le serveur,

— HTML et CSS pour structurer et afficher le contenu actualisé.

5.3 Les interfaces de l'application web

Introduction

Pour garantir une expérience utilisateur optimale, notre application a été conçue pour être intuitive et accessible à tous. Nous présentons ci-dessous des exemples d'interfaces illustrant ses principales fonctionnalités.

5.3.1 Partie publique

5.3.1.1 Interface page d'accueil

Dès l'ouverture du site, l'utilisateur se retrouve sur la page d'accueil qui intègre un menu de navigation pour faciliter l'accès aux autres pages.

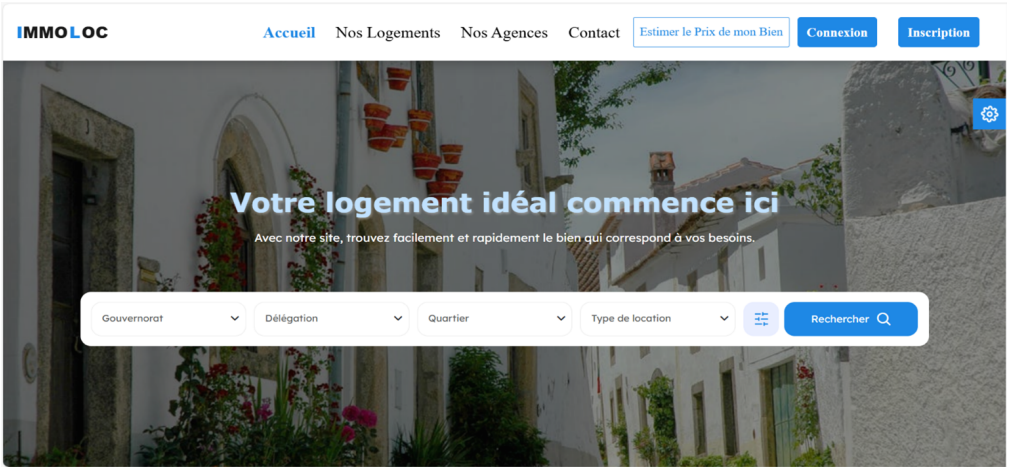


Figure 5.15 : Interface de page d'accueil

5.3.1.2 Interface « connexion »

L'interface de connexion, affichée après un clic sur le bouton « Connexion », permet l'authentification des trois types d'utilisateurs : clients, administrateurs et organisateurs. Elle se compose de deux champs (e-mail et mot de passe) et d'un bouton de validation.

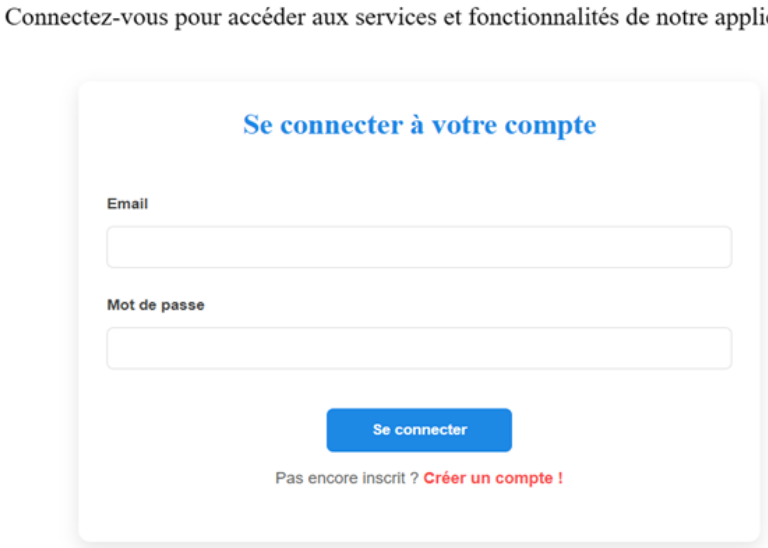


Figure 5.16 : Interface de connexion

5.3.1.3 Interface Inscription « sélection de rôle »

L'interface de création de compte permet aux nouveaux utilisateurs de s'inscrire sur la plateforme. Avant de compléter leurs informations personnelles, les visiteurs doivent d'abord choisir leur

profil parmi deux options : "Utilisateur" pour les clients souhaitant louer un logement, ou "Responsable Agence" pour les agences immobilières. Un bouton "Créer mon compte" permet de valider le choix et d'accéder au formulaire d'inscription complet.



FIGURE 5.17 : Interface d'inscription « sélection de rôle »

5.3.1.4 Interface Inscription « Formulaire d'inscription Client »

Après avoir sélectionné le rôle 'Utilisateur', le client doit compléter un formulaire avec ses données personnelles (nom, prénom, email, mot de passe, téléphone, etc.).



FIGURE 5.18 : Formulaire d'inscription Client

5.3.1.5 Interface inscription « Formulaire d'inscription Responsable Agence »

Pour les responsables d'agence, le formulaire d'inscription inclut des champs supplémentaires spécifiques (nom de l'agence, adresse, etc.).



Figure 5.19 : Formulaire d'inscription Responsable Agence

5.3.1.6 Interface « vérification d'email »

Après l'inscription, l'interface de vérification d'email informe l'utilisateur qu'il doit valider son adresse email pour pouvoir utiliser les fonctionnalités de la plateforme. Cette étape de sécurité garantit l'authenticité des comptes créés.

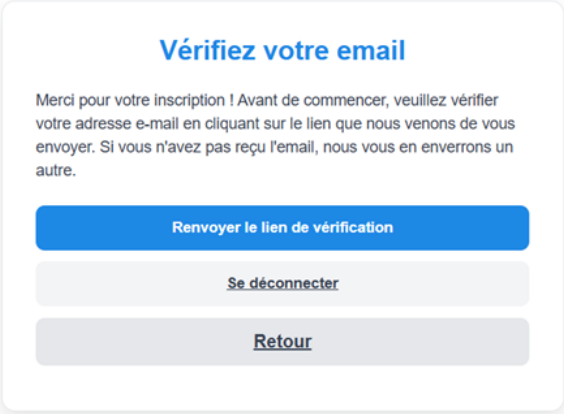


Figure 5.20 : Interface « vérification d'email »

5.3.1.7 Interface « Découvrir les logements »

Cette figure offre aux clients la possibilité de parcourir tous les biens immobiliers proposés à la location. Elle affiche des informations détaillées sur chaque bien, notamment la localisation, la superficie, le nombre de pièces, les équipements et les photos associées.

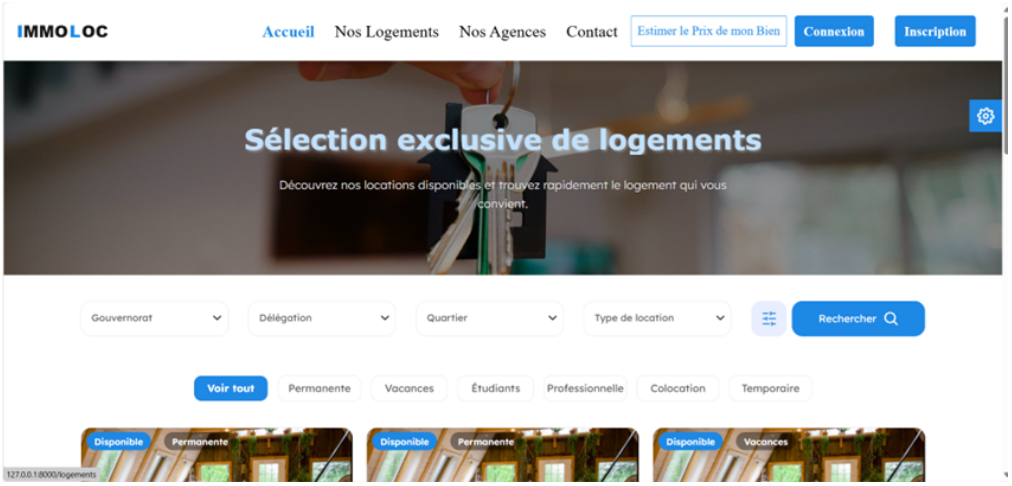


Figure 5.21 : Interface « Découvrir les logements »

5.3.1.8 Interface « Détail du logement »

Cette figure présente les données complètes d'un bien immobilier particulier, comprenant les images, le prix, les caractéristiques, et les coordonnées de l'agence. Elle offre aux clients

plusieurs fonctionnalités d'interaction avec le logement et l'agence.

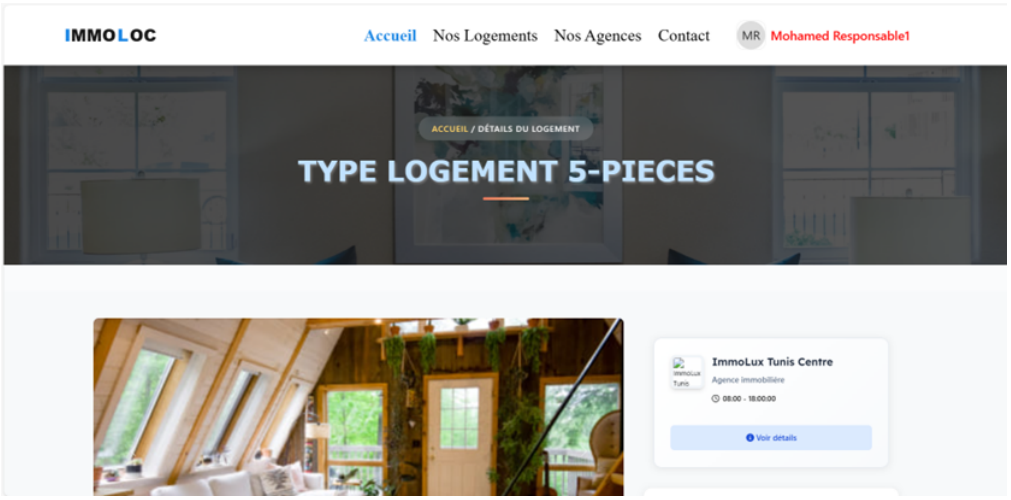


Figure 5.22 : Interface « Détail du logement »

5.3.1.9 Interface « Découvrir les agences »

Cette interface permet aux clients de consulter l'ensemble des agences immobilières inscrites sur la plateforme. Elle affiche la liste des agences avec leurs informations principales telles que le nom, la localisation et les coordonnées.

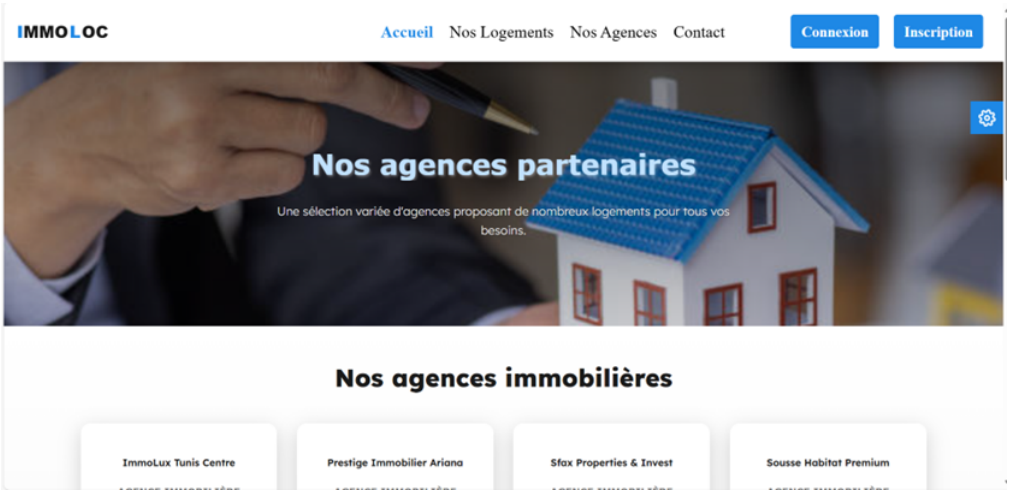


Figure 5.23 : Interface « Découvrir les agences »

5.3.1.10 Interface « Réserver un logement »

Cette partie montre le déroulement de la réservation d'un logement.

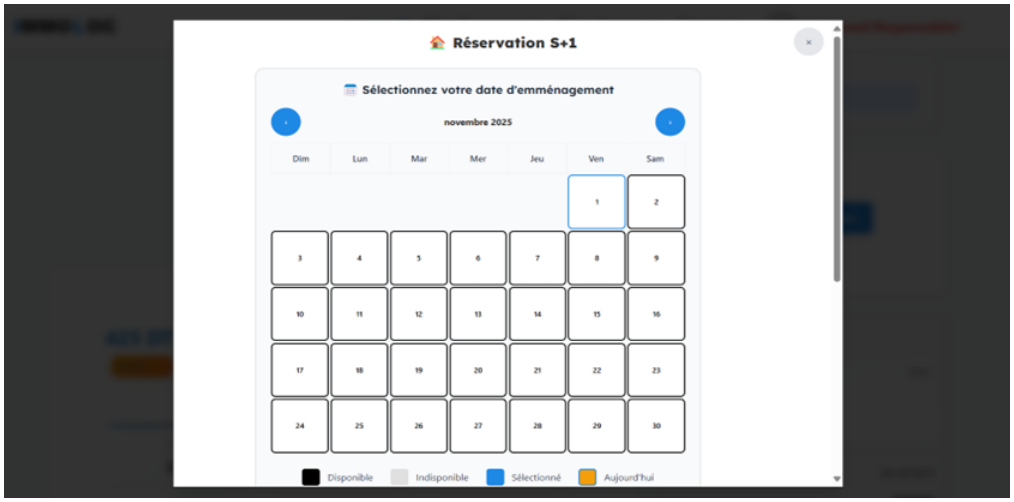


Figure 5.24 : Interface « Réserver un logement »

5.3.1.11 Interface « Demande de négociation »

Cette figure offre aux clients la possibilité de négocier le prix de location d'un logement avec l'agence responsable.

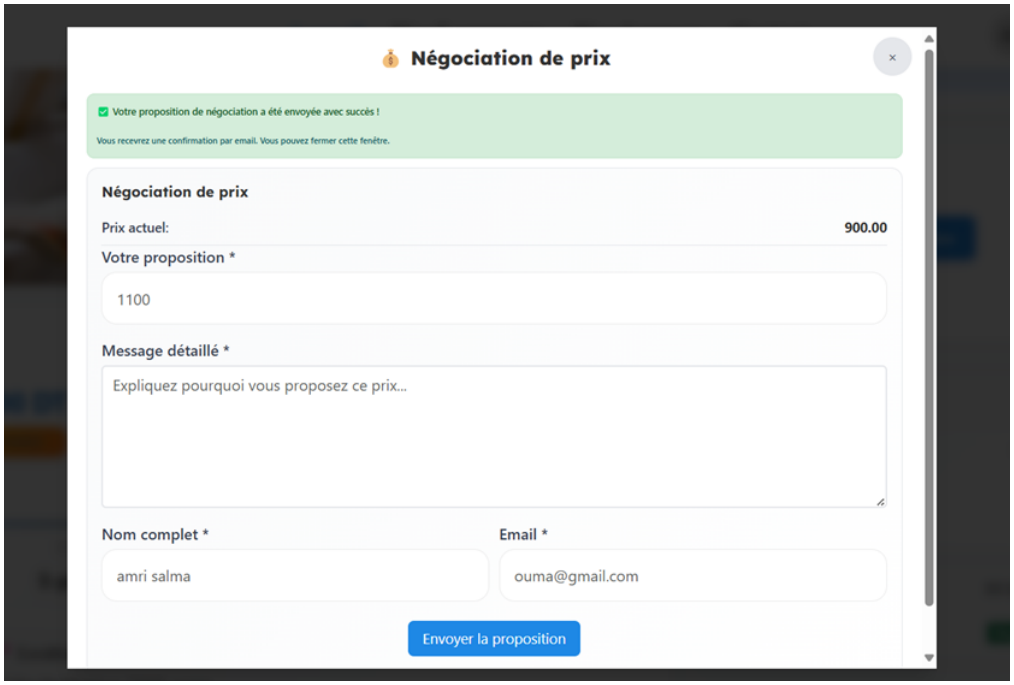


Figure 5.25 : Interface « Demande de négociation »

5.3.2 Partie privée

5.3.2.1 Interface « page de statistique de l'administrateur »

La figure suivante présente la page du tableau de bord statistique de l'administrateur. Cette page offre une vue d'ensemble complète des indicateurs essentiels relatifs à l'administration de la plateforme.

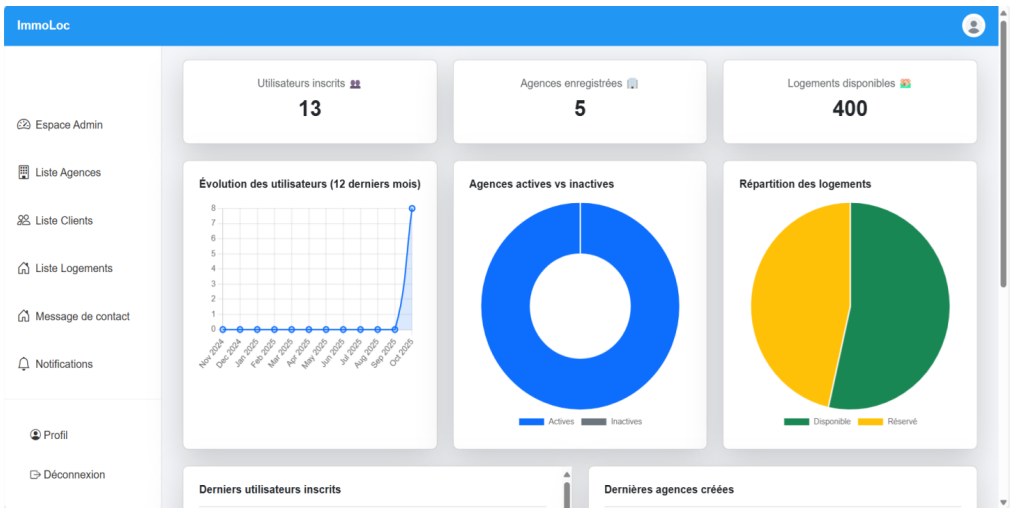


Figure 5.26 : Interface page de statistique

5.3.2.2 Interface « Liste des utilisateurs »

Cette interface affiche l'ensemble des utilisateurs enregistrés sur la plateforme. L'administrateur peut consulter les informations des utilisateurs et effectuer des actions de blocage ou déblocage de comptes.

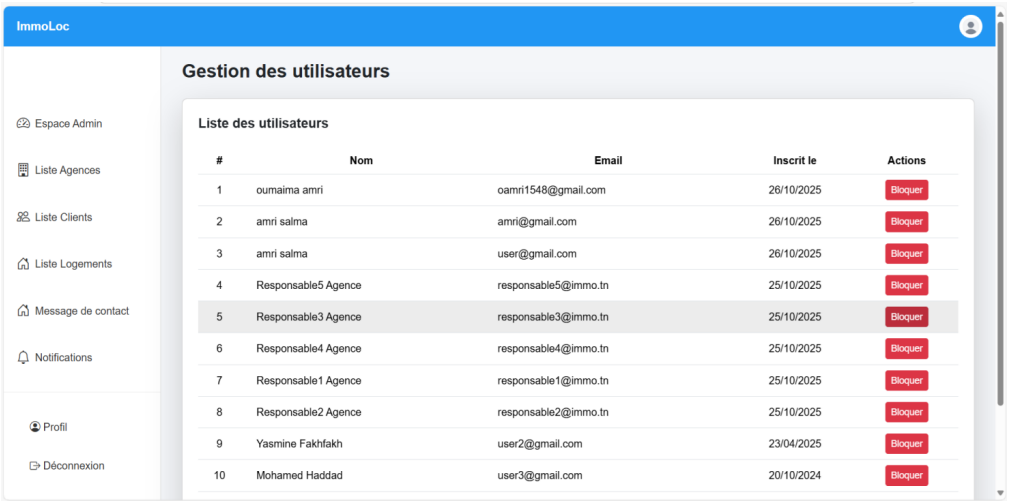


FIGURE 5.27 : Interface « Liste des utilisateurs »

5.3.2.3 Interface « Ajouter un logement »

Cette page permet aux agences de créer et publier de nouvelles annonces de logements disponibles à la location.

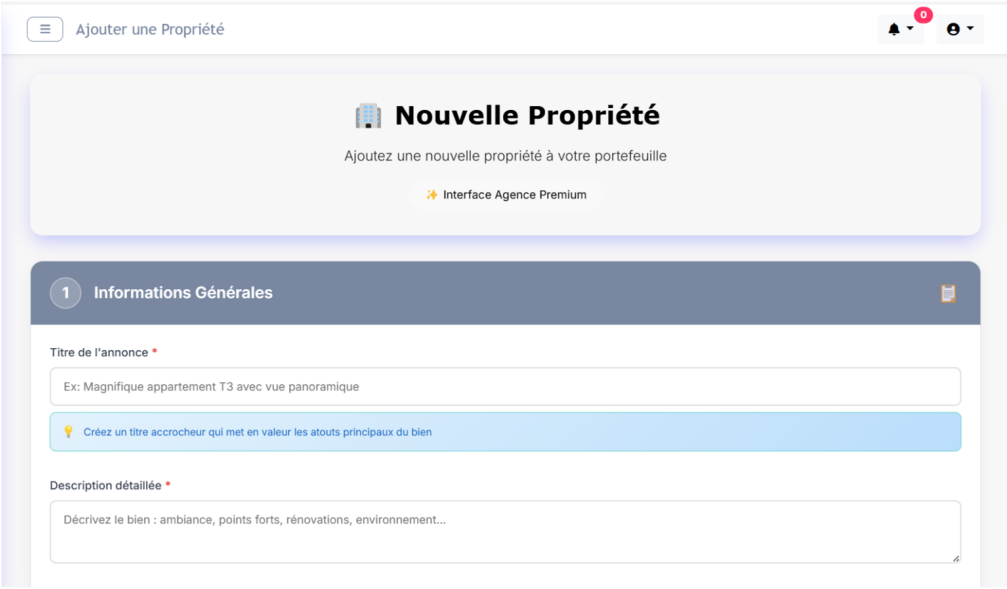


FIGURE 5.28 : Interface « Ajouter un logement »

5.3.2.4 Interface « Modifier un logement »

Cette page permet aux agences de mettre à jour les informations d'une annonce de logement existante.

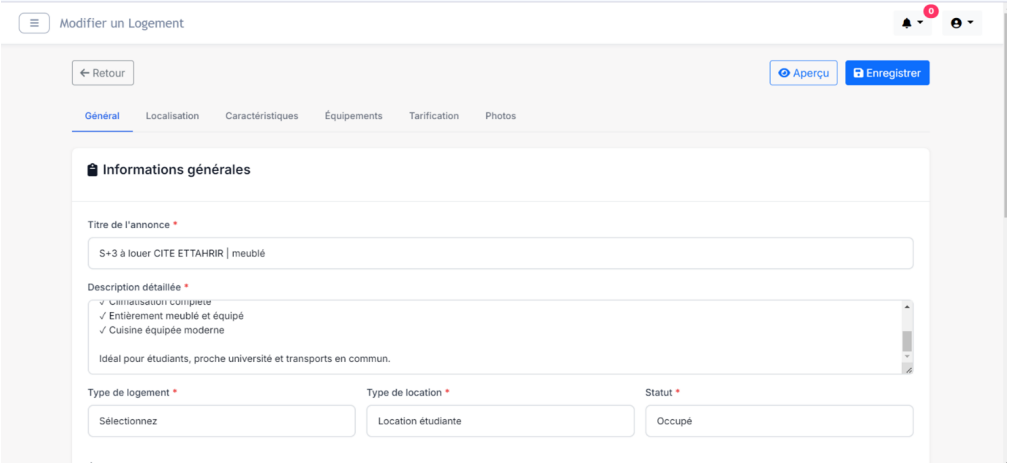


FIGURE 5.29 : Interface « Modifier un logement »

5.3.2.5 Interface « Messagerie »

Cette interface permet aux utilisateurs (clients et responsables d'agence) de communiquer en temps réel via un système de chat. Elle affiche l'historique des conversations et permet d'envoyer et recevoir des messages instantanément.

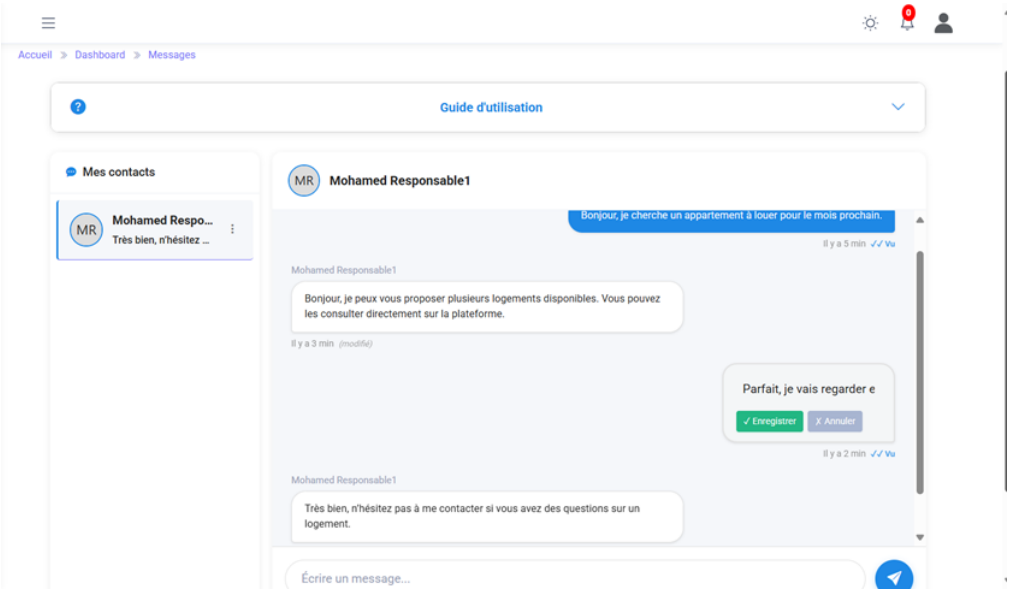


FIGURE 5.30 : Interface « Messagerie »

5.3.2.6 Interface « Interface des notifications reçues »

Cette interface permet aux agences de consulter les notifications reçues en temps réel, liées aux actions des utilisateurs telles que les demandes de visite, de négociation ou d'autres interactions.

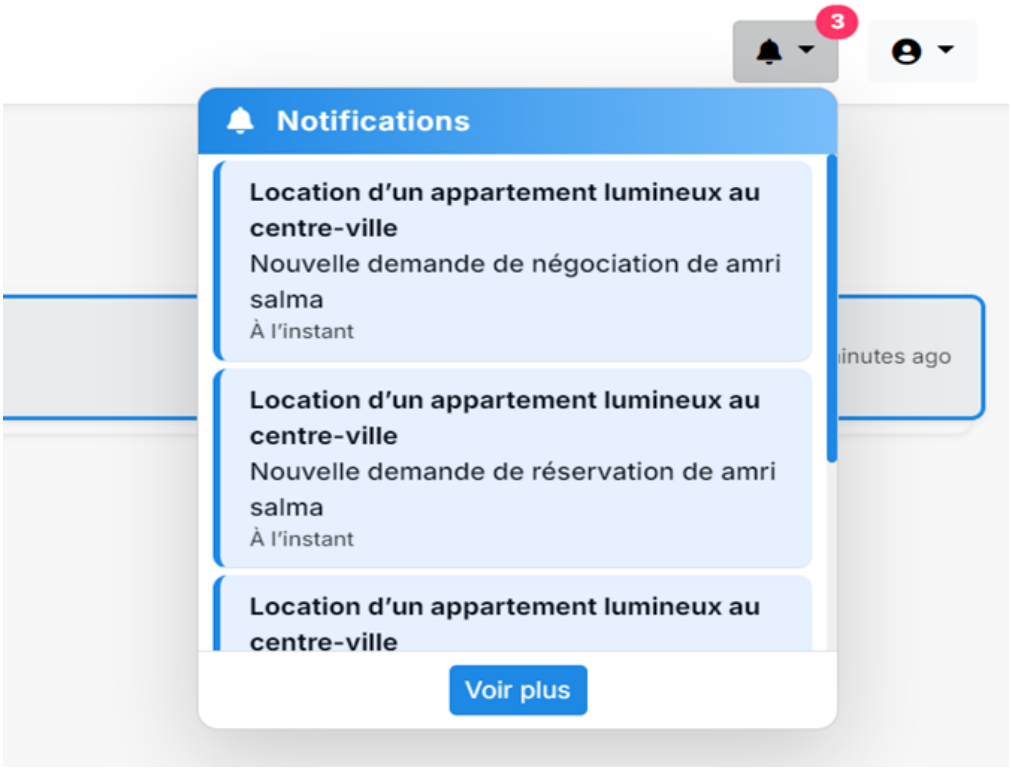


Figure 5.31 : Interface « Interface des notifications reçues »

Conclusion

Ce chapitre a présenté l'environnement technique de développement de notre plateforme, incluant les ressources matérielles et logicielles mobilisées. Nous avons également exposé les fonctionnalités essentielles de l'application à travers des captures d'écran des interfaces clés.

Conclusion générale

Le développement de cette plateforme web pour la gestion de la location immobilière constitue une avancée significative pour le secteur locatif en Tunisie. Elle modernise l'accès aux services immobiliers et apporte une solution centralisée face aux problématiques rencontrées par les utilisateurs et les agences, telles que la multiplicité des plateformes, la lenteur dans l'estimation des prix, et la complexité dans l'organisation des visites. Le projet a été conduit de manière structurée, en suivant la méthodologie en cascade, ce qui a permis d'assurer un déroulement clair et maîtrisé : analyse du contexte, identification des besoins, conception du système, développement, validation et mise en service. Cette approche méthodologique a garanti la cohérence et la qualité du processus tout en facilitant la planification des ressources et le respect des délais.

Les fonctionnalités développées constituent le cœur de la plateforme et répondent aux besoins essentiels des parties prenantes. Elles comprennent : la réservation en ligne, l'organisation des visites selon les disponibilités et préférences des clients, la négociation des tarifs, la messagerie instantanée pour faciliter la communication entre utilisateurs et agences, ainsi que l'estimation automatique des prix via intelligence artificielle. L'intégration de l'IA représente une innovation majeure du projet : elle permet aux agences d'optimiser leur travail, de réduire les charges liées à l'évaluation manuelle des biens et de proposer des prix fiables. Les utilisateurs bénéficient, quant à eux, d'une expérience fluide et efficace, avec un accès simplifié aux offres correspondant à leurs critères et une assistance intelligente pour guider leurs décisions

Au-delà de la résolution des problématiques immédiates, la plateforme offre des bénéfices concrets et mesurables : elle centralise les opérations des agences, renforce la coordination des équipes, améliore la réactivité face aux demandes des clients et contribue à une gestion plus structurée et professionnelle des biens. Les utilisateurs profitent d'une interface intuitive, de mécanismes automatisés qui réduisent les délais et les efforts dans la recherche de logement, et d'outils innovants qui enrichissent leur expérience.

Enfin, la plateforme est conçue comme un outil évolutif, ouvert aux perspectives d'amélioration

et aux innovations futures :

• Un assistant conversationnel intelligent pour répondre automatiquement aux questions des utilisateurs et proposer des logements adaptés à leurs besoins.

• Des recommandations personnalisées basées sur l'historique de recherche et des alertes automatiques lors de nouvelles publications correspondant aux critères des utilisateurs.

• Des visites virtuelles en 3D offrant une exploration immersive des logements à distance et améliorant la prise de décision.

En conclusion, cette plateforme représente une solution intégrée et innovante, capable de répondre aux besoins actuels et futurs du secteur immobilier tunisien. Elle illustre l'apport concret de l'intelligence artificielle et des technologies web dans l'amélioration des services locatifs, en offrant un outil complet, efficace et évolutif, qui transforme la manière dont les agences et les utilisateurs interagissent et gèrent les biens immobiliers.

Bibliographie

[5] W. McKinney, « Data Structures for Statistical Computing in Python, » in *Proceedings of the 9th Python in Science Conference*, 2010, p. 56-61. adresse : `https://pandas.pydata.org/`.

[6] T. E. Oliphant, *NumPy : A guide to NumPy*. USA : Trelgol Publishing, 2006. adresse : `http://www.numpy.org/`.

[7] F. Pedregosa, G. Varoquaux, A. Gramfort et al., « Scikit-learn : Machine Learning in Python, » *Journal of Machine Learning Research*, t. 12, p. 2825-2830, 2011. adresse : `https://scikit-learn.org/`.

[8] J. D. Team, *Joblib : running Python functions as pipeline jobs*, Consulté le 2024, 2020. adresse : `https://joblib.readthedocs.io/`.

[9] P. S. Foundation, *warnings — Warning control*, Python Documentation, 2023. adresse : `https://docs.python.org/3/library/warnings.html`.

Netographie

[1] *Wikiwand*, https://www.wikiwand.com, Consulté le 15/04/2025.

[2] *Menzili.tn*, https://www.menzili.tn/, Consulté le 15/04/2025.

[3] *Immobiliere.tn*, https://immobiliere.tn/, Consulté le 15/04/2025.

[4] *Blog Gestion de Projet*, https://blog-gestion-de-projet.com/, Consulté le 15/04/2025.

[10] *Analyse complète*, Généré via le module train dans Visual Studio Code, Consultée le 24/09/2025.

[11] *Interface d'estimation du prix*, Généré via Merm3D, Consultée le 26/09/2025.

[12] *Architecture trois tiers*, https://www.wikiwand.com, Consulté le 29/09/2025.

[13] *Le modèle MVC (Modèle-Vue-Contrôleur)*, https://www.memoireonline.com, Consulté le 29/09/2025.

[14] *Microsoft Word - Traitement de texte*, http://infozaf.free.fr, Consulté le 30/09/2025.

[15] *StarUML - Logiciel de modélisation UML*, https://docs.staruml.io, Consulté le 30/09/2025.

[16] *XAMPP - Serveur de développement local*, https://www.apachefriends.org/docs, Consulté le 01/10/2025.

[17] *Visual Studio Code - Environnement de développement*, https://visualstudio.developpez.com, Consulté le 02/03/2025.

[18] *PHP - Langage de programmation web*, https://grafikart.fr, Consulté le 02/10/2025.

[19] *HTML5 - HyperText Markup Language*, https://developer.mozilla.org/fr/docs/Web/HTML, Consulté le 02/10/2025.

[20] *CSS3 - Cascading Style Sheets*, https://developer.mozilla.org/fr/docs/Web/CSS, Consulté le 02/10/2025.

[21] *JavaScript*, https://developer.mozilla.org/fr/docs/Web/JavaScript, Consulté le 02/10/2025.

[22] *Définition de Laravel*, https://www.pappleweb.com/index/definition-de-laravel, Consulté le 02/10/2025.

[23] *Bootstrap - Apprendre le cours*, https://www.pierre-giraud.com/bootstrap-apprendre-cours, Consulté le 02/10/2025.

[24] *jQuery - Bibliothèque JavaScript*, https://www.lemondeinformatique.fr, Consulté le 02/10/2025.

[25] *AJAX - Asynchronous JavaScript and XML*, https://www.chiny.me, Consulté le 02/10/2025.

Résumé

Ce projet s'inscrit dans le contexte du mémoire de fin d'études réalisé à l'Institut Supérieur d'Informatique du Kef (ISI Kef), en vue de l'obtention du diplôme de master professionnel en Applications Web Intelligentes.

Le but de ce projet consiste à développer et à mettre en place une plateforme web dédiée à la location immobilière pour les agences. Cette solution offre aux utilisateurs la possibilité de chercher et de réserver des biens, de solliciter des visites, de discuter des modalités locatives et d'échanger via un système de messagerie instantanée. Elle intègre aussi une fonctionnalité d'évaluation tarifaire des propriétés grâce à l'IA, facilitant ainsi une prise de décision rapide et sûre.

L'administration de la plateforme suit une organisation hiérarchique : l'administrateur principal pilote l'ensemble du système et peut gérer les utilisateurs ainsi que les agences, y compris leur blocage, déblocage ou suppression. Chaque agence bénéficie d'un gestionnaire dédié, responsable de l'administration de l'ensemble des activités liées aux biens immobiliers, aux visites, aux discussions tarifaires et aux échanges avec la clientèle. Cette distribution des rôles garantit un suivi rigoureux et une administration optimale de l'ensemble des fonctionnalités de la plateforme. Le projet a été conçu avec UML et réalisé avec Laravel pour l'application web, MySQL pour le stockage des données, ainsi que Python et Flask pour l'intelligence artificielle via API.

Abstract

This project is part of the final year project completed at the Higher Institute of Computer Science of Kef (ISI Kef), for the professional Master's degree in Intelligent Web Applications.

The goal of this project is to develop and implement a web platform dedicated to real estate rentals for agencies. This solution allows users to search for and book properties, request viewings, discuss rental terms, and communicate via an instant messaging system. It also integrates an AI-powered property pricing feature, facilitating quick and reliable decision-making.

The platform's administration follows a hierarchical structure: the main administrator manages the entire system and can manage users and agencies, including blocking, unblocking, or deleting them. Each agency has a dedicated manager responsible for administering all activities related to properties, viewings, pricing discussions, and client communication. This distribution of roles ensures rigorous monitoring and optimal administration of all platform functionalities. The project was designed with UML and implemented with Laravel for the web application, MySQL for data storage, and Python and Flask for artificial intelligence via API.